Table of Contents

Object Views

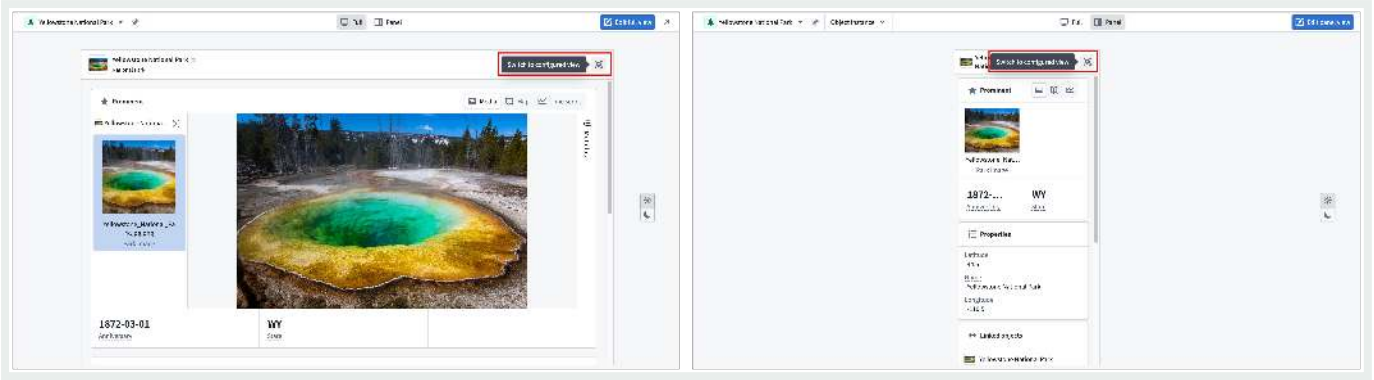
Object Views are reusable representations of object data. They provide a central hub for all information related to an object and include key information about the object, including property data, object links, and related applications.

Standard and configured Object Views

There are two types of object views:

1. **Standard Object Views:** Standardized, out-of-the-box representations that automatically reflect an object type's configuration. Standard Object Views are available for all object types and provide a consistent way to view object data without any configuration. Learn more about standard Object Views.
2. **Configured Object Views:** Fully customizable representations built using Workshop that you can configure to provide contextualized experiences for specific workflows. When a configured Object View is created, it becomes the default view, though users can always switch back to the standard Object View.

Standard Object Views exist alongside configured Object Views as a first-class viewing option. While standard Object Views display by default when no configured Object View is created, they remain accessible even after a configured Object View is built. Users can toggle between standard and configured Object Views at any time based on their needs.



Object View form factors

Both standard and configured Object Views are available in two form factors to accommodate different levels of detail. These different form factors offer flexibility in how object data appears across different workflows.

1. **Full Object Views:** A comprehensive overview of an object, representing an in-depth display of all related information.
2. **Panel Object Views:** Intended for integration with other applications and should focus on displaying the most critical data for a specific workflow.

Example: Configured Patient Object View

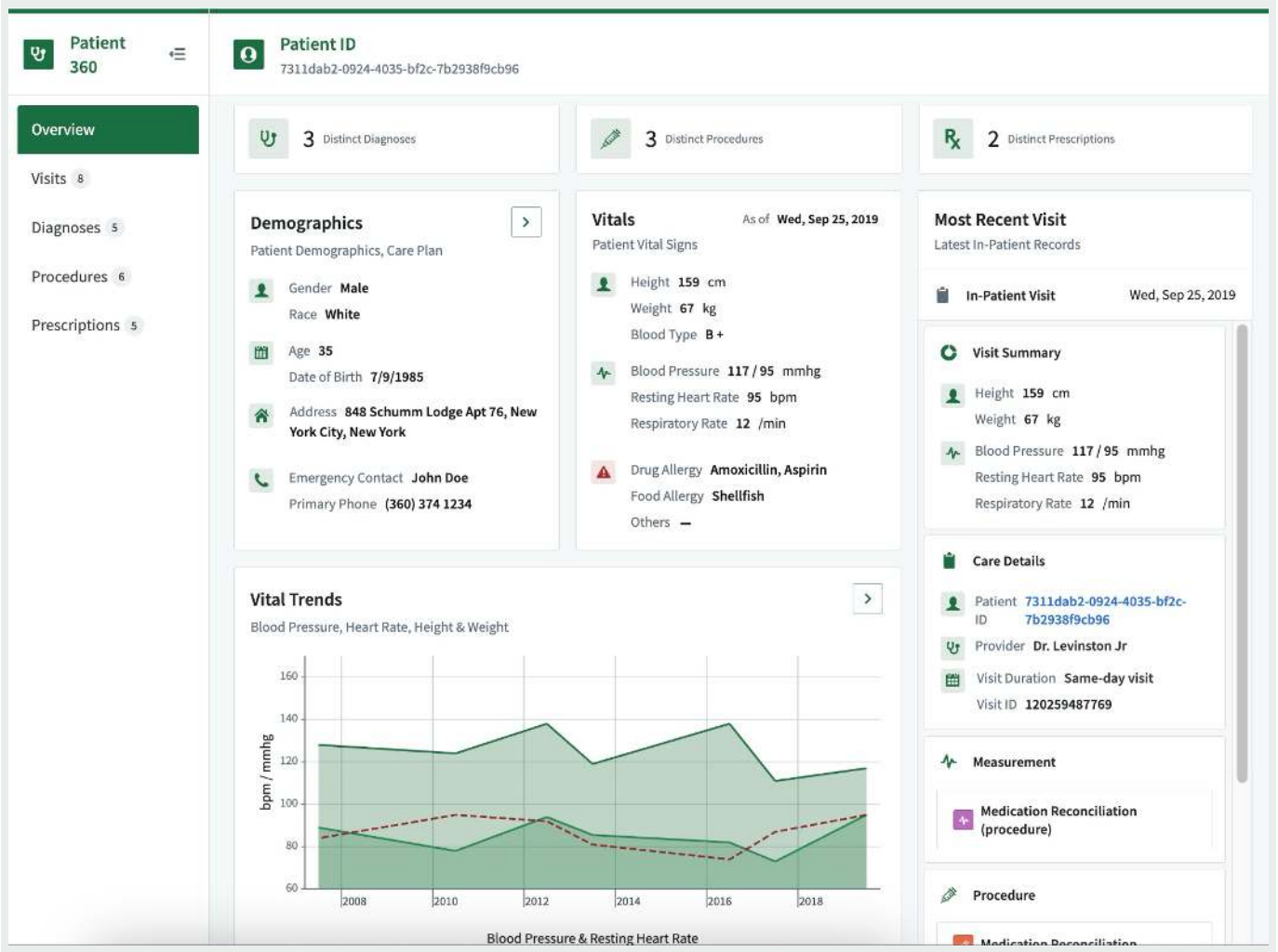
A configured full Object View for a `Patient` object might include:

- **Core demographics, vitals, and care details:** A comprehensive snapshot of the patient's basic information, health status, and ongoing care plans.
- **Linked procedures, prescriptions, and diagnoses:** A consolidated list of medical interventions, medications prescribed, and confirmed diagnoses, providing a holistic view of the patient's medical history.
- **Analytical trends from historical in-patient records:** Insights derived from past hospital stays, highlighting patterns and trends in the patient's health over time.

This configured full Object View could serve as an exhaustive resource for all relevant in patient, facilitating better-informed healthcare decisions and personalized care planning.

API Reference ↗

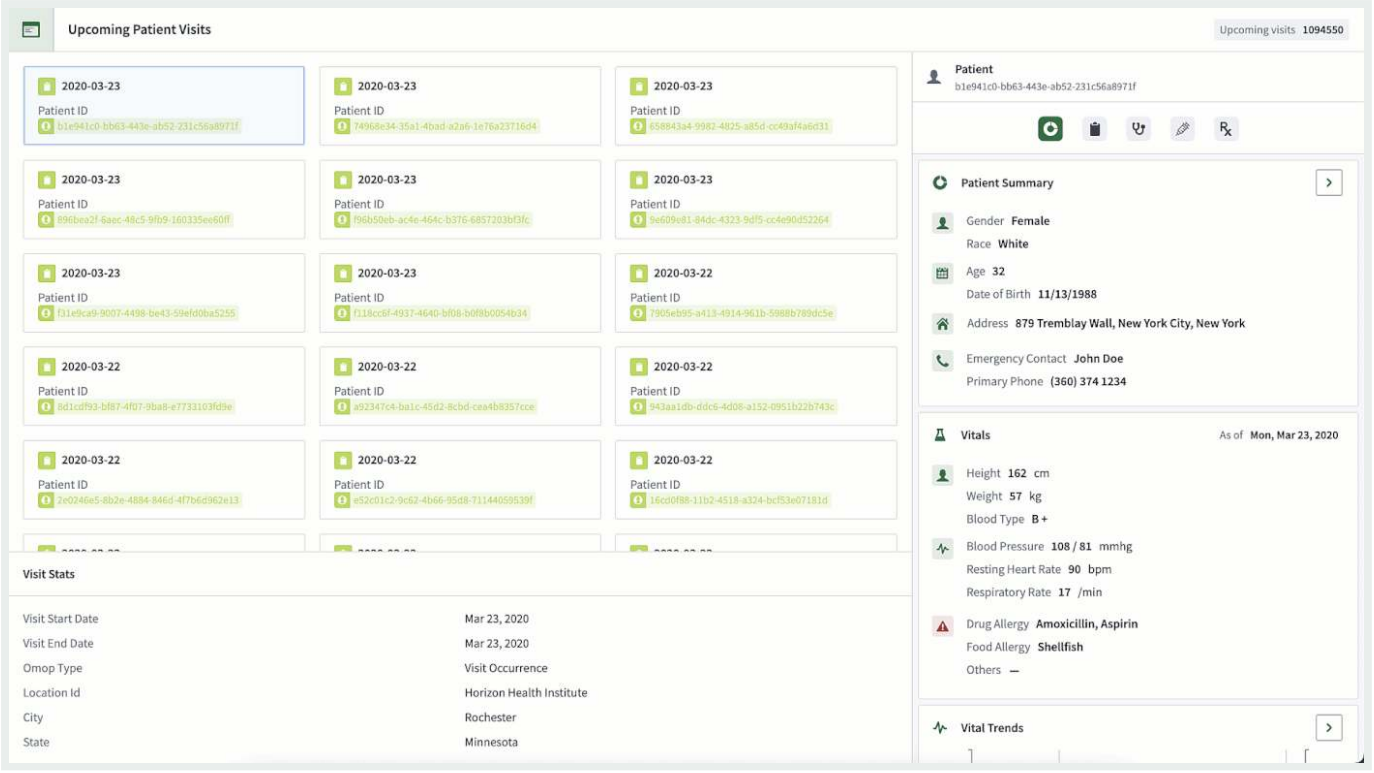
Send feedback



The configured panel Object View for the same `Patient` object may only show the demographic and vital information, so when it appears in other applications it provides users with easy access to the most critical data for their workflow.

API Reference ↗

Send feedback



Learn more about object types and the concepts behind Ontology-based data modeling.

PREVIOUS
← Object Monitors [Sunset] / Error reference

NEXT
Standard Object Views →

API Reference ↗

Send feedback

Object Views

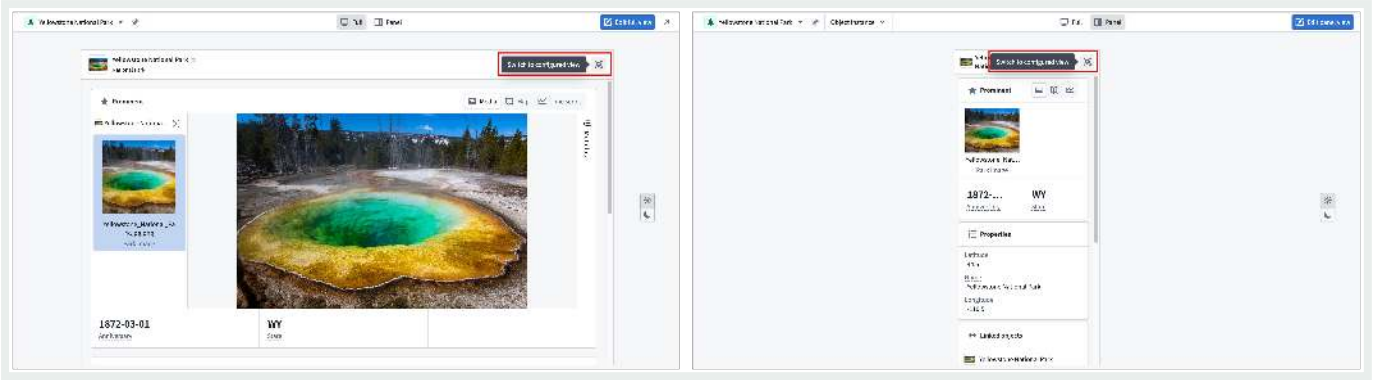
Object Views are reusable representations of object data. They provide a central hub for all information related to an object and include key information about the object, including property data, object links, and related applications.

Standard and configured Object Views

There are two types of object views:

1. **Standard Object Views:** Standardized, out-of-the-box representations that automatically reflect an object type's configuration. Standard Object Views are available for all object types and provide a consistent way to view object data without any configuration. Learn more about standard Object Views.
2. **Configured Object Views:** Fully customizable representations built using Workshop that you can configure to provide contextualized experiences for specific workflows. When a configured Object View is created, it becomes the default view, though users can always switch back to the standard Object View.

Standard Object Views exist alongside configured Object Views as a first-class viewing option. While standard Object Views display by default when no configured Object View is created, they remain accessible even after a configured Object View is built. Users can toggle between standard and configured Object Views at any time based on their needs.



Object View form factors

Both standard and configured Object Views are available in two form factors to accommodate different levels of detail. These different form factors offer flexibility in how object data appears across different workflows.

1. **Full Object Views:** A comprehensive overview of an object, representing an in-depth display of all related information.
2. **Panel Object Views:** Intended for integration with other applications and should focus on displaying the most critical data for a specific workflow.

Example: Configured Patient Object View

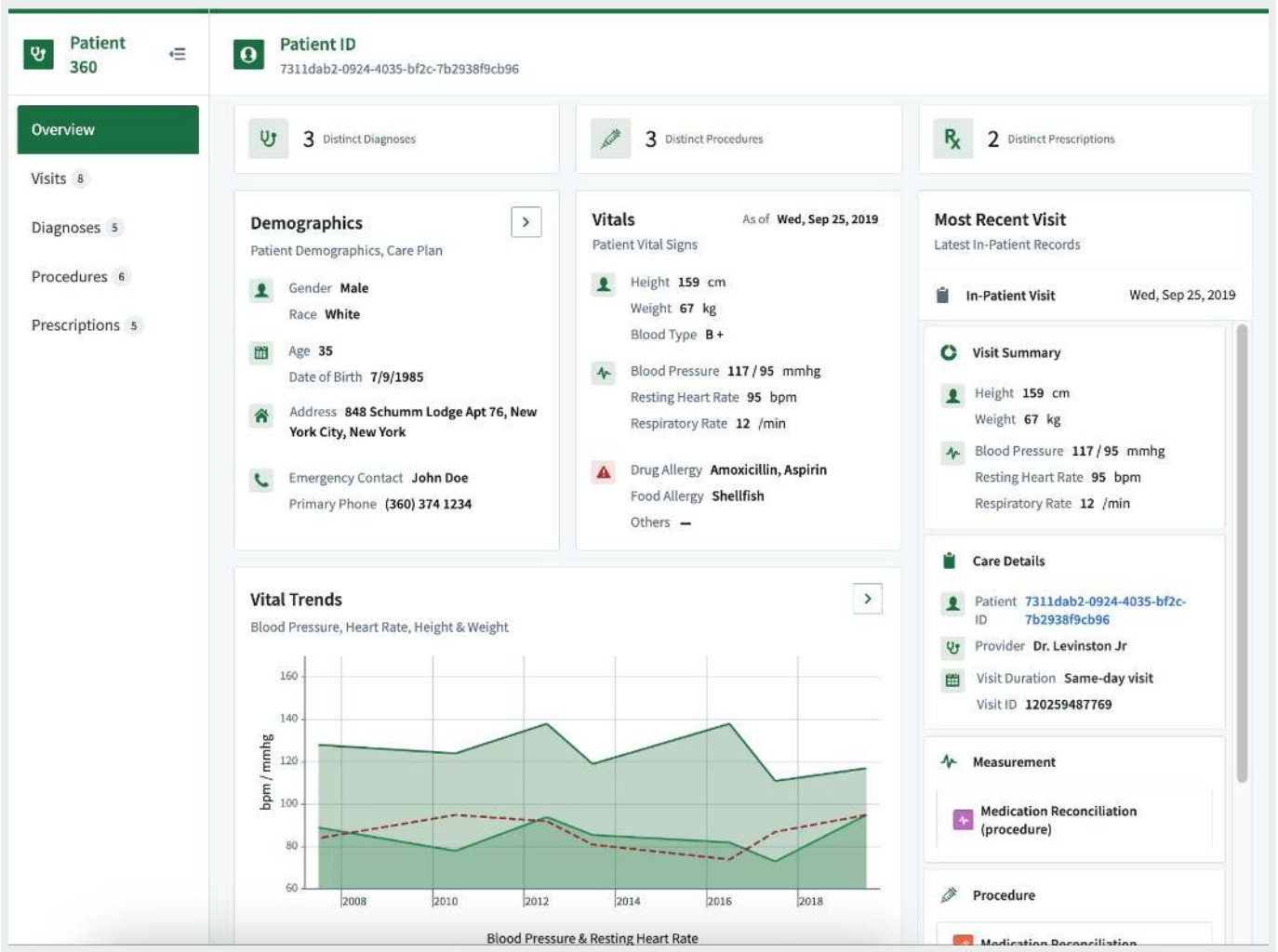
A configured full Object View for a `Patient` object might include:

- **Core demographics, vitals, and care details:** A comprehensive snapshot of the patient's basic information, health status, and ongoing care plans.
- **Linked procedures, prescriptions, and diagnoses:** A consolidated list of medical interventions, medications prescribed, and confirmed diagnoses, providing a holistic view of the patient's medical history.
- **Analytical trends from historical in-patient records:** Insights derived from past hospital stays, highlighting patterns and trends in the patient's health over time.

This configured full Object View could serve as an exhaustive resource for all relevant in patient, facilitating better-informed healthcare decisions and personalized care planning.

API Reference ↗

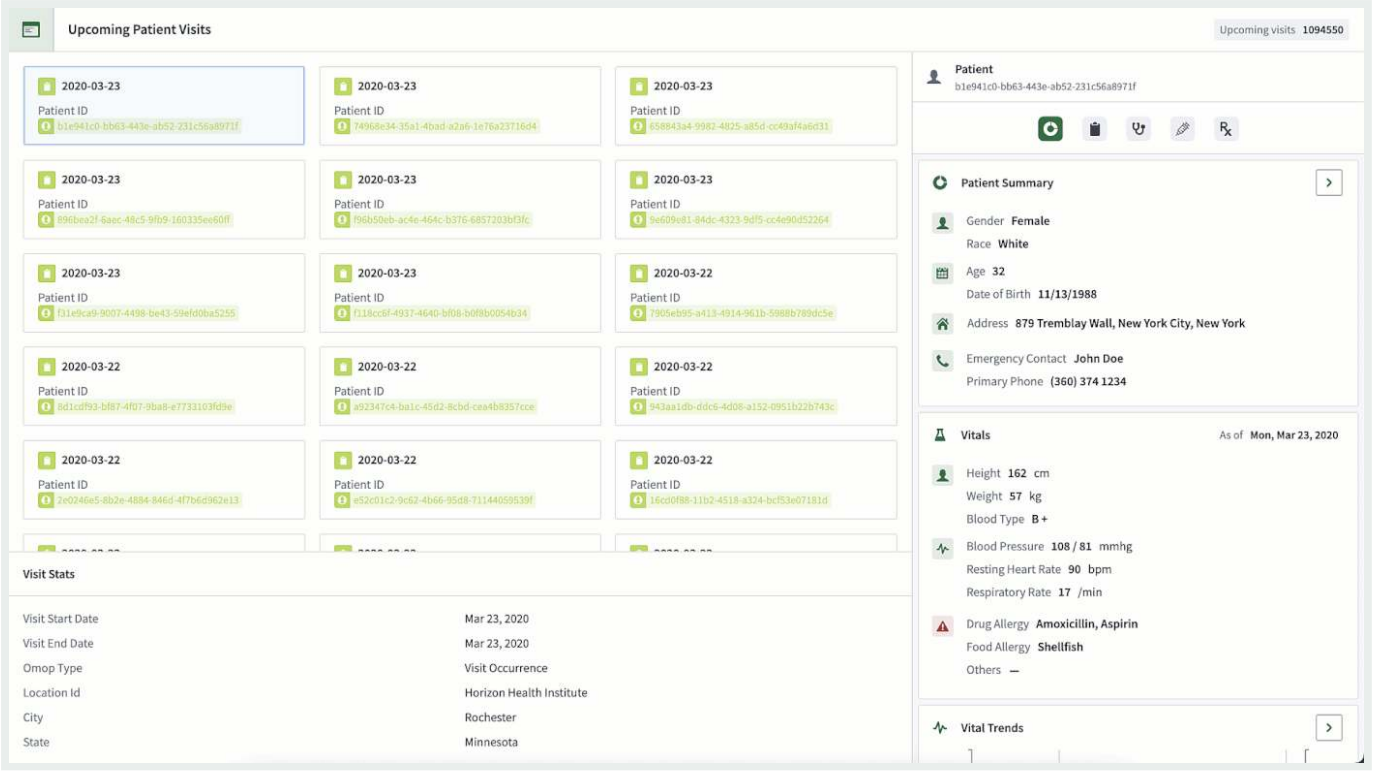
Send feedback



The configured panel Object View for the same `Patient` object may only show the demographic and vital information, so when it appears in other applications it provides users with easy access to the most critical data for their workflow.

API Reference ↗

Send feedback



Learn more about object types and the concepts behind Ontology-based data modeling.

PREVIOUS
← Object Monitors [Sunset] / Error reference

NEXT
Standard Object Views →

API Reference ↗

Send feedback

Standard Object Views

When you create and configure an object type in your Ontology, Foundry automatically creates a standard Object View to provide a standardized representation of all its objects, ensuring other users have a holistic understanding of its schema and links. The standard Object View matches the object type's configuration by spotlighting prominent properties in either a dedicated table or in other visual formats if the property's base type is a time series, media reference, or geospatial property. Normal properties are displayed in a regular table, and hidden properties are not visible.

While standard Object Views are available for all object types, you can create configured views to provide a contextualized and curated Object View experience for any object type based on its function in your Ontology. When you create a configured Object View, Foundry makes it available as the default view for a user; however, users can choose to select the standard view to see object data in its standard format.

Prominent property displays

Foundry surfaces prominent properties at the top of the standard Object View to provide immediate context about an object's most important information.

Display behavior

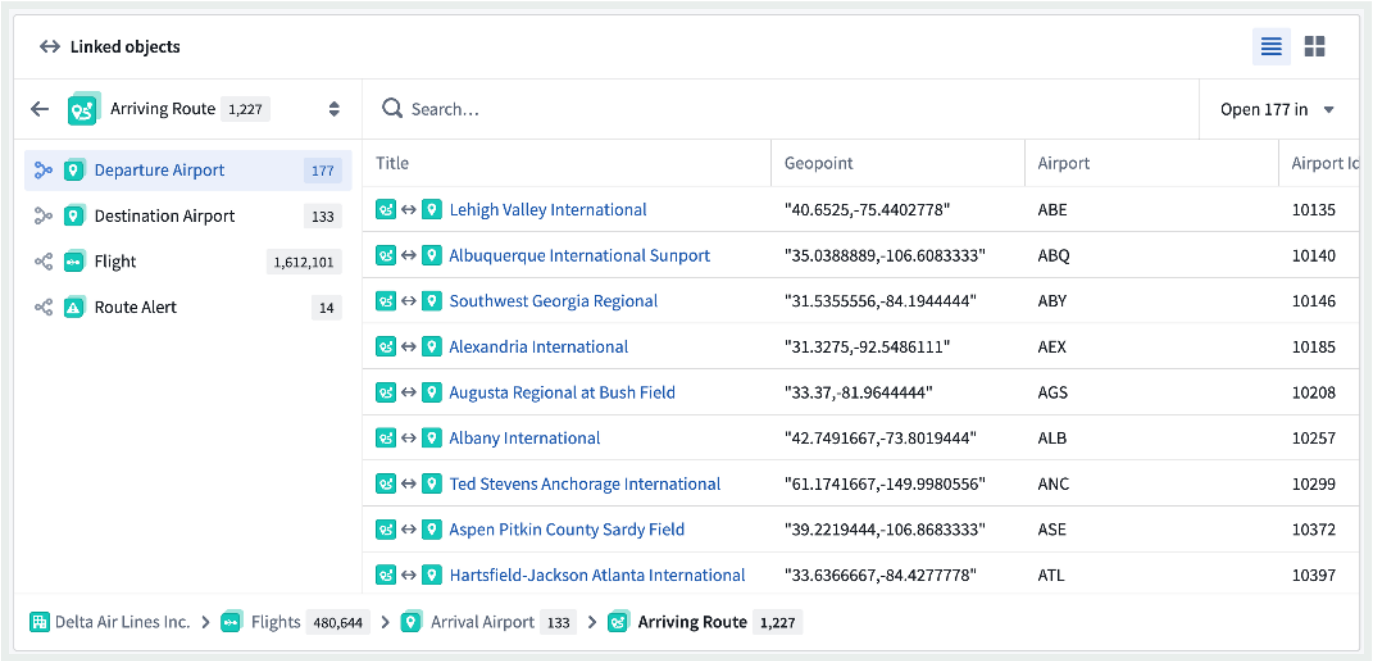
Properties marked as prominent receive enhanced visual treatment based on their type:

- **properties:** Rendered with a dedicated media viewer for viewing all supported media types.
- **ies properties:** Displayed as interactive charts showing t

- **Geospatial properties:** Objects with prominent geohash, geoshape, or geotemporal series reference (GTSR) properties will render on a Map. Time series properties that represent an object's latitude and longitude over time are also displayed on a Map.
- **Other property types:** All other prominent properties are displayed using a larger, card-style format elevated above a table displaying the remaining standard properties.

Linked objects component

The **Linked objects** component enables you to traverse across linked objects directly within the standard Object View.



Use the **Linked objects** component of a standard Object View to:

- View linked objects grouped by link type.
- Preview properties of linked objects inline without leaving the current view.
- Open a subset of linked objects in a new tab for further exploration.
- Preview a selected linked object in the side panel of the standard Object View.

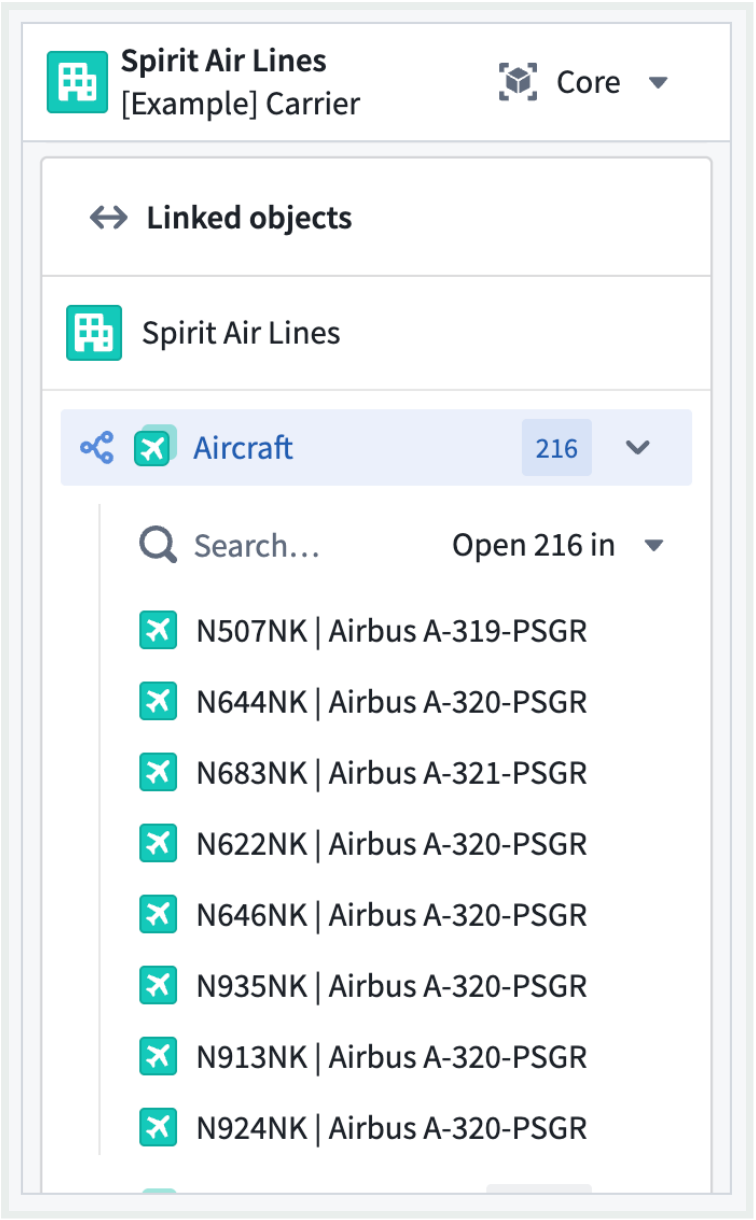
Panel standard Object View display behavior

API Reference ↗

You can also interact with all of a standard Object View's functionality

Send feedback

factor.



PREVIOUS
← Overview

NEXT
Manage configured Object View versions →

Object View versioning

Saved edits to an Object View will be stored as a new version. A version can contain several changes, such as adding, editing, and deleting tabs. Versioning enables you to:

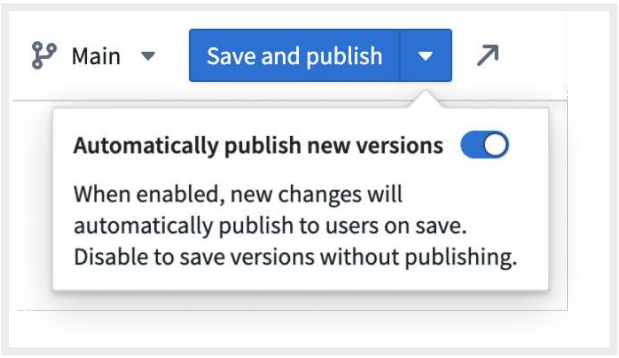
- Iterate on Object Views safely by saving your changes incrementally
- Control which version is published and visible to users
- Republish older versions in case newer versions contain errors
- Collaborate with other editors by adding descriptions to versions

There are separate versions for each Workshop module included in the Object View. Both version numbers appear in the Object View editor header.

Save new versions

After editing the Object View, you can save changes by selecting the **Save** button in the Object View editor header. If **Automatically publish new versions** is enabled, this button will display **Save and publish** instead, and will publish both tab changes and any changes to the current Workshop module. Automatic publishing is enabled by default.

Disabling **Automatically publish new versions** will display two separate buttons for saving and publishing. The **Save** button will save both tab and workshop module changes, and the **Publish** button will publish both of these changes to the user.



As you edit the Workshop module, it will be periodically auto-saved like any other Workshop module, but these changes will not be visible to users until the Object View is published.

> If your Object View is actively used by many users or there are multiple editors collaborating on the view, we recommend disabling automatic publishing. This improves collaboration and lowers the chances of breaking user workflows.

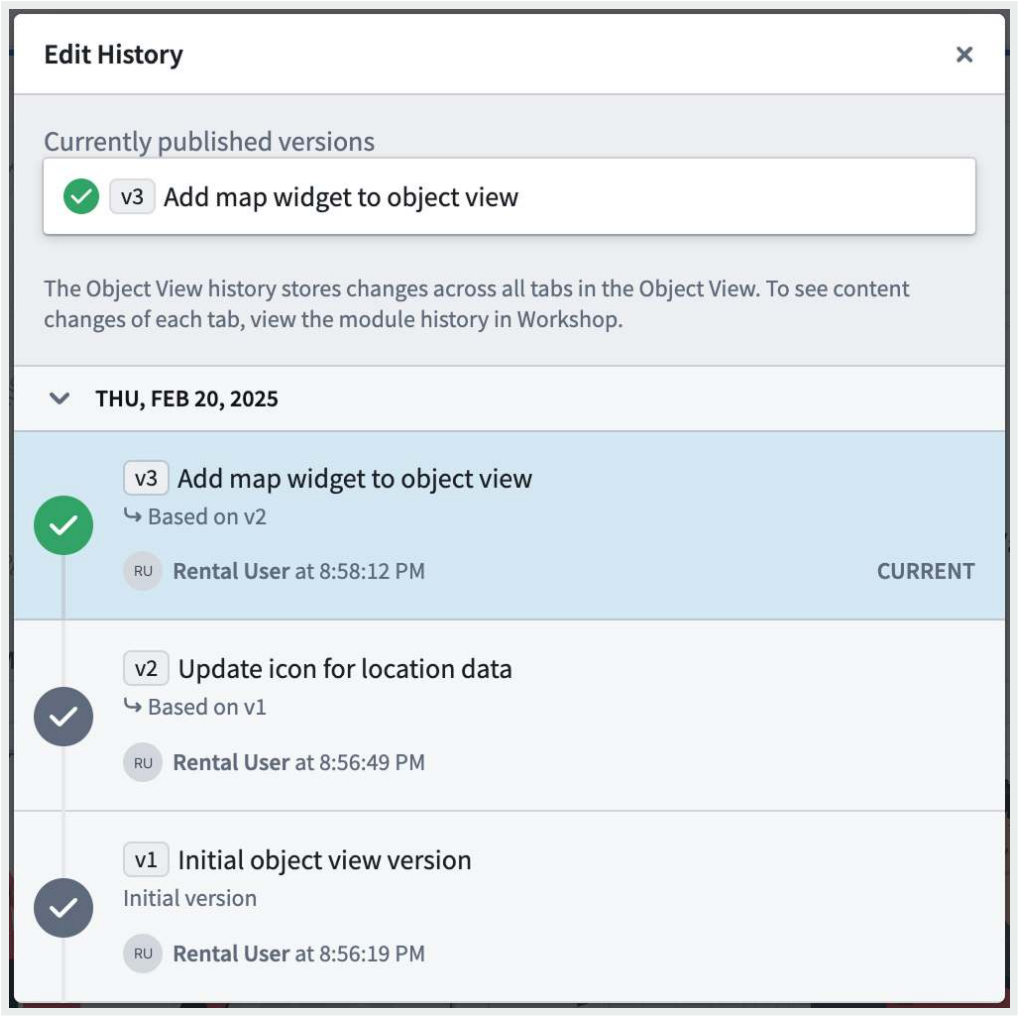
Access prior versions

To view a list of prior versions of the Object View, select the blue Object View version number in the Object View editor header. This will open a dialog displaying the date, author, and description of all prior versions. From this dialog, all versions can be previewed.

- The currently published version will be marked with a green checkmark.
- Prior published versions will show a grey checkmark.
- Versions that were never published will not have a checkmark.

API Reference ↗

Send feedback



PREVIOUS
← Standard Object Views

NEXT
Configured Object View overview →

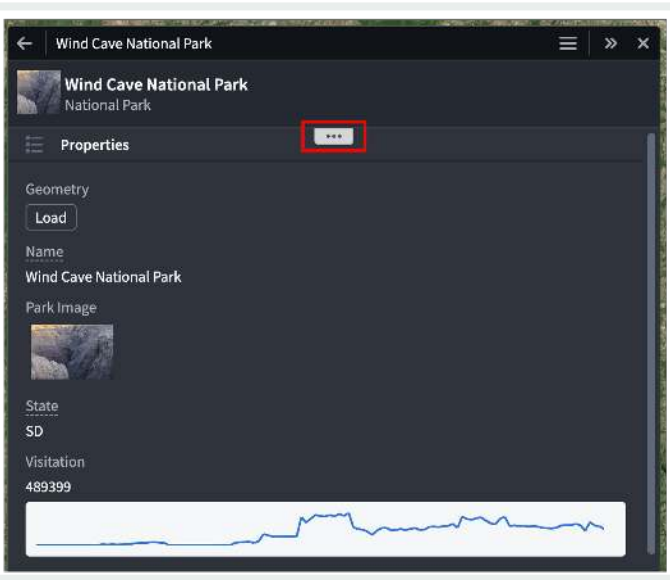
API Reference ↗

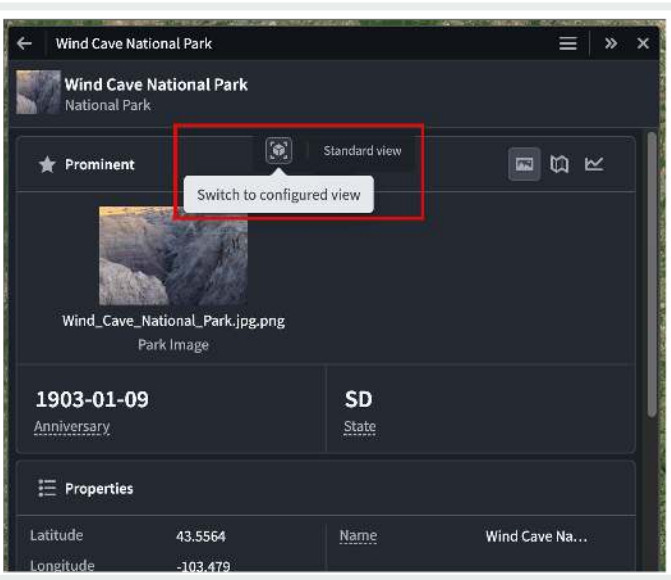
Send feedback

Configured Object View overview

Configured Object Views are customizable, reusable representations of object data. You can build both full and panel Object Views for an object type through Workshop modules, enabling flexibility in how objects are represented across the platform.

Foundry creates a standard Object View for all object types by default. When you create a configured Object View, it becomes the default view for users, though they can switch back to the standard Object View through a toggle button packaged with the Object View. Additionally, users can hover over the ellipsis drawer icon in an Object View rendered in Palantir applications that use their own custom header, such as Gaia or Vertex, to toggle between standard and configured views.





The ability to toggle between standard and configured Object Views is not yet
rkshop.

API Reference ↗

Default configurations

Default configured Object Views are automatically created for each object type. The default full Object View contains a list of prominent properties, or all non-hidden properties if none are prominent, and a list of the object's links. The default panel contains the same list of properties. The default views will dynamically update to reflect changes made to the object type, such as new properties or property renames, but once an Object View is edited it becomes user-managed and all further updates must be made manually.

Permissions

The permissions required to edit the Object View for an object type depend on whether the object type uses Ontology roles:

- If the object type does not use Ontology roles, a user must have the `Object View Admin` application permission in Control Panel, as well as the `Editor` role on any of the object type's input datasources.
- If the object type uses Ontology roles, the user only requires the `Ontology Editor` role on the object type.

Unless you manually convert the Workshop module for an Object View tab to a standalone module through legacy configuration options, the Workshop module's permissions will be managed by the object type. This ensures that permissions between the module and the object type are kept aligned, so users with permission to edit or view the object type will also be able to edit or view all modules inside the Object View.

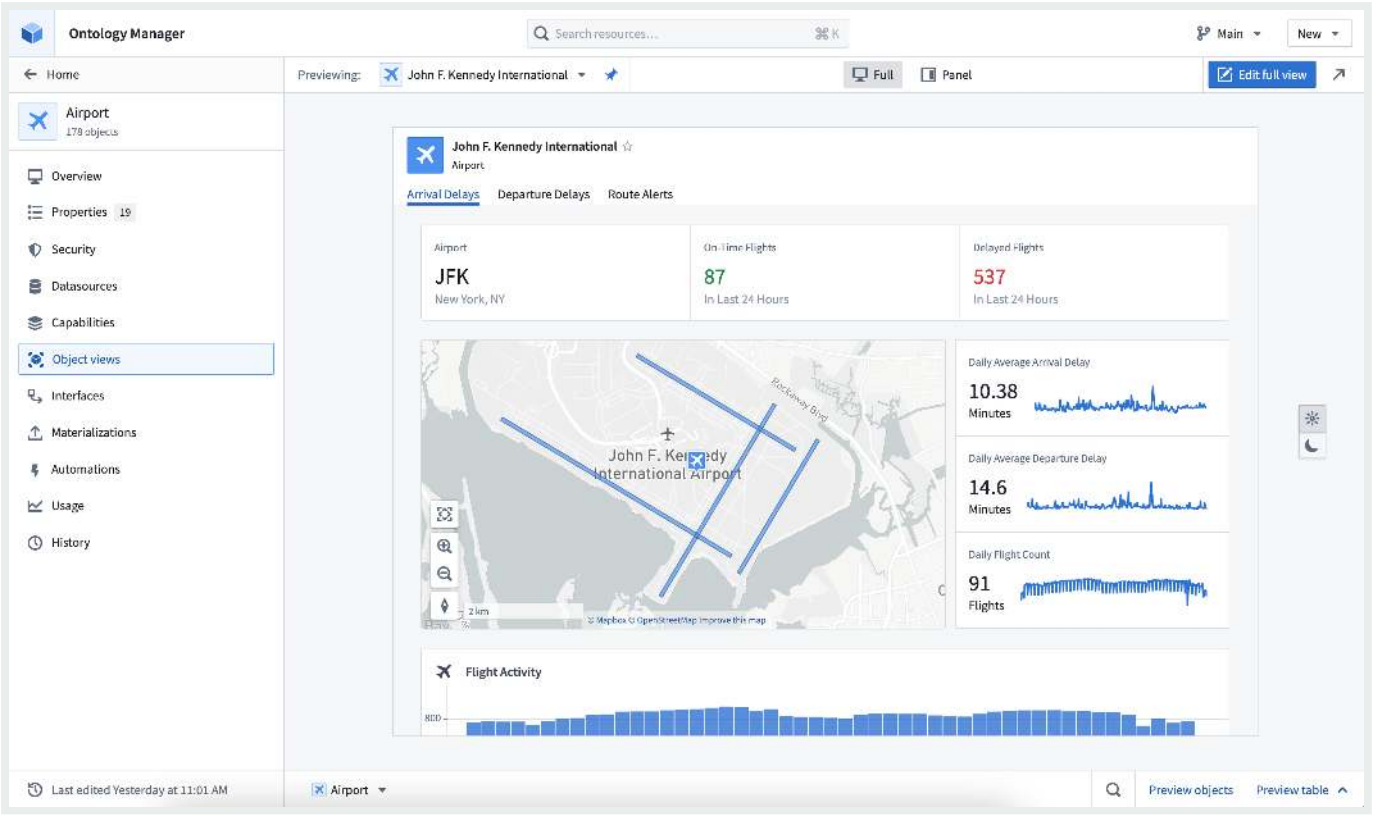
Edit configured Object Views

There are many ways to access configured Object View configuration.

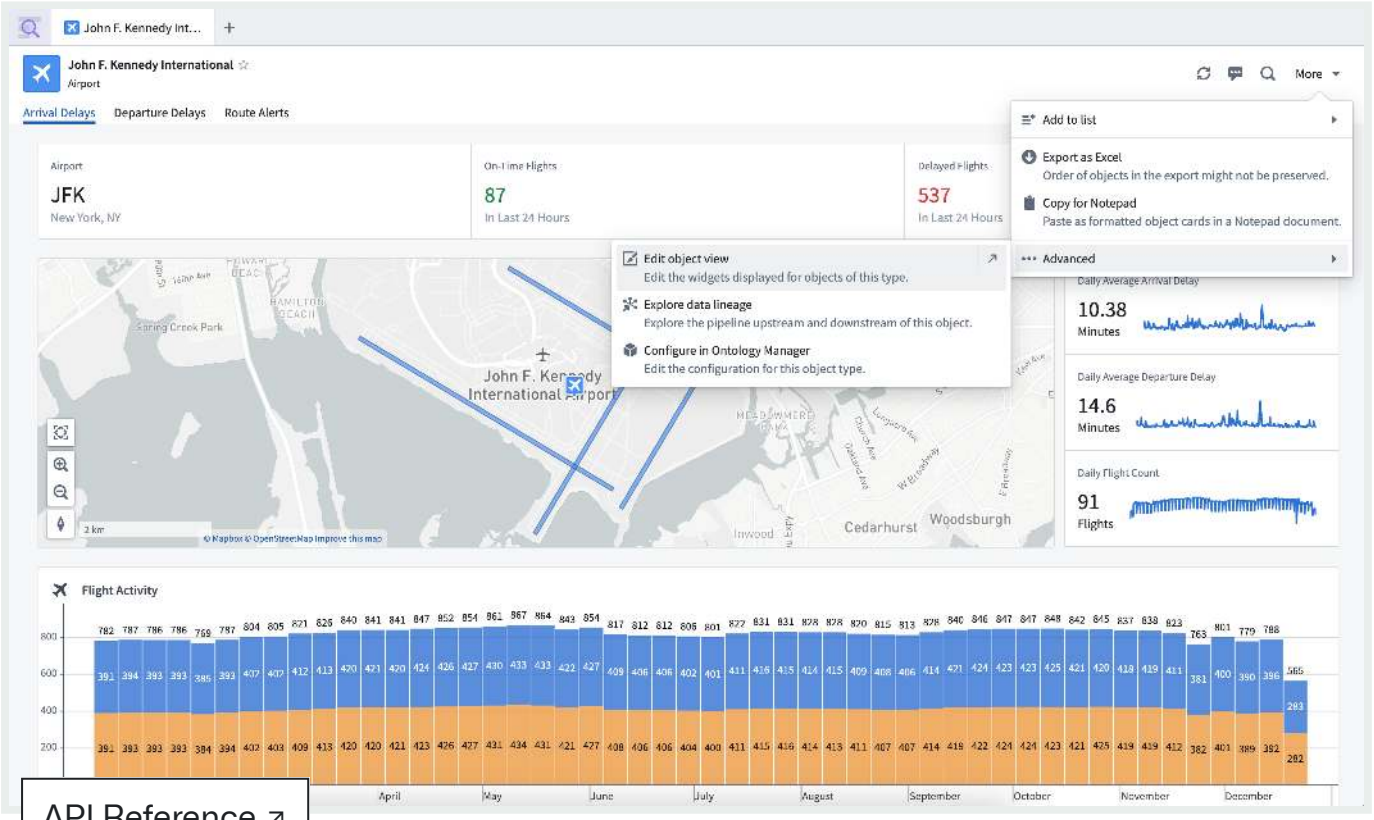
The Object Views for an object type can be previewed in the **Object views** tab in Ontology Manager. In the header, you can select and pin a default display object to preview. You can a and panel form factors, and test how the Object View appears in light a ng the configured Object View can be accessed using the **Edit** option in the right side of the header.

API Reference ↗

Send feedback



In Object Explorer, an object type's configured Object View can be accessed when viewing an object by selecting **More > Advanced > Edit object view**.



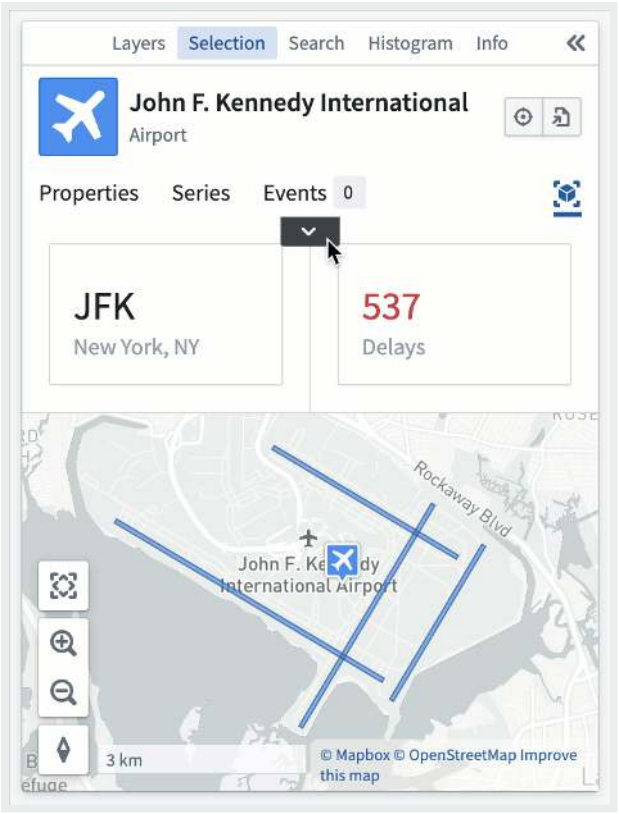
API Reference ↗

When viewing a panel Object View within an application, configuratio

Send feedback

hovering over the dropdown ellipsis and selecting **Edit**. The dropdow

users with permission to edit the Object View.



These edit entry points lead to the configured Object View editor, where the Object View tabs can be managed, and content can be edited with all the standard features of a Workshop module. Once published, edits will apply to all objects of the object type. For more editing information, refer to the full Object View configuration and panel Object View configuration.

PREVIOUS
← Manage configured Object View versions

NEXT
Branching object views →

Branching object views

Overview

Object Views integrates with Global Branching to enable safe, isolated development of object view configurations. This documentation covers how to work with object views on branches, including adding and modifying resources, cross-application compatibility, merge requirements, rebasing, and known limitations.

For general information on Global Branching concepts and workflows, refer to the Global Branching documentation.

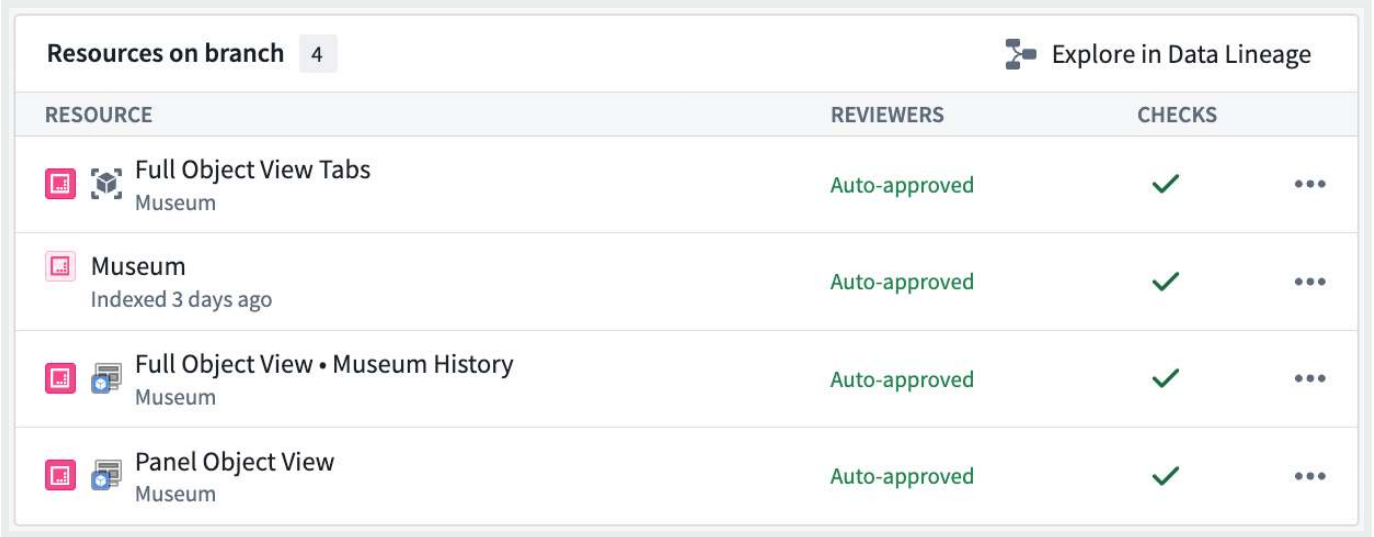
Adding, removing, and modifying resources

To add an object view to a branch, save an object view version while on a branch in the Object View editor.

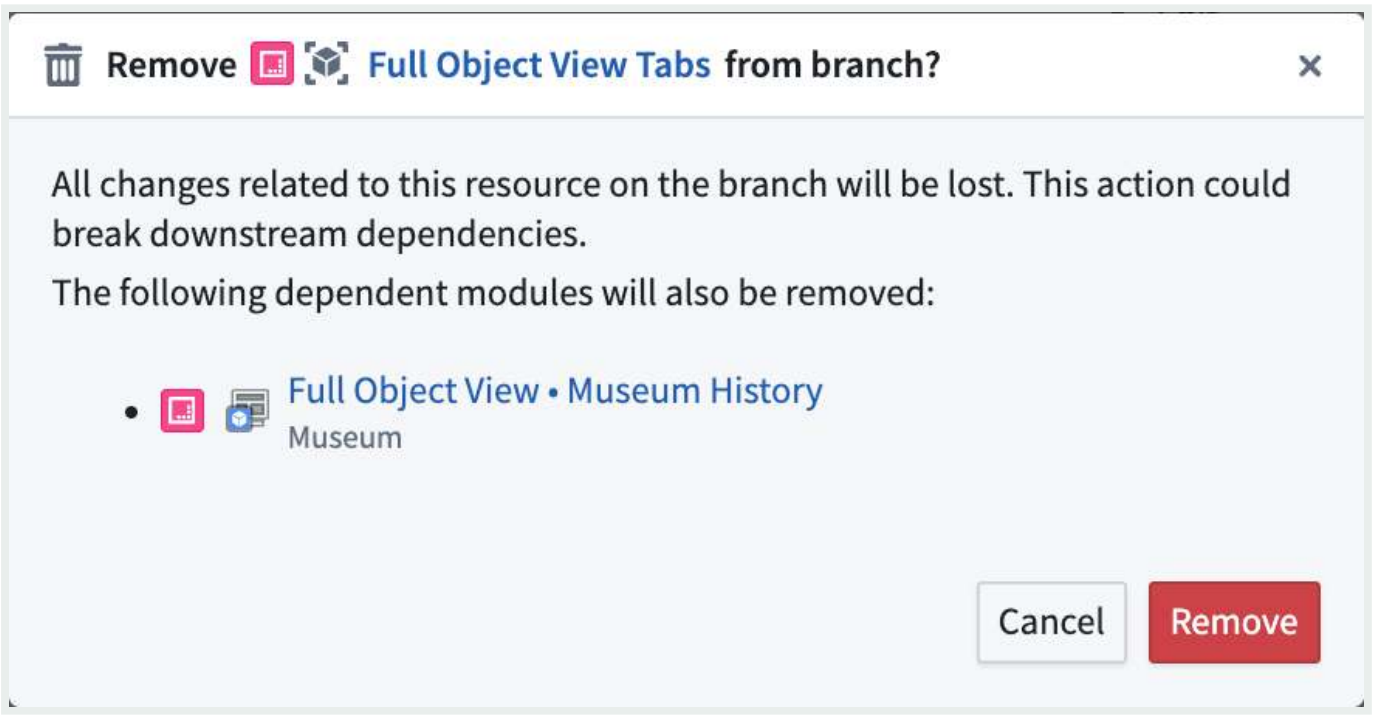
Object views support two types of branch-compatible resources:

- **OV-managed modules:** Capture Workshop content changes made to an object view on a branch. A separate OV-managed module is created for each full object view tab, the object instance panel, and the object set panel. All branch-aware features available in Workshop applications are also available inside object views.
- **Full object view tabs:** Capture structural changes to object view tabs, including additions, deletions, renames, profile changes, and visibility condition modifications.

The image below shows different resources on a branch for the `Mu` object t… from top to bottom: the full object view tabs resource, the `Museum` object t… **History** tab of the full object view, and the panel object view.



You can remove any object view resource from a branch individually using the branch taskbar. Removing a full object view tabs resource automatically removes all of its associated tabs from the branch.



Cross-application compatibility

Object views can be edited for the latest state of the ontology on a branch, including object types and action types created on a branch. The Object View widget can also be used to e bject view inside a standalone Workshop application. An object view can be previewed on a branch within the **Object views** tab in Ontolog

API Reference ↗

Send feedback

Merge requirements

Deployability checks

Before deploying an object view to `main` from a branch, the following deployability checks must succeed:

- **User has publish permissions:** Permissions to edit the object view is required. This is the same permission check that is done when publishing a new object view version on `main`.
- **Object view must be rebased with main:** Before merging, rebase each object view resource on the branch with the latest changes on `main`.
- **No Legacy fields modified:** Verifies that no features unsupported by the new Object View editor have been modified on the branch. This check should always pass. If it fails, contact Palantir Support.

Approvals checks

Object views use approvals to determine whether the user making changes on a branch has the permissions required to merge those changes. The specific permissions required depend on the permission model of the object type that the object view is linked to.

Object type uses datasource-derived permissions

When the object type uses datasource-derived permissions, the contributor or an approving reviewer must have:

- `View` access on the object type and `Editor` access on **all** of its backing datasources.
- The `Object View Admin` application permission in Control Panel, which is required to publish object views in Object Explorer. This is the same application permission required when publishing a new object view version on `main`.

API Reference ↗

Datasource permissions are stricter at merge time

The datasource permission requirement above is stricter than t

Send feedback

permissions check used outside of branching. Editing an object view on `main` only

requires the `Editor` role on **any** of the object type's backing datasources. Merging changes from a branch requires the `Editor` role on **every** backing datasource.

Object type uses ontology roles or project-based permissions

When the object type uses ontology roles or project-based permissions, the contributor or an approving reviewer only needs edit access on the object type. This is typically granted through the `Ontology Editor` role under ontology roles, or through the `Editor` project role under project-based permissions. Edit access on the object type already covers publishing object views in Object Explorer, so no separate `Object View Admin` application permission is required.

Inherited project approval policies

Object views and the Workshop modules that make up object view tabs and panels are logical children of the parent object type. If the object type is protected and lives in a project with a project approval policy, that policy applies to the object view and its modules. Approval requests for both must satisfy the policy before the proposal can be merged.

> **Inherited protection from the parent object type**
> Inherited resource protection is under development and will be released soon. Once available, it will prevent direct edits to `main` for an object view or its modules whenever the parent object type is protected. Until then, an object view and the modules that make up its tabs and panels can still be edited directly on `main` even if the parent object type is protected.

Rebasing and conflict resolution

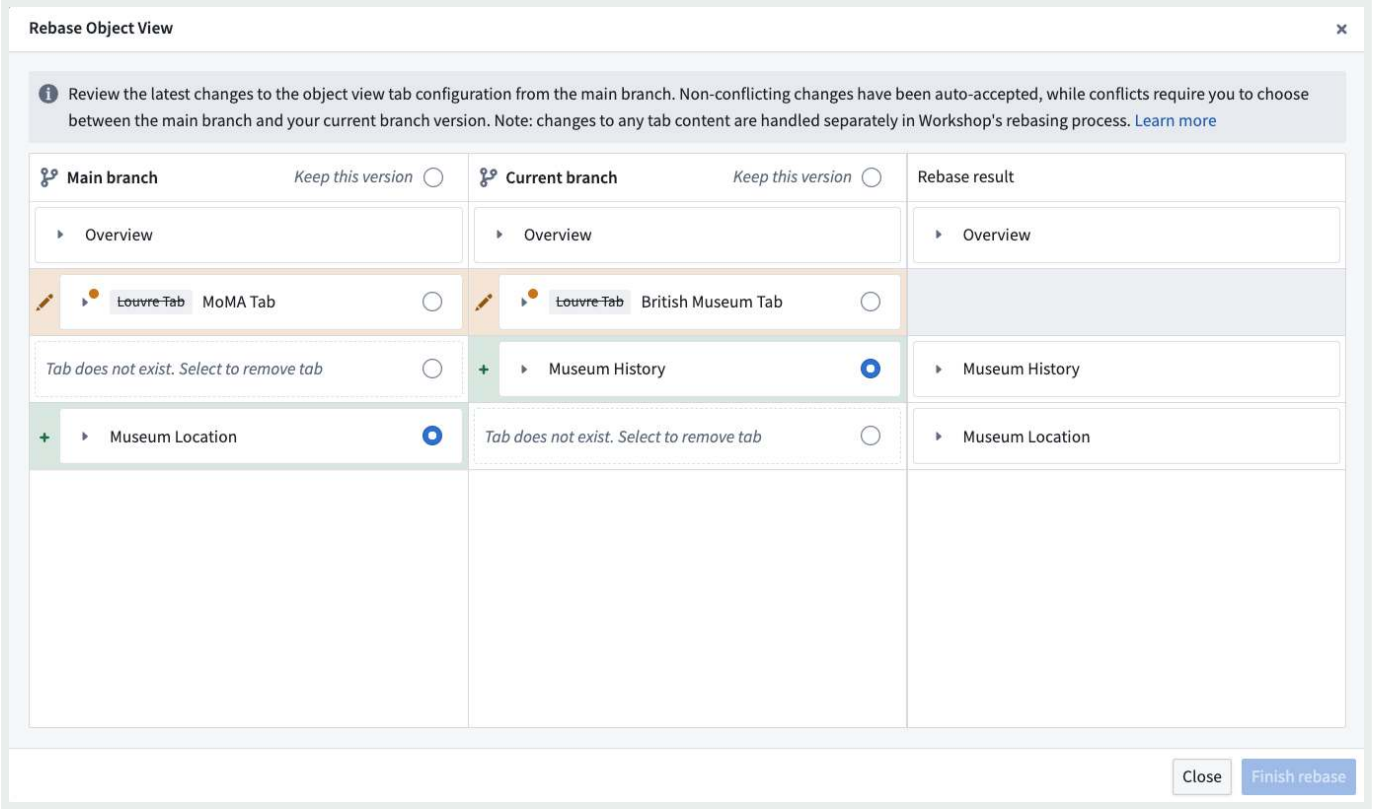
Each object view resource on a branch must be rebased with main before being deployed. Object view module resources can be rebased using Workshop rebasing. Tab configuration changes on a full object view tabs resource must be rebased separately from tab content c sing is required for full object view, a notification message will appear below the object view section in the Object View editor.

The object view rebase dialog displays three columns:

- The main branch state
- Your branch state
- The proposed rebase result

Non-conflicting changes between `main` and your branch are automatically accepted and included in the result. If there is a conflict, you must choose whether to keep the version from `main` or from your branch for each affected tab. The result column shows a preview of the final state after rebasing. You can modify any auto-accepted changes before completing the rebase.

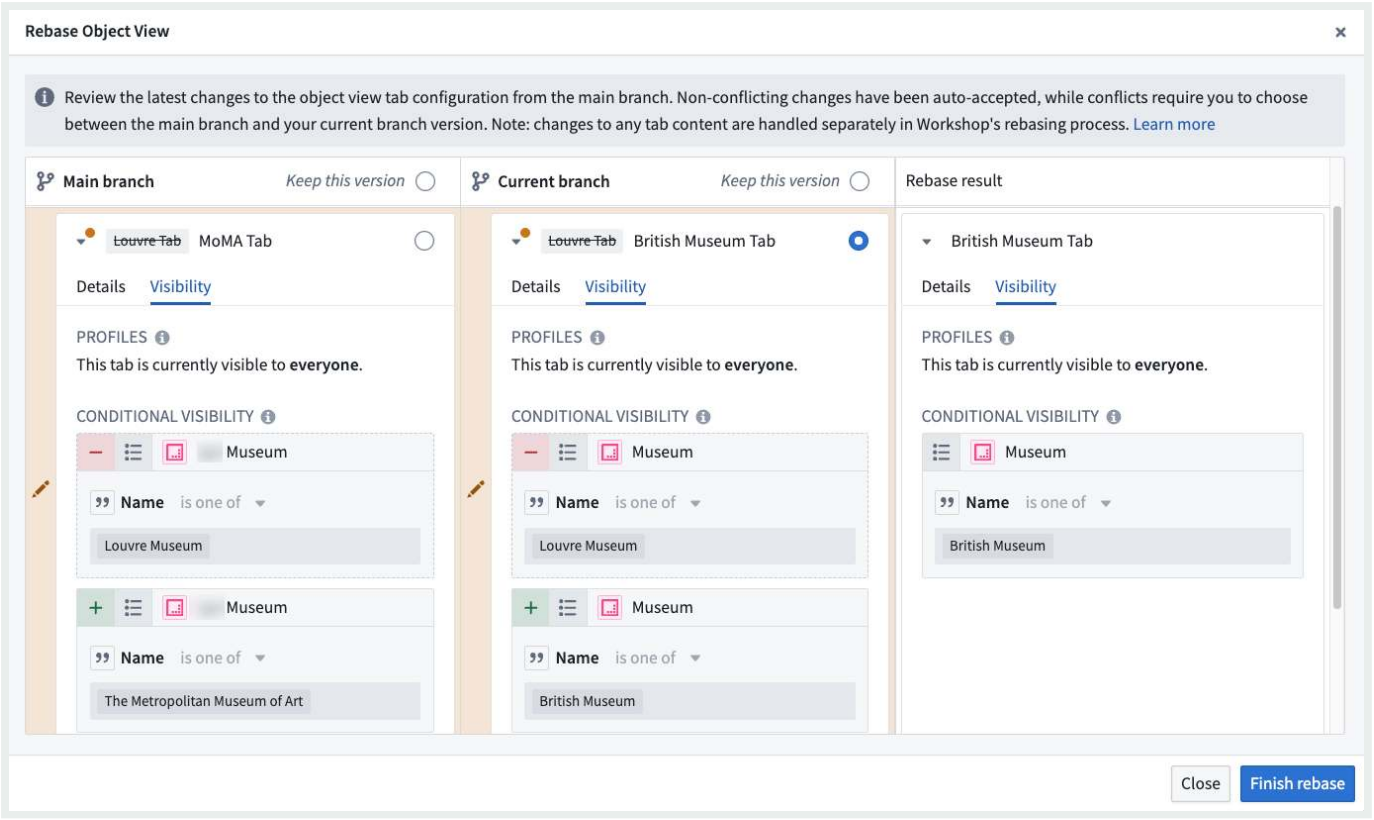
Below is an example of the object view rebase dialog. The example demonstrates two non-conflicting changes that have been auto-accepted. A conflict was detected between the two edits of the `Louvre` tab.



To resolve the conflict, choose either the branch or `main` version. This enables a successful rebase. You can expand the conflicting row to view visibility details.

API Reference ↗

Send feedback



Known limitations

Legacy object view tabs cannot be edited on a branch.

← PREVIOUS
Configured Object View overview

NEXT
Full Object Views / Use full Object Views in the platform →

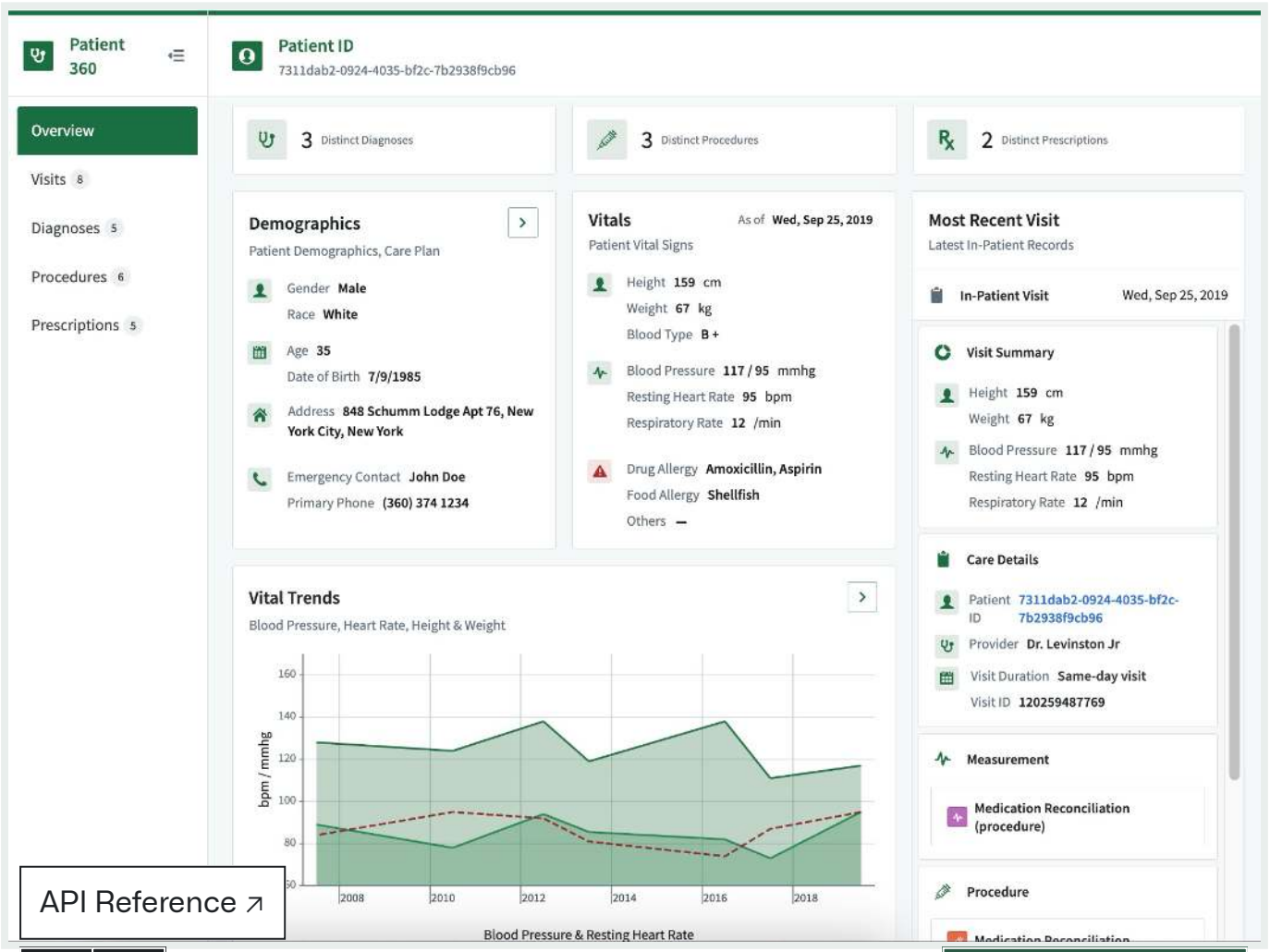
API Reference ↗

Send feedback

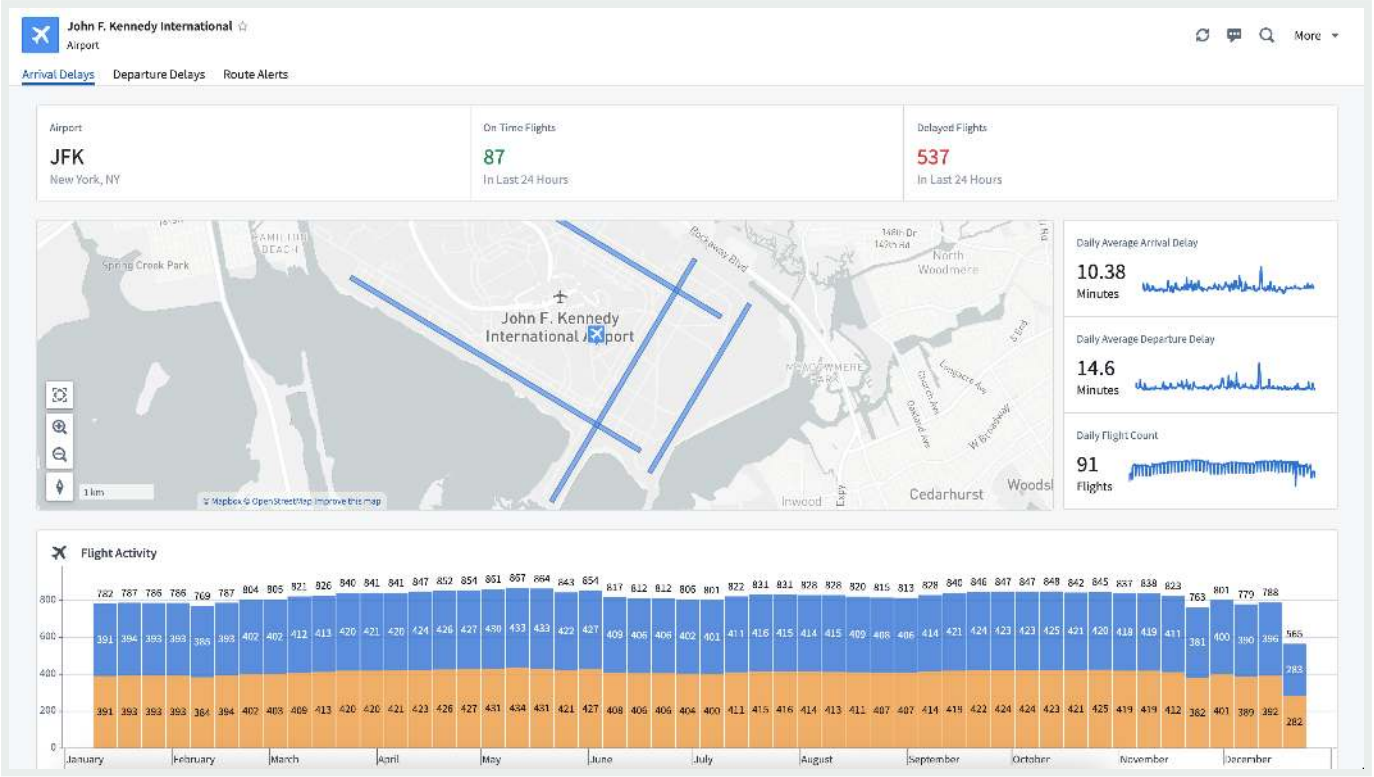
Use full Object Views

Full Object Views provide comprehensive displays of an object's data. They are the primary view for the object in-platform and can be accessed within platform applications or embedded into custom Workshop applications.

An example of a full `Patient` Object View:

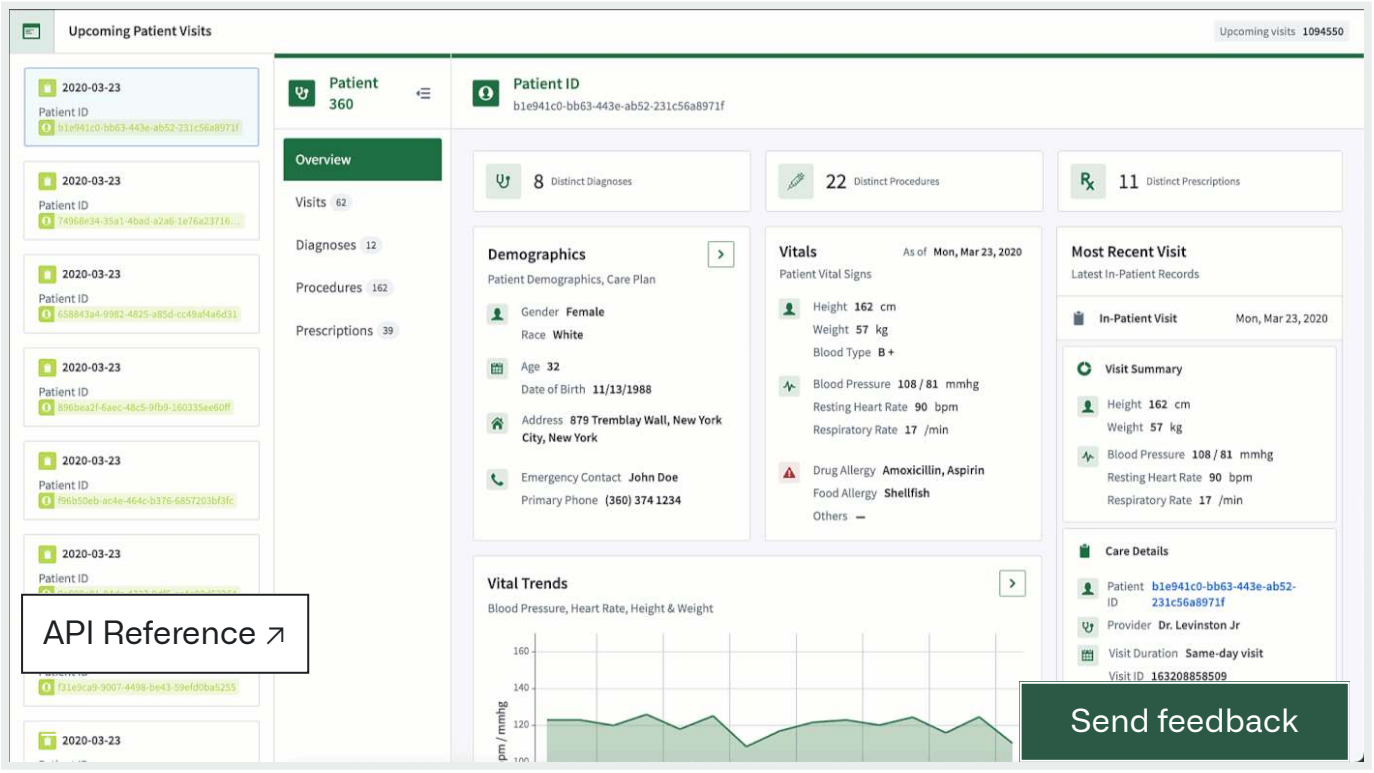


An example of a full `Rental` Object View:



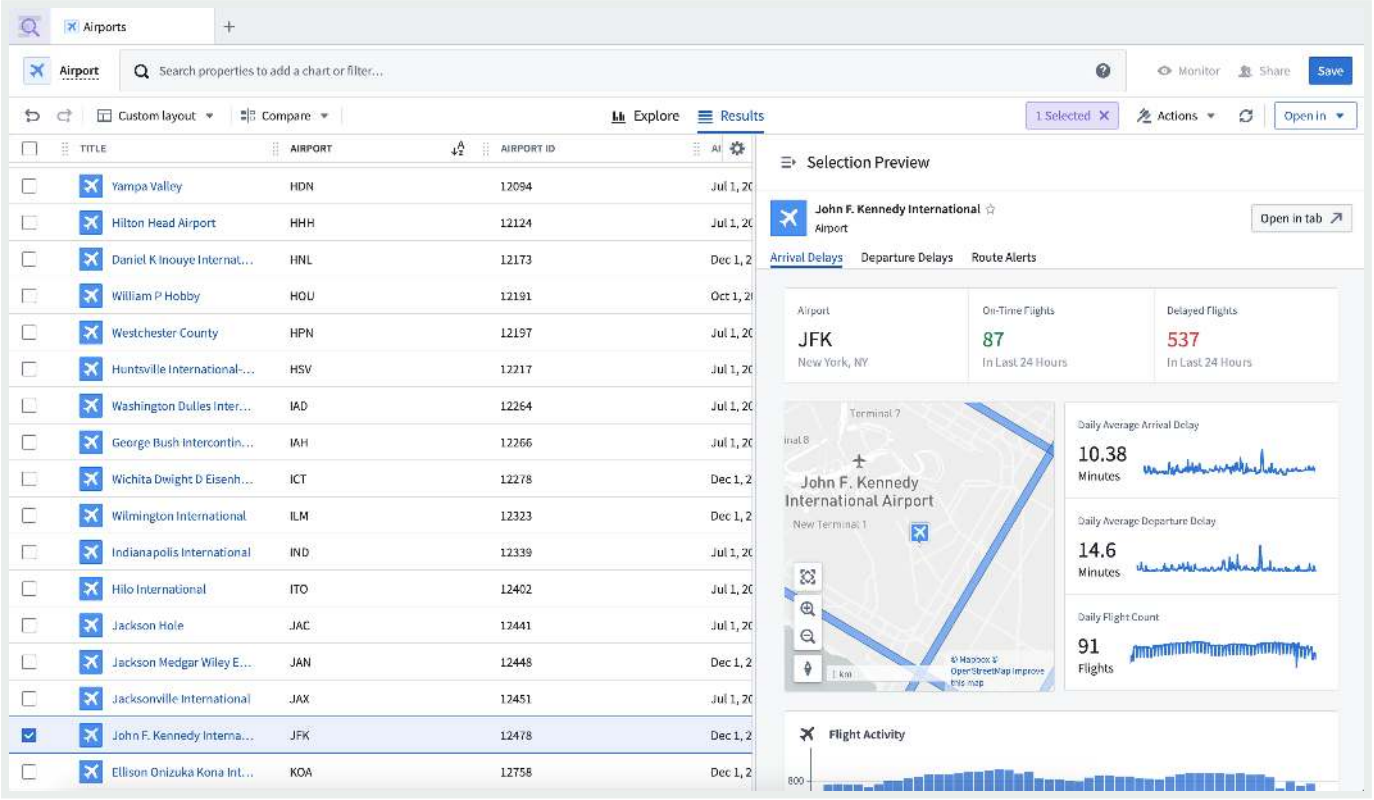
Workshop

Workshop's Object View widget can display a detailed Object View inside custom Workshop applications. When configured to display the full Object View, this widget is often used within an overlay or modal to accommodate the full page resolution.



Object Explorer

In Object Explorer, you can search for an object, then select it to open the full Object View, providing detailed information about the objects defined in the Ontology.

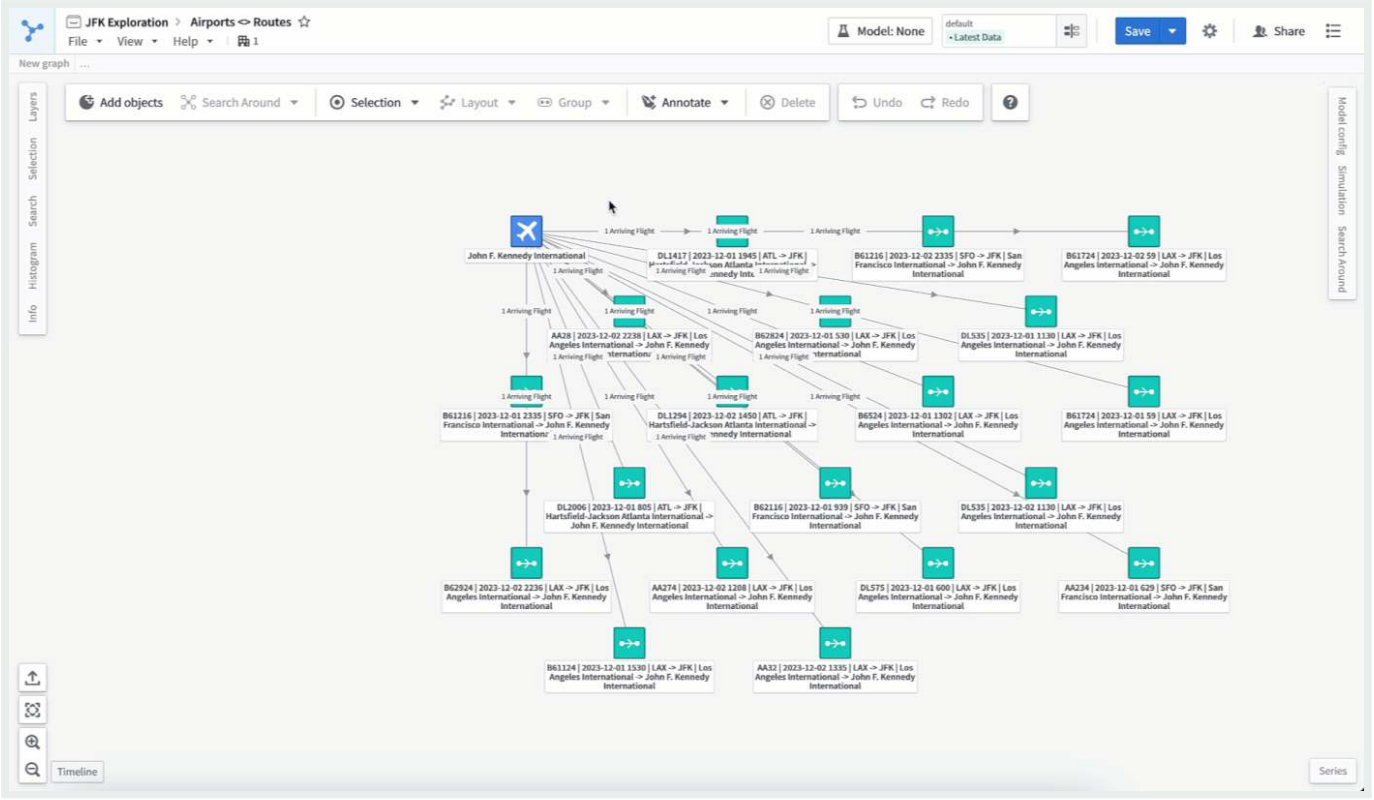


Platform applications

Applications like Vertex, Maps, and Gaia use panel Object Views to provide a customizable, compact view of the selected object. Within the panel, selecting the object's title will open the full Object View in a moveable and resizable modal.

API Reference ↗

Send feedback



PREVIOUS
← Branching object views

NEXT
Configure full Object Views →

Configured full Object View

Edit configured full Object Views

The default configured full Object View for all object types shows a single Property List widget displaying prominent properties of the object type, and a Links widget that displays the object's links, if any exist. To make changes to the configured full Object View, use one of the edit entry points to navigate to the configured Object View editor. Once in the editor, the content of each tab of a configured full Object View can be configured just like a regular Workshop module.

Edit Object View tabs

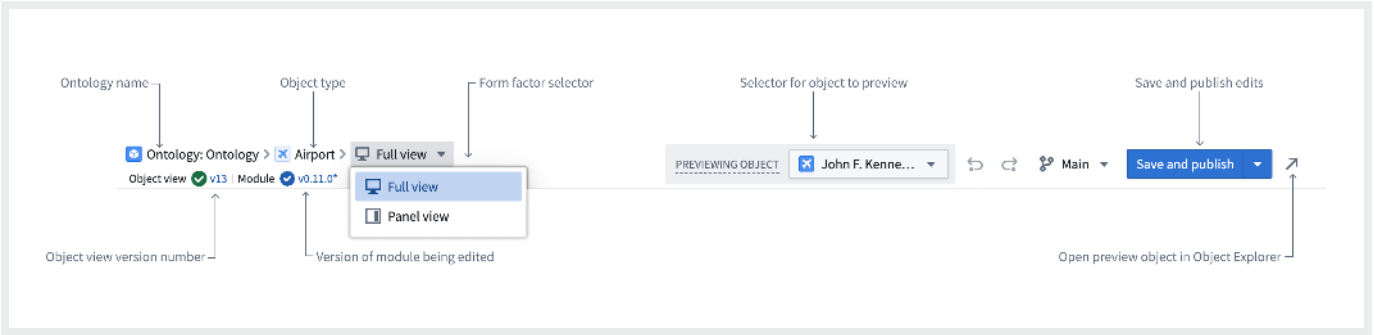
There are two parts of a full Object View that can be edited: the tabs and the tab content. Each Object View tab is backed by a Workshop module, which enables you to use Workshop to create Object View content with advanced capabilities and features. You can use Workshop tabs to:

- Have full flexibility over the layout
- Flexibly and dynamically load information using Workshop variables
- Display simulations or model-backed results using Scenarios

Use the Object View editor

T API Reference ↗ sections within the Object View editor: the **header**, the **object title bar**, a **kshop module**.

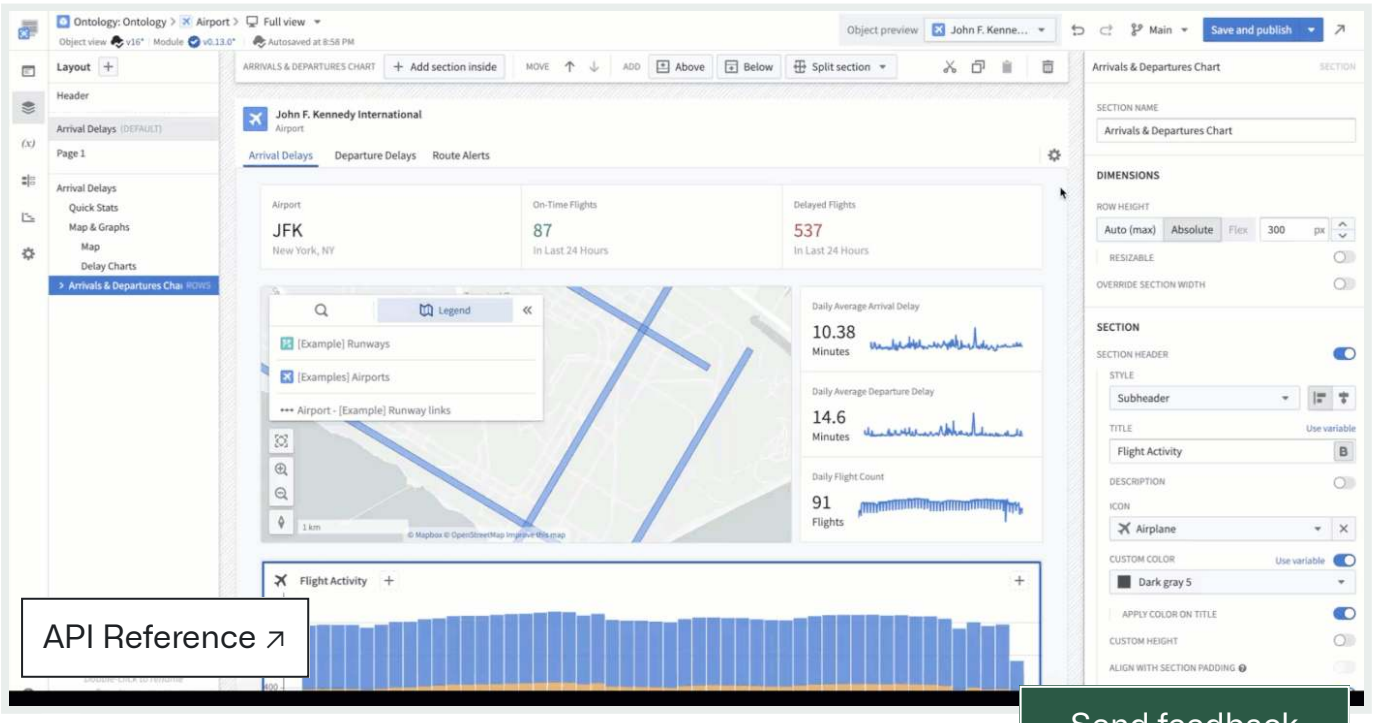
The breadcrumbs in the header display the Ontology name, object type, and form factor. The form factor is a dropdown that can be used to switch between editing the full and panel Object View. Below this, version numbers are displayed for the Object View and the current workshop module. On the right, you can select different objects to preview, save and publish your edits, and open the object in Object Explorer.



In the object title bar, you can manage tabs by selecting the gear icon. Each tab corresponds to a single workshop module. If only one tab is configured, the tab title will be hidden when viewing the Object View, even though it appears in edit mode.

Selecting the gear icon opens a dialog that allows you to add, reorder, rename, and delete Object View tabs. Deleting a tab also deletes the Workshop module that the tab contains.

This dialog also allows you to configure a tab's visibility settings, or access the legacy Object View editor if you need to edit other legacy configuration options.



The content of the tab can be edited just like a regular Workshop module. Once edits are complete, the **Save and publish** button will save and publish both tab edits and edits to the current module, unless automatic publishing is disabled.

PREVIOUS
← Use full Object Views in the platform

NEXT
Panel Object Views / Use panel Object Views in the platform →

Use panel Object Views

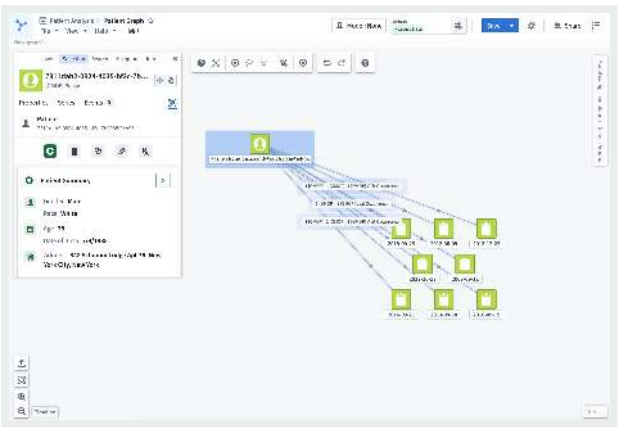
Panel Object Views are embedded into applications across the Palantir platform to provide users with a consistent experience of object data across workflows. All object types have a standard Object View panel available by default, and you can build configured panel Object Views to display either an *object instance* or an *object set*.

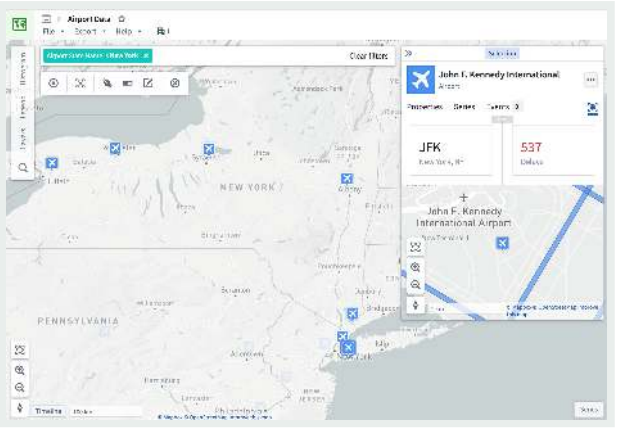
Panel Object Views in platform applications

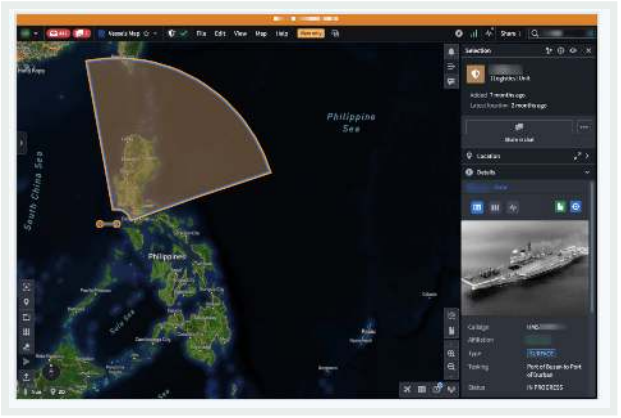
Panel object views can appear across Palantir platform applications, such as Vertex, Map, and Gaia.

Object instance panels

Object instance panels display a single instance of an object type and appear in an application's selection panel.

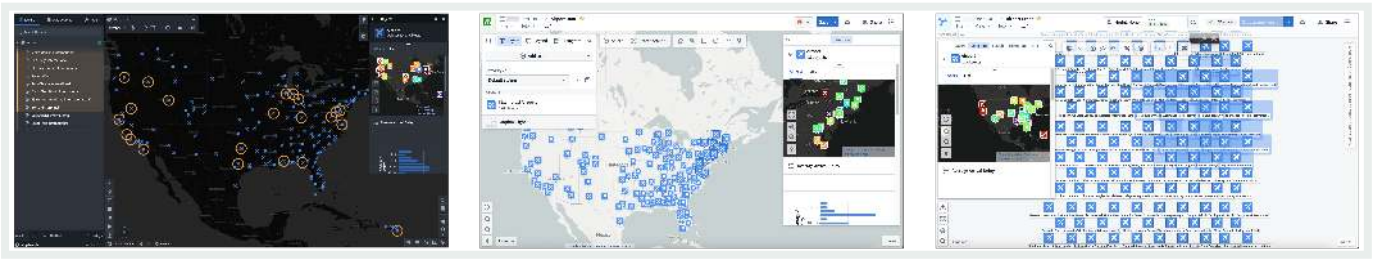






Object set panels

Object set panels display an aggregated view of multiple instances of a single object type. They appear in applications when you select an object set comprised of several instances of the same object type.

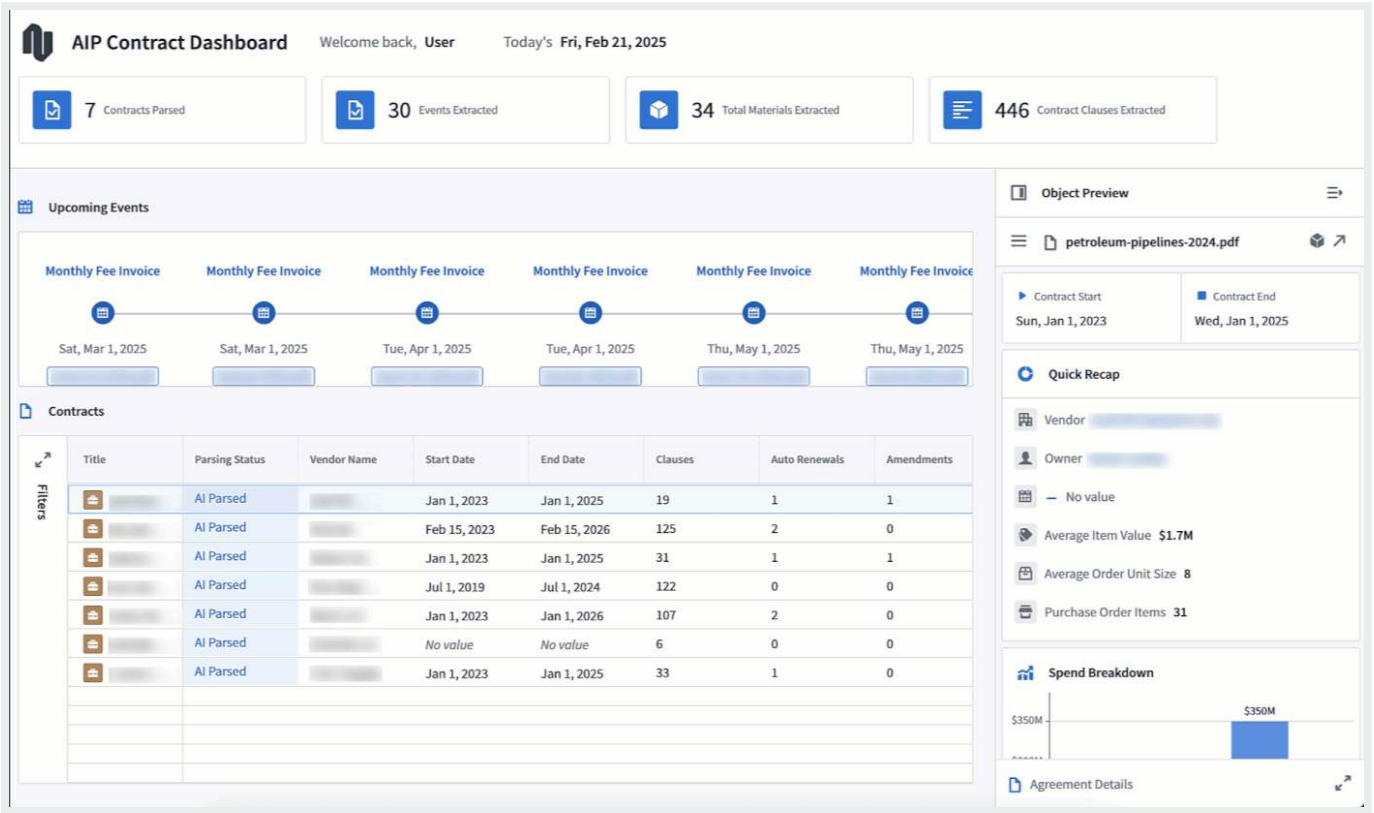


API Reference ↗

Panels in Workshop applications

Send feedback

Panels can be used in custom Workshop applications to provide a compact view of object data. To enable this, use Workshop's Object View widget, configured to show the panel form factor.



PREVIOUS
← Full Object Views / Configure full Object Views

NEXT
Configure panel Object Views →

API Reference ↗

Send feedback

Configured panel Object View

There are two types of configured panel Object Views you can build to display one or multiple objects of an object type: *object instance panels* display individual objects, while *object set panels* display multiple objects as an object set. Both types of configured Object Views share the same edit entry points and configuration experience.

Object instance panels

The default object instance panel view shows a single Property List widget that displays prominent properties of a single instance of the object type.

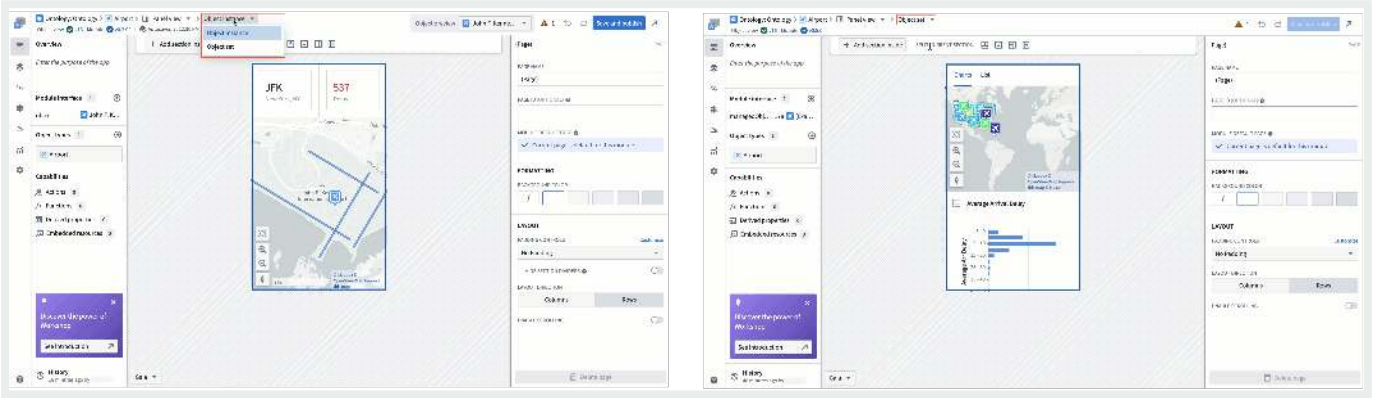
Object set panels

The object set panel displays an aggregated view of multiple instances of the same object type. The default object set panel provides a tabbed layout with two interfaces to explore object collections:

- The **Charts** tab displays up to five XY Charts that visualize object aggregations grouped by property values.
- The **List** tab shows an Object List widget displaying up to three properties per object, including the object's title, prominent properties, and media when present.

To make changes to either type of configured panel Object View, use one of the edit entry points to navigate to the configured Object View editor. Once in the editor, you can customize the configured panel Object View's content as you would customize a Workshop module.

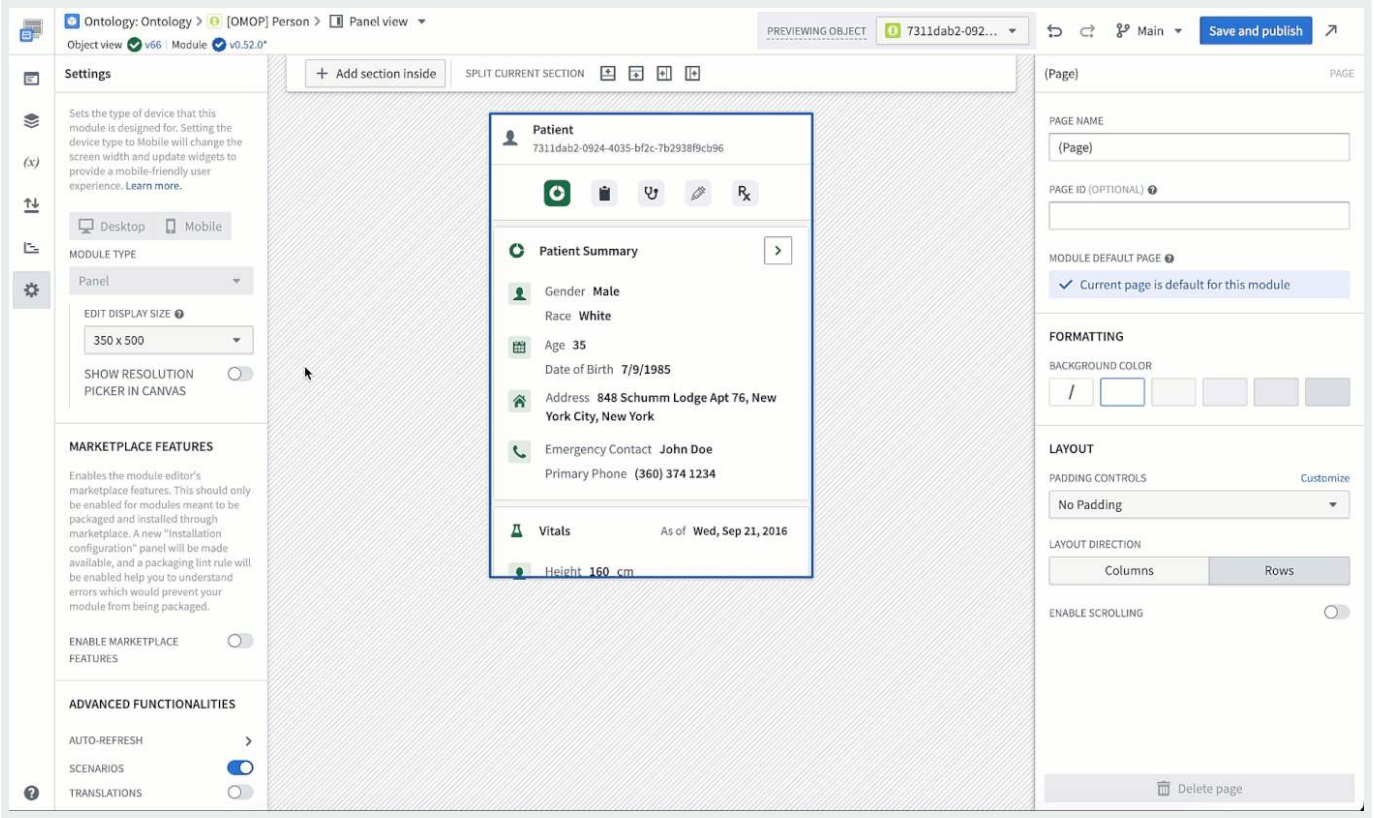
If you are configuring an object instance panel view but want to switch to an object set, select **Object instance** from the top ribbon before choosing **Object set** from the dropdown menu. You can use the same menu to switch *from* **Object set** *to* **Object instance**, as well.



In the settings section of the left-side panel, the **Module Type** section allows you to configure the size of the editing canvas to ease building a compact module.

- **Edit display size:** Selecting an option in the dropdown will adjust the preview size of the panel on the canvas. The actual size of the panel will vary between devices and applications, so this is just a tool for builders to use to approximate the available space within different workflows.
  - Display options include **application presets** that match the size of the panel in different platform applications, common **resolution presets**, and **manual entry** of the module's height and width in pixels.
- **Show resolution picker in canvas:** When enabled, a resolution picker will be shown in the bottom left corner of the editor, allowing builders to edit the display size of the module directly on the canvas.
- **Fit to canvas:** When the resolution size exceeds the canvas size, a button will appear next to the resolution picker that toggles between fitting the entire panel in view or viewing the panel at standard zoom.

API Reference ↗



PREVIOUS
← Use panel Object Views in the platform

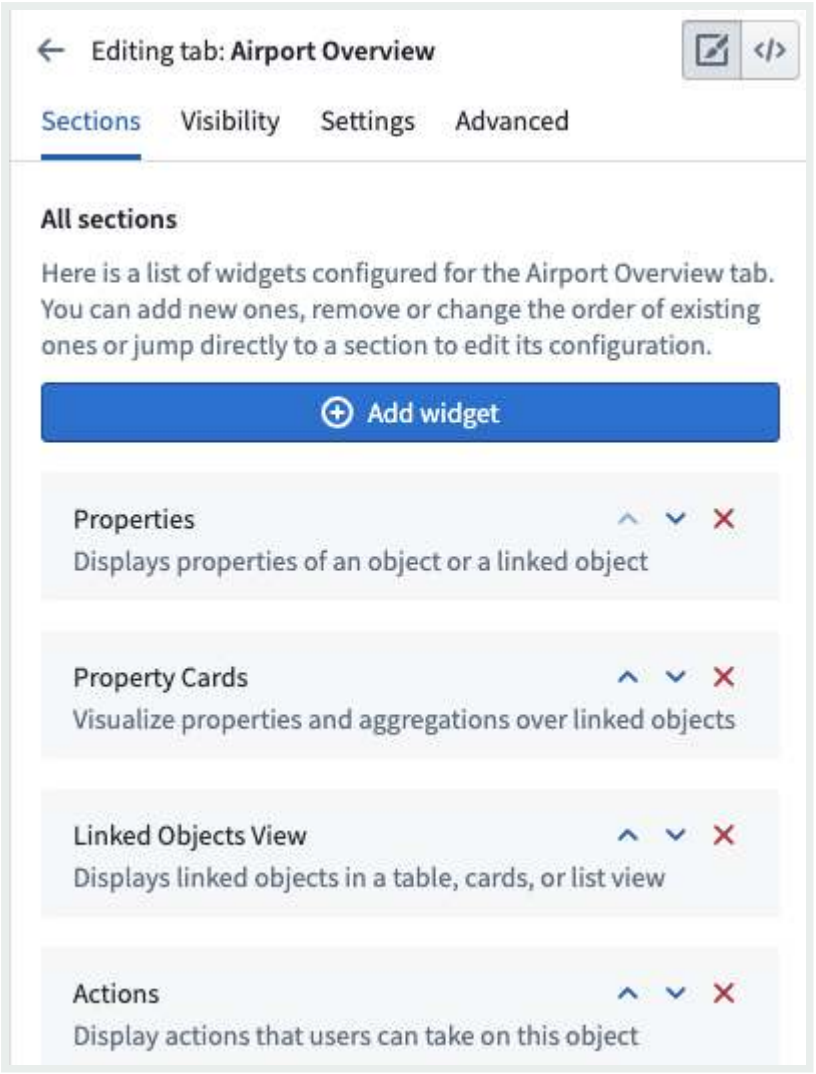
NEXT
Legacy Object Views / Configure legacy Object Views →

Configure legacy Object Views

> To access legacy Object View configuration, navigate to the Object View editor for an object type, select the **Manage tabs** cog icon, and hover over a tab to select the **Open in legacy editor** icon.

Configure tabs

A legacy widget-based tab displays a list of widgets organized into sections. You can add, delete, and re-order widgets or navigate to other settings from within the tab settings.



Configure widgets

The building blocks of legacy Object View tabs are called **widgets**. Widgets are sometimes referred to as *Sections* or *Plugins*. Widgets are the primary way to display some form of data on an Object View. Use them to visualize data as KPIs or charts, arrange the layout of an entire Object View, and manipulate displayed data.

There are a several widget types available in Object View:

- Properties and links
- Visualize
- Filters
- API Reference ↗
- Embed other applications and files

Send feedback

Widget-specific settings

Widgets are used to visualize or manipulate data related to an object. Widgets typically access one of the following:

- Properties of the current object
- Objects linked to the current object
- Aggregations on properties of objects linked to the current object

When you configure a widget on an Object View, determine which object and what property you want to visualize. You must set up the relevant objects and define links in advance within the Ontology metadata.

The specific settings for each widget are unique to the functionality it provides. In general, there are two major components to widget-specific configuration:

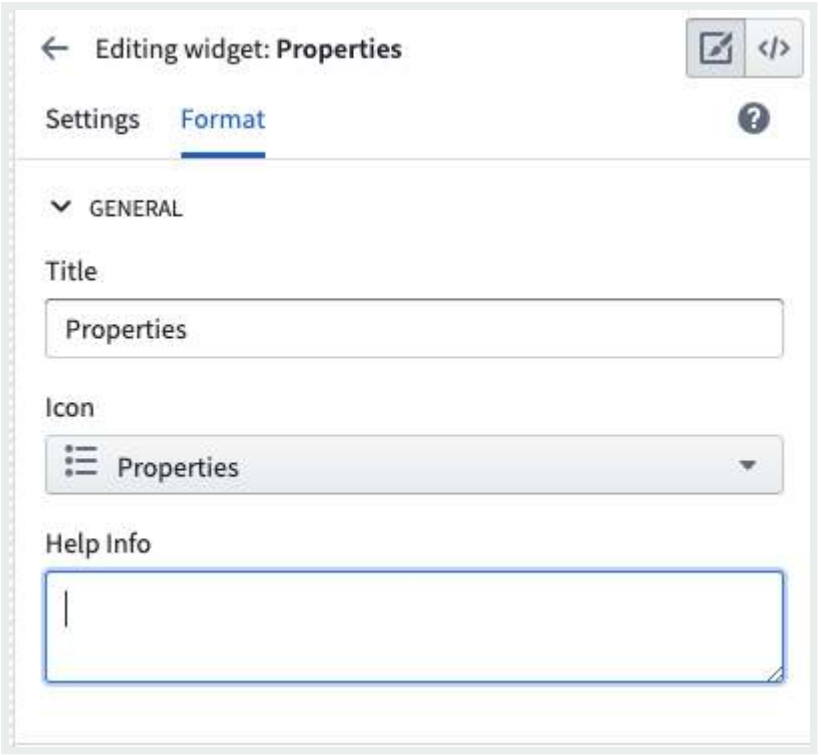
- The **object model** configuration requires selecting the object or linked object and which properties should be used. For example, when configuring a chart, you may need to aggregate all Flights in an Airport per Destination.
- The **visual** configuration requires selecting options such as chart types, colors, formats, text labels, etc.

Widget format settings

All widgets have default format settings:

General

- **Title:** Add a title to display in the widget header. By default, this is either the widget name an empty space. You must add a title to the widget to save it.
- **Icon:** Choose an icon to display in the top left of the widget header. Each widget has a default icon.
- **Help Info:** Empty by default, you can add text to explain the widget to users.

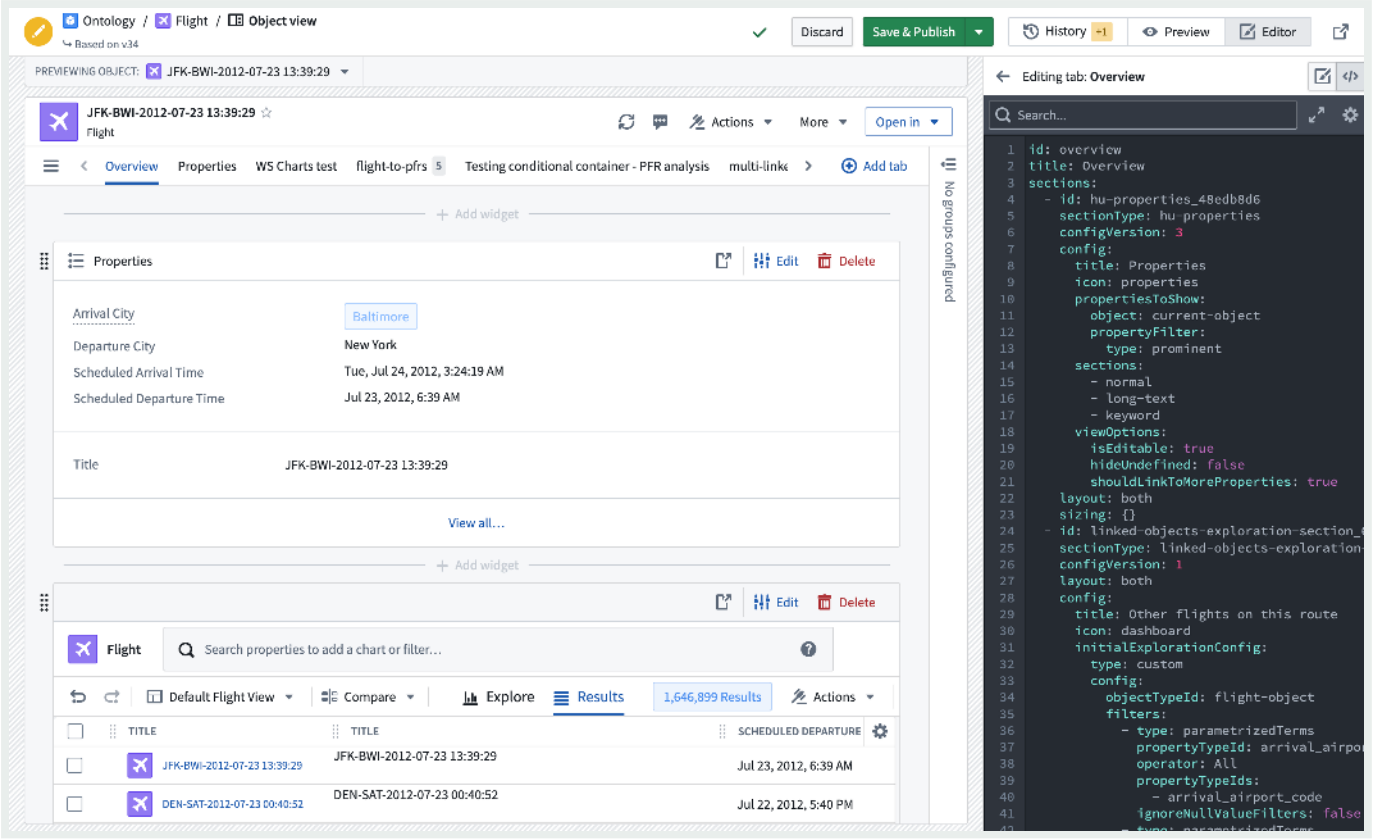


Layout

- **Alignment:** You can choose whether a widget should be the full width of the Object View or have a right or left alignment to place two widgets side-by-side. Alignment is set to full width by default.
- **Sizing:** The sizing setting does not exist on all widgets. When it is available, you can control the minimum and maximum height of the section within the Object View.

Reuse widget configurations

You can reuse the configuration of a previously created widget by copying the YAML configuration. Access the configuration by selecting the code icon in the upper right corner of the Object View editing screen. Then, copy and paste the configuration into the YAML configuration of a new widget.



PREVIOUS
← Panel Object Views / Configure panel Object Views

NEXT
Configure tabs →

Configure tabs

A **tab** is the main method of navigation and grouping content within Object View. Each tab contains a Workshop module and has customization options for conditional visibility, filtering settings, and layout settings.

There are two types of tabs available to add to your Object View: Managed Workshop, and Standalone Workshop modules.

- **Managed Workshop modules:** We recommend using Workshop to build a new tab within Object View. This option allows you to develop more sophisticated views that can leverage the full power of Foundry's application building capabilities. Learn more about Workshop-backed tabs. Managed modules have their permissions automatically kept in sync with the Object View, and cannot be reused.
- **Existing Workshop modules:** You can embed modules that have already been built in Workshop directly into Object View tabs. You can use the same module in multiple Object Views.

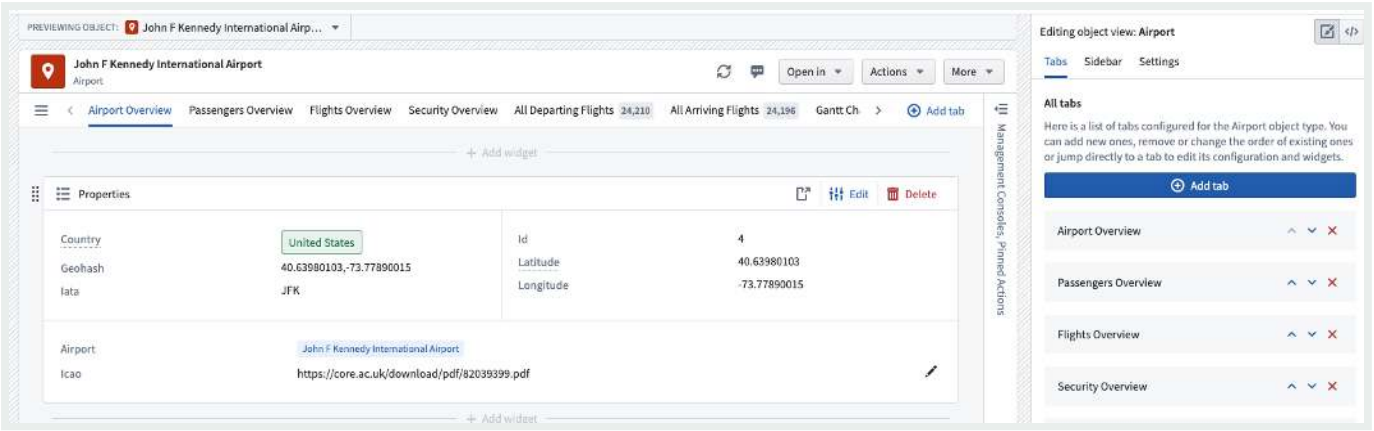
Some tabs were built with the **Legacy** builder, which allowed you to create simple tabs with limited flexibility for layouts, widgets, and data options. New tabs using this builder can no longer be added, but existing tabs are still supported.

Manage Tabs

Add a new tab

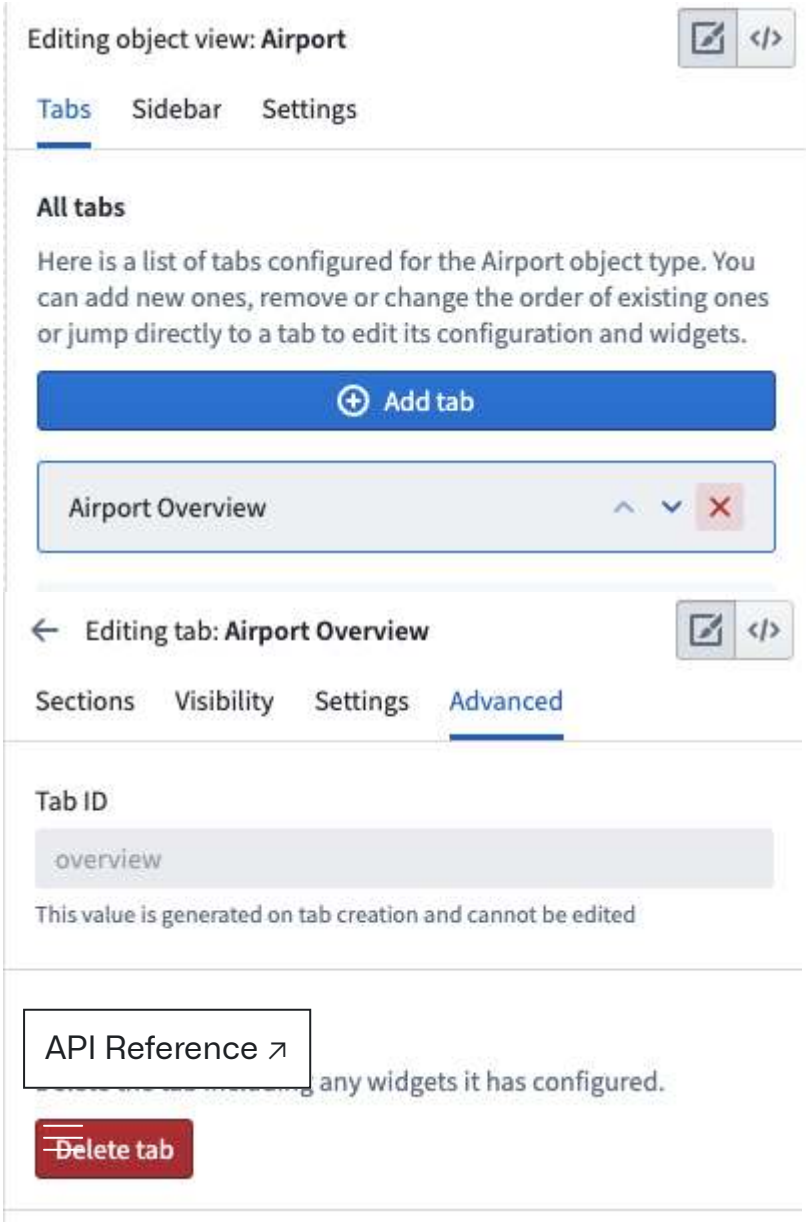
API Reference ↗

You can add a tab from two places within the editor: the **Tabs** section of the Object View e ar, or the **Add tab** button in the tab list on the Object View option will allow you to select the tab type of your choice.



Delete a tab

You can delete a tab from the list of tabs within the Object View editor sidebar. You can also click into the tab in the editor sidebar, navigate to **Advanced** , and click **Delete tab**.

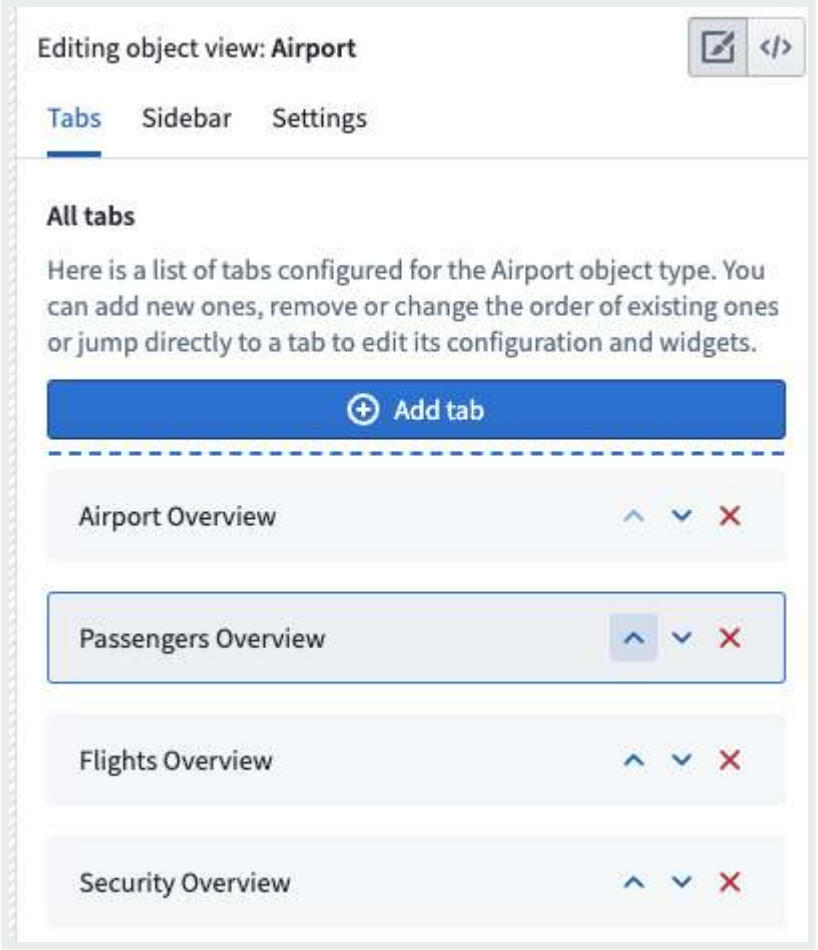


API Reference ↗

Send feedback

Move a tab

Move a tab using the Object View editor sidebar list of tabs. Click the up or down arrows to rearrange their order in the Object View preview.



Configure tab settings

Tab visibility

A tab can be conditionally shown to viewing users in several ways. One method is through the use of configured Object Views and profiles. You can also configure a tab to be conditionally visible based on two other factors:

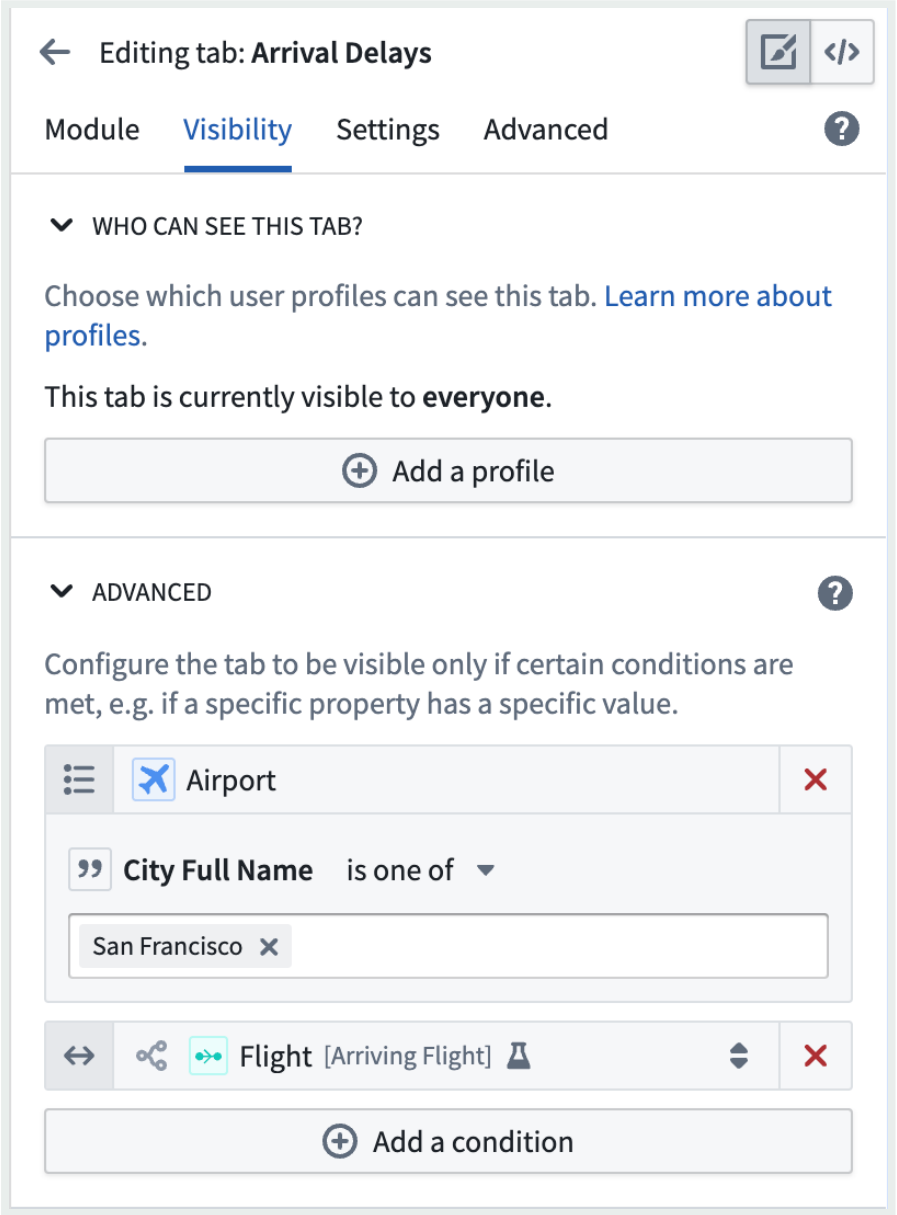
- This condition is fulfilled if the value of a property on the currently viewed object is equal or unequal to a given value. Property value conditions are useful when a tab shows content that is only relevant to an object whose property matches an expected value.

API Reference ↗

Send feedback

    - For example, you may want to add a *Regional View* tab to an Airport object that only applies to Airports in a specific region. You may have a different *Regional View* tab with conditional visibility for other regions. Each tab could have different visual components.
- **Link visibility:** This condition is fulfilled if the user viewing the tab has permission to see the object type to which the currently viewed object may be linked. Link visibility conditions are useful when a tab shows content based on an object's links, but the viewing user may not have permission to see the objects on the other side of those links.

Here is an example of how these conditions may appear:



API Reference ↗

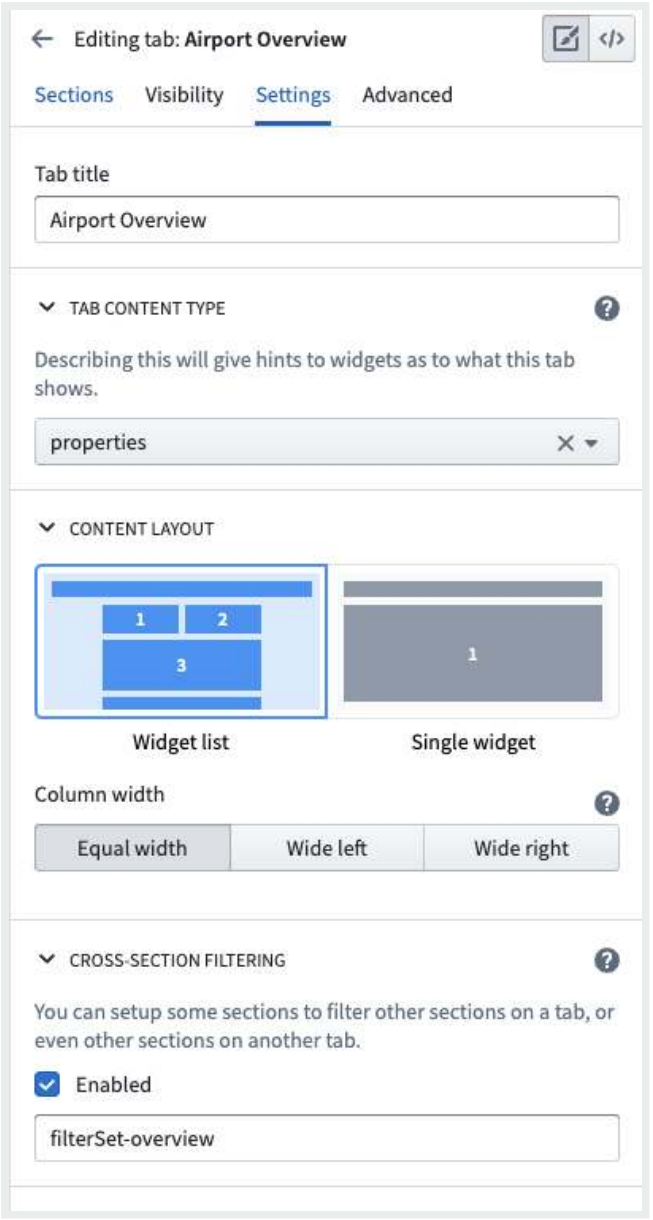
Tab settings

Send feedback

Other general tab settings that you may configure include:

- **Title:** The title setting controls the label shown in the tab list within the Object View. The tab title should be short and descriptive of the content.
- **Content type:** You can use this configuration to specify a link type or link that appears within the tab, if relevant. If you select the *Link* option, you will see a badge next to the tab title in the Object View which shows how many objects are linked to the currently viewed object.
- **Content layout:** All legacy tabs support a two column widget list layout which is activated when widgets specify that they should be aligned to a specific column. Use this configuration option to control whether or not the tab level columns should be equal width or have different sizes.
- **Cross-section filtering:** Enable this setting to allow widgets within this tab to publish and consume filters controlled by interactions with the widget. For example, you may want to select an entry within a chart and filter all other charts to only show related data. You can also customize the ID of the filter set and then use this same ID on other tabs in order to consume and persist filter across multiple tabs.

Here is an example of configured tab settings:



PREVIOUS
← Configure legacy Object Views

NEXT
Configure the applications sidebar →

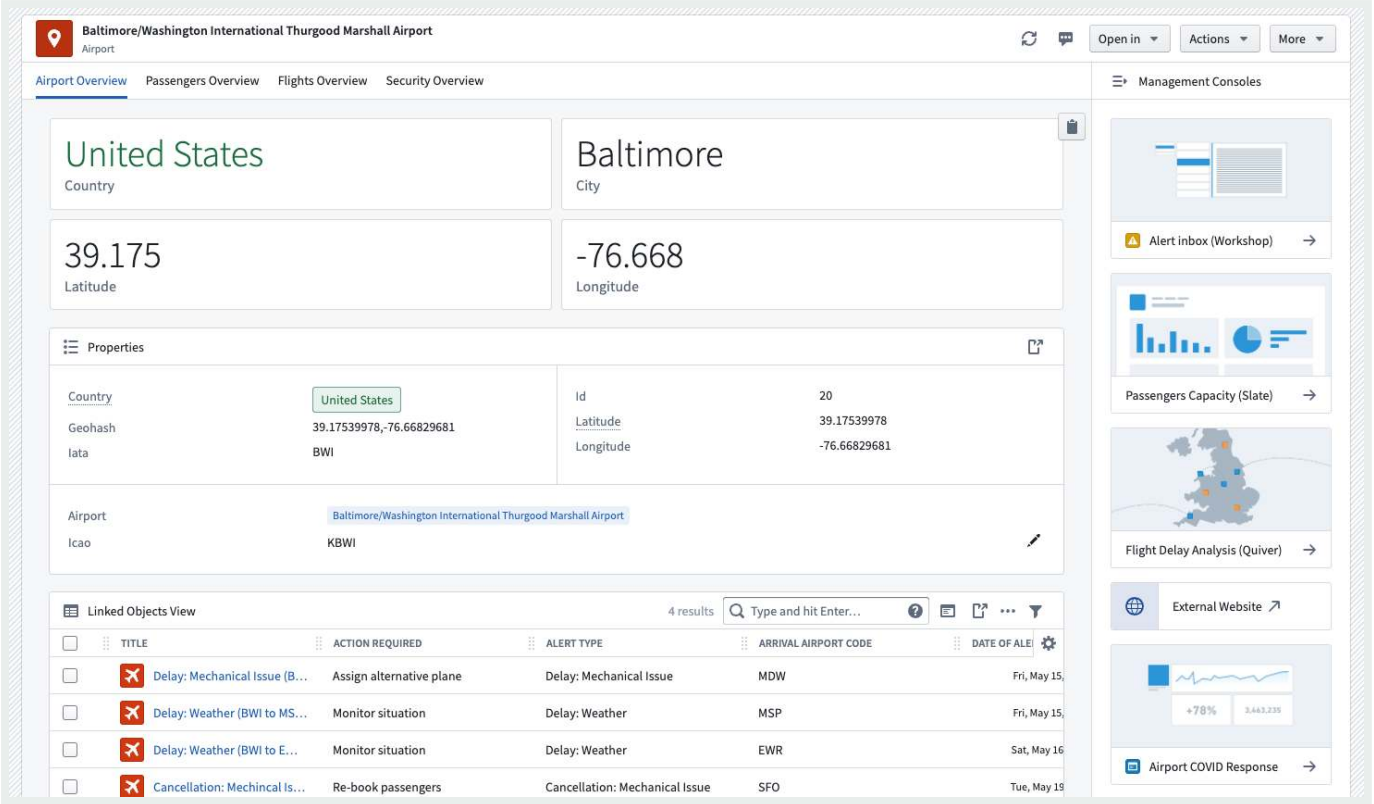
Configure the applications sidebar

The **applications sidebar** is used to display and embed applications, analyses, actions, and other resources related to the current object. The applications sidebar visually differentiates these resources from the main content of that object.

The applications sidebar supports embedding all object-based applications, including Workshop, Quiver, Slate, Action types, and more. Once added to the sidebar, you can open these applications within the context of the object. The sidebar also allows parameterized URLs to link to other apps and external websites.

In the example below, you can see an `Airport` object. The body tab includes primary information about that object, while related applications include:

- A Workshop Alert Inbox on flights delays, used to triage traffic;
- A Slate application used to manage the airport's passengers capacity; and
- A dedicated application for airport COVID response.

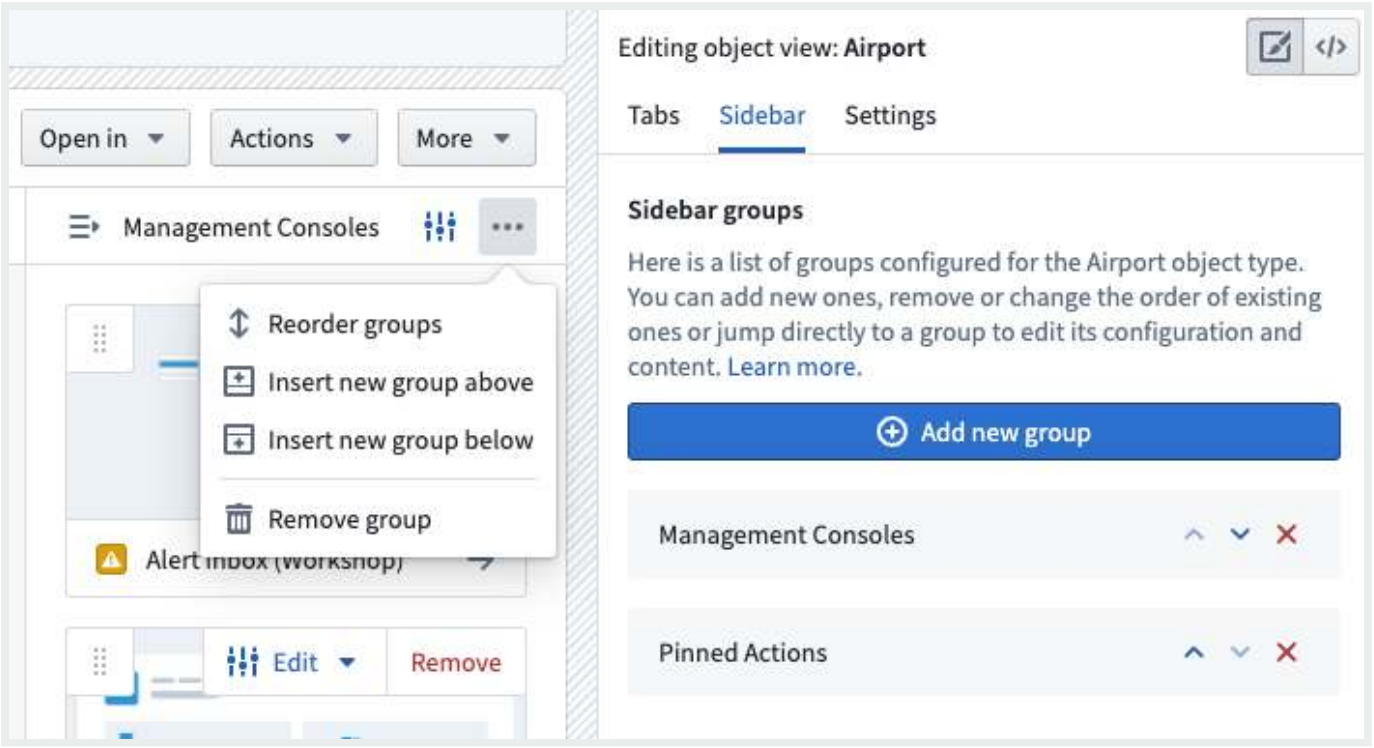


Set up the applications sidebar

The applications sidebar is an optional, opt-in addition per Object View. The sidebar is not visible to users until you add a group to it. The **Add new group** option can be found under the **Sidebar** tab or by expanding the sidebar itself (as shown in the image below). Once a builder adds applications and/or actions and publishes that version, the sidebar will be displayed to end users. The sidebar will not be displayed if it only contains an empty group or groups.

API Reference ↗

Send feedback



Once you add applications and/or actions and publish your changes, the sidebar will be displayed to users. The sidebar will not be displayed if it only contains an empty group or groups.

Edit the Applications Sidebar

The applications sidebar is modular and configurable. The sidebar can be split into groups with different application cards and actions in each group.

The sidebar has a dedicated configuration for each group and application card in the sidebar. As numbered in the images below, options include:

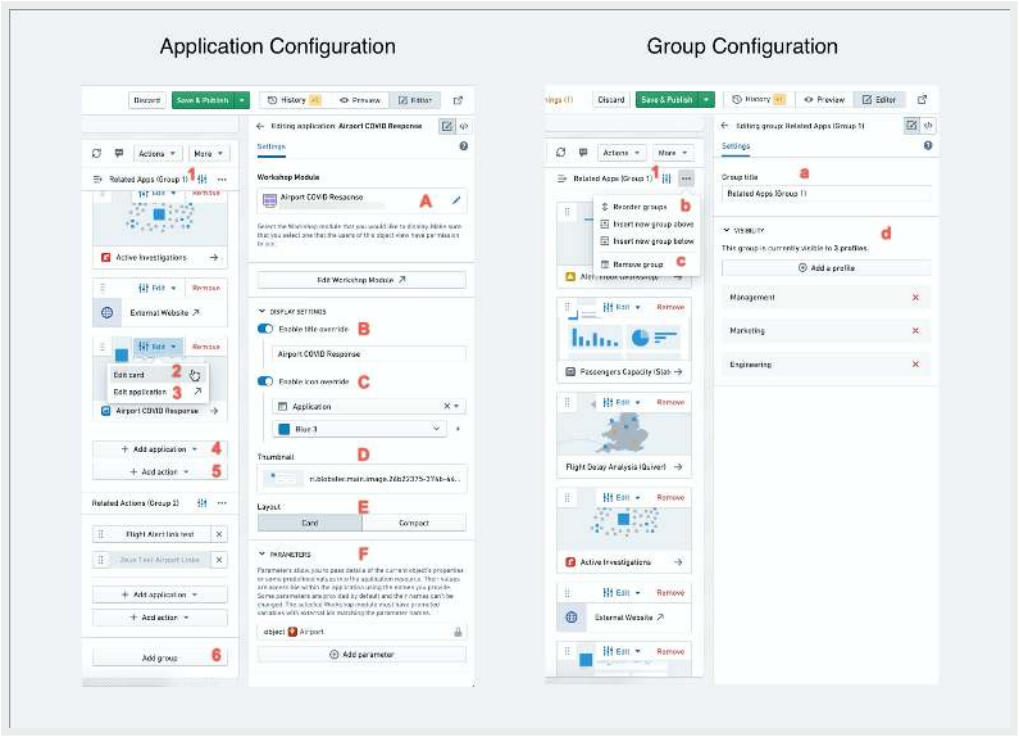
1. Edit the entire group with the following options:
   - (a) Configure the group title
   - (b) Reorder application cards and actions within the group
   - (c) Remove the entire group
   - (d) Edit visibility (make the group visible to only specific user profiles)
2. Edit a specific application card, which opens up a secondary menu enabling you to:
   - (A) Change the application resource used for a single card (more details below)
   - (B) Override the title
   - (C) Override the icon

API Reference ↗

Send feedback

- (D) Use a thumbnail on the card; thumbnails must be uploaded to Foundry and saved in a folder
- (E) Select Card mode or Compact mode
- (F) Add parameters

3. Edit the backing application (this will open the specific module in the source app, such as Workshop or Slate)
4. Add a new application to a group
5. Add a new action to a group
6. Add new groups



Add/change an application in the sidebar

There are two main ways to add an application to the sidebar:

1. For object applications (Workshop, Quiver, Slate, etc.) embedded an Object View:
   - Select **Add application**, choose an application (e.g., Workshop) to open a resource
   - elect the specific resource (e.g., Workshop module) you would like to
   - The parameter configuration for Workshop and Slate applicati
   - details of the current object's properties, linked objects set, or predefined values into

the linked resource.
    - The parameter values are accessible within the embedded Workshop or Slate application as variables with the same name as the parameter name. In Workshop, these must be configured in the variable **Settings** panel as module interface variables.
2. For other Foundry applications and external websites to open in a new tab:
    - Add an **Application link** to create a new card with a parameterized URL.
    - All links on the sidebar are opened in a new browser tab and not embedded like object applications mentioned above.
    - These URLs can be used both for Foundry applications (Workshop, Vertex, another Object View, or Foundry documentation, for example) and for external websites.
    - The URL card requires a template that combines an address and/or any parameters defined on the parameter configuration. To place parameters in the URL, wrap the parameter name like this: `{{parameterName}}`. If the property is an entire URL, check the box for **Encode property value** to mark it as an entire URL, and include only the `{{parameterName}}` in the URL text box.
    - By default, the details of the current object are available using these parameters: `{{objectId}}` & `{{objectTypeId}}`.

> If a user doesn't have permissions to the embedded application, they would not be able to open it but would still see the application card. Make sure you set up the right permissions on each application in the sidebar.

PREVIOUS
← Configure tabs

NEXT
Configure profiles →

Configure profiles

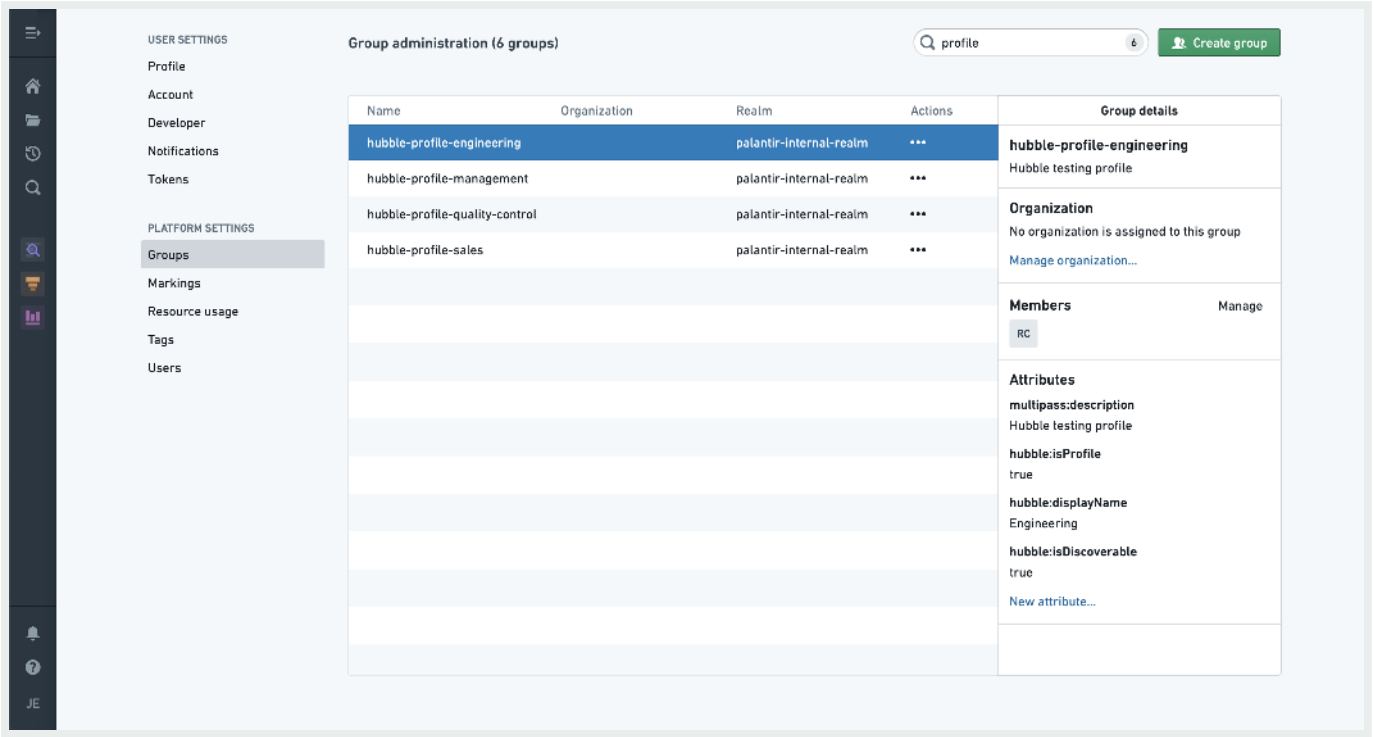
Profiles enable you to configure how Object Views should be surfaced to users with different roles. You can use profiles to control the visibility of Object View tabs for different users so they see views specific to their needs.

Configure a profile

Object Explorer is powered by a service called `Hubble`. To use a group as a profile in Object View, add the following Hubble attributes from within the **Groups** tab in Platform Settings:

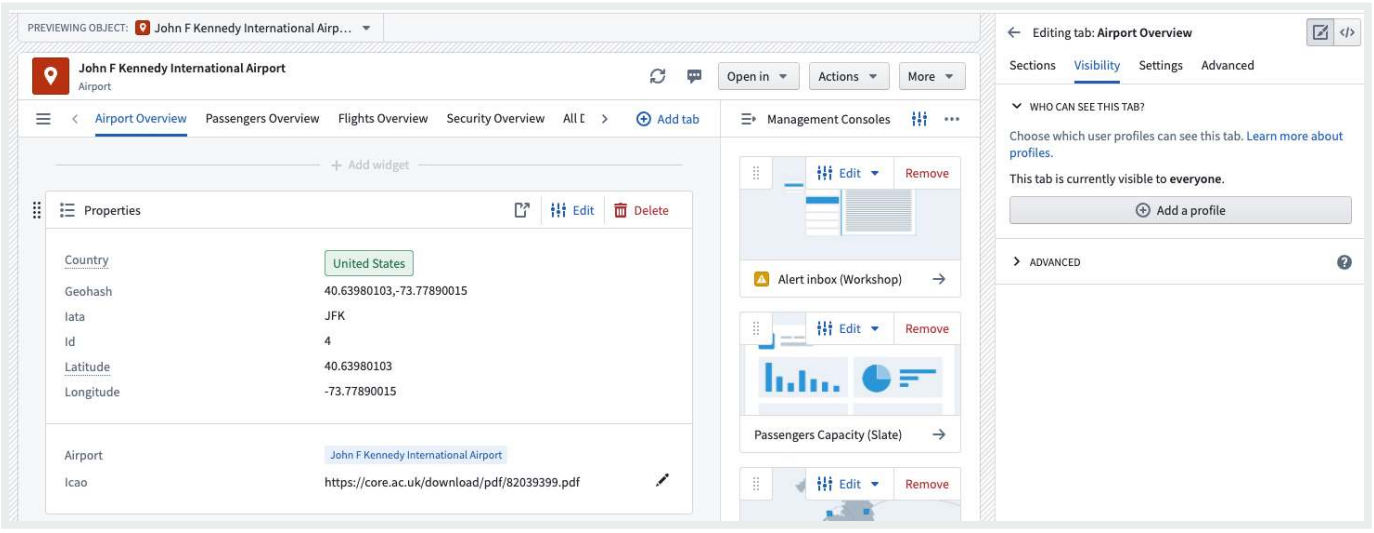
- `hubble:isProfile` : `true`
- `hubble:displayName` : `Set a name you want end user to see`
- [OPTIONAL] `hubble:isDiscoverable` : `true`
  - Setting the `hubble:isDiscoverable` attribute to `true` will make the profile visible to users who are not members of the group itself. Omitting this attribute means that only users who are in the group can access views assigned to this specific profile.

> Newly-created profiles may take up to five minutes to become available in the Object View editor.



Assign a profile to an Object View

Profiles are assigned on a tab level, meaning that for each tab you can assign specific profiles. To add a profile to a tab, access the editor sidebar, click on a tab in the **Tab** settings, select **Visibility**, and click **Add a profile**.

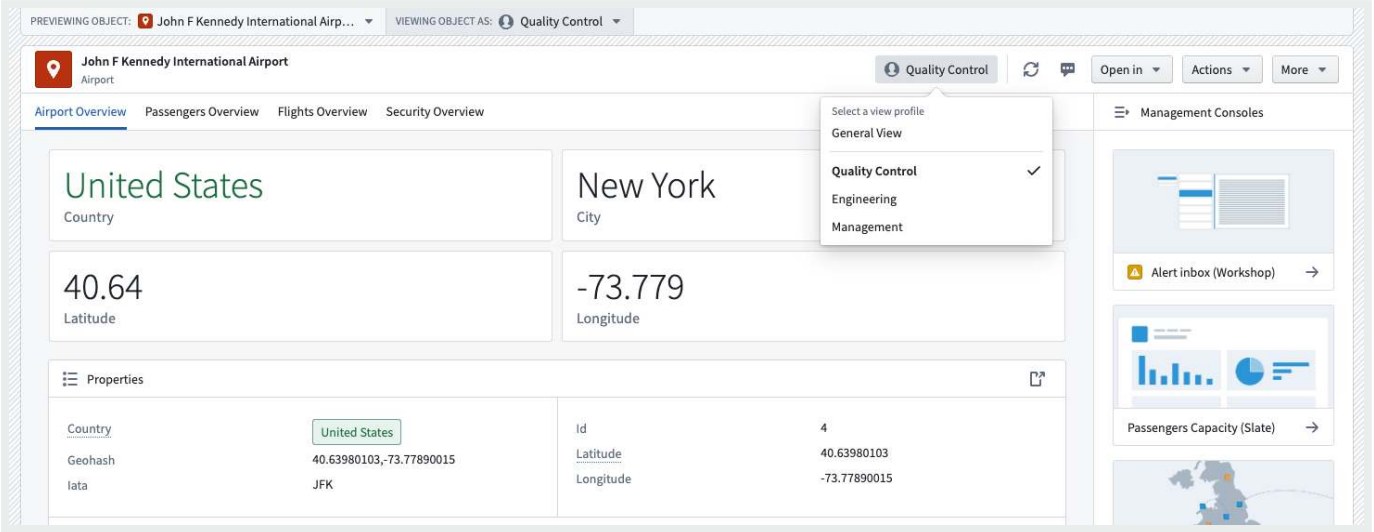


API Reference ↗

Switch profiles as a user

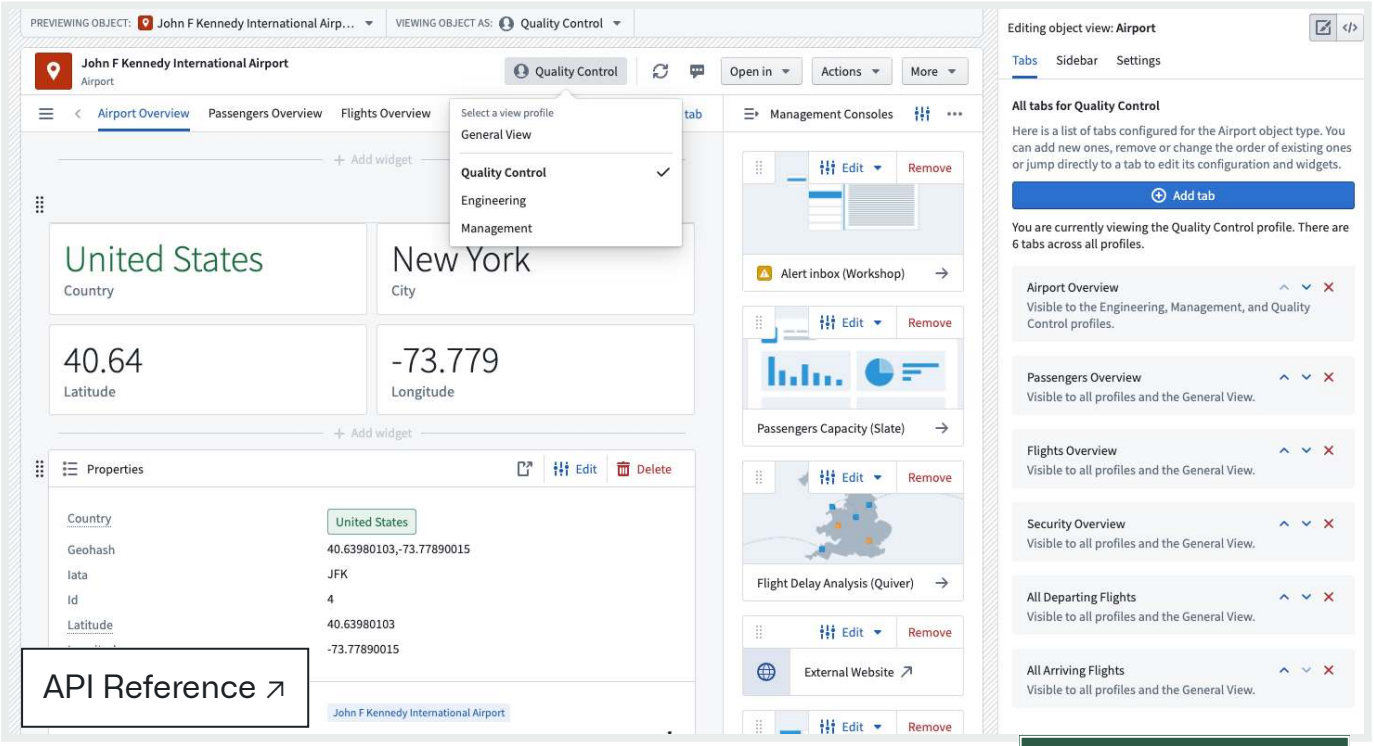
Send feedback

Once you add a profile to an Object View, you can switch between profiles. Select the profile type in the Object View header to access a dropdown menu containing available profiles. You can find the same dropdown menu by clicking **Viewing Object As:** at the top of the Object View.



Switch profiles as an editor

You can also access different profiles when editing an Object View. Doing so will allow you to see which tabs are visible to each profile.



API Reference ↗

Send feedback

Set a default profile for a user

To set a default profile for a user or user group, add them as a member to the group backing the profile. This action will work only if a user is a member of a single profile.

> You can add a maximum of ten profiles to each Object View.

PREVIOUS
← Configure the applications sidebar

NEXT
Properties and Links →

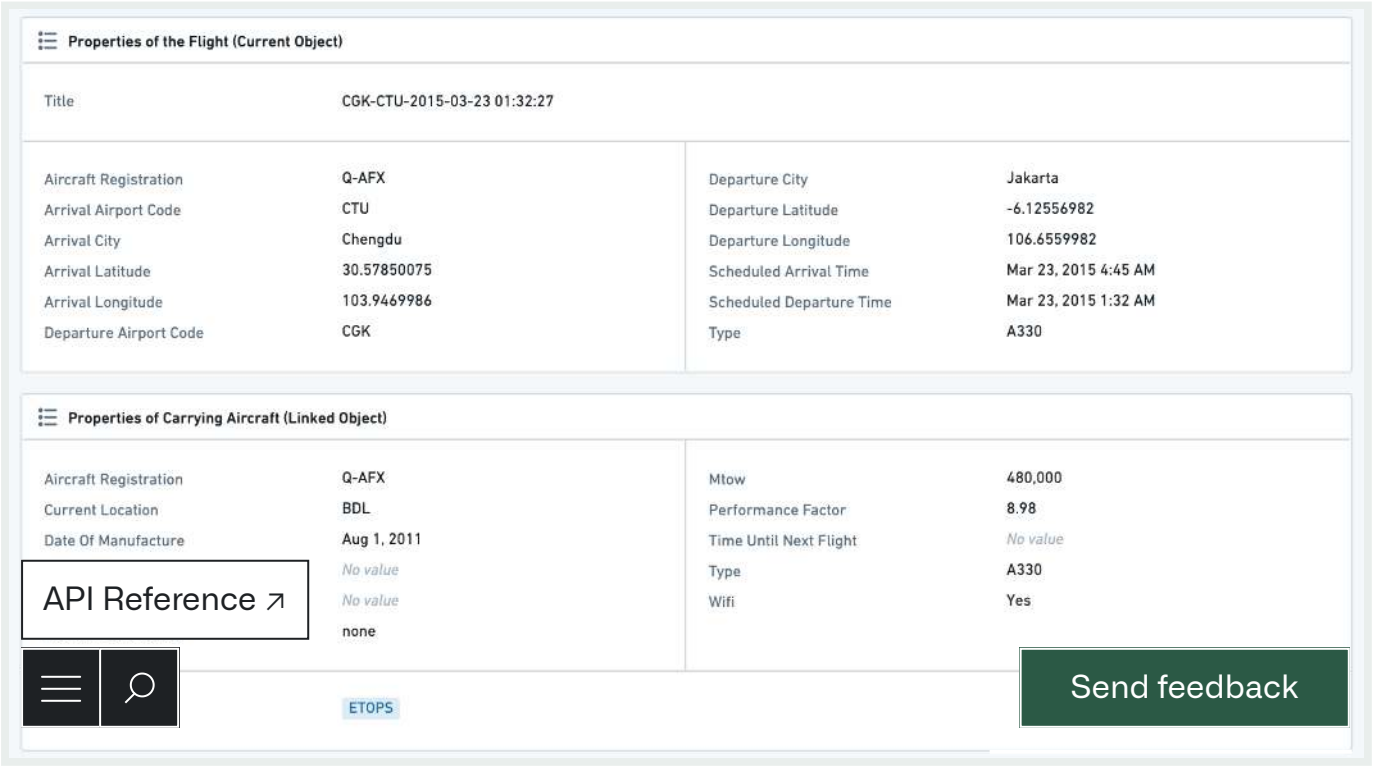
Properties and links

This page discusses all widgets displaying simple tabular views of an object's data model and ontology, highlighting the properties of the current object, and the links that the object has to other objects.

This category also includes widgets to present statistics, metrics or KPIs that are either pre-calculated in the object's pipeline and available as a property of the object, or that require aggregations over Linked Objects to the current object.

Properties widget

The **Properties** widget displays properties of an object or a linked object. A tab containing this widget is created as the default Object View for any newly defined object type.



Properties widget configuration

Properties definition in the Ontology Manager

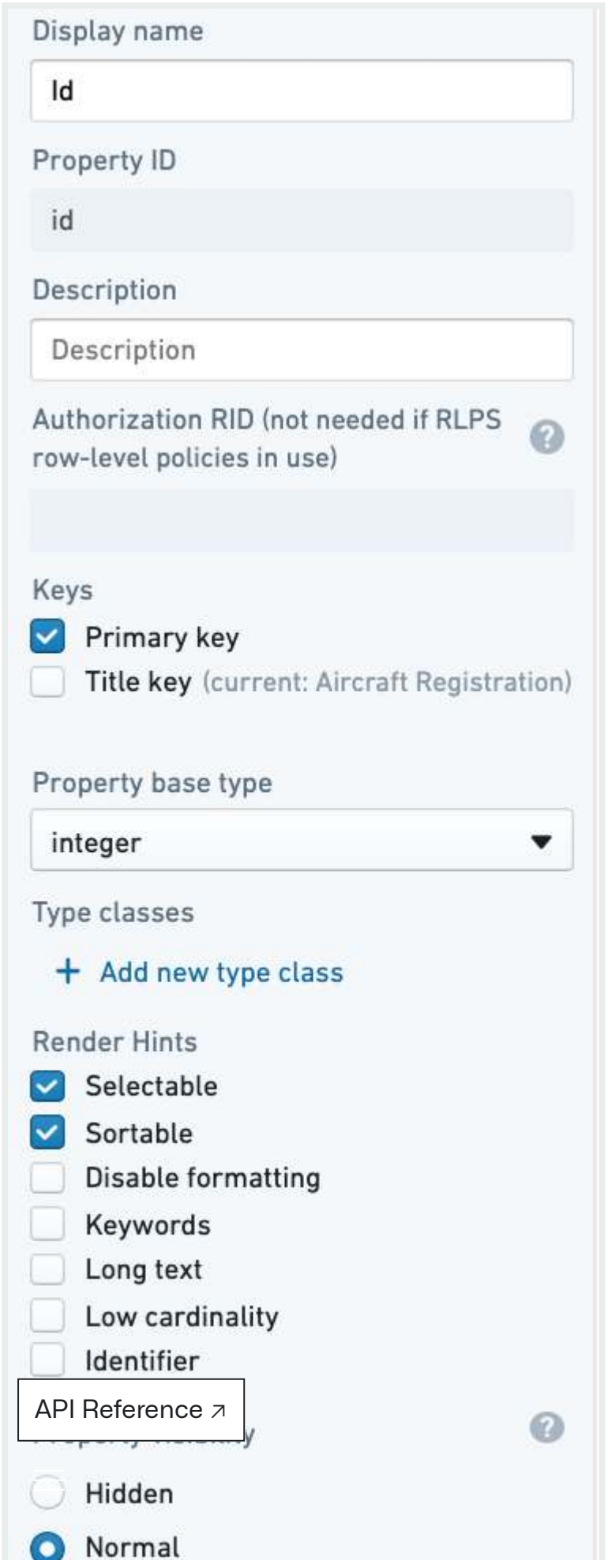
The Properties widget configuration is closely related to the properties definition in the Ontology Manager. As you “Edit Properties and Backing Dataset”, each Property has a separate settings panel, which allows you to define how it will be displayed in the Object View.

There are several settings that affect the display in the Properties Widget:

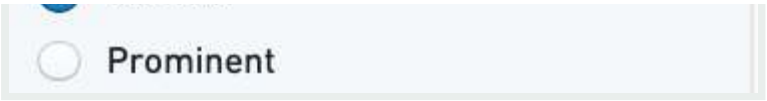
- Property visibility
  - Prominent: The property will be displayed in the Properties widget, and there is a configuration option in the widget to only display prominent properties. This option is used for the most important core properties of an object.
  - Normal: The property will be displayed in the Properties widget. This option is used for most properties.
  - Hidden: The property will not be displayed in the Properties widget (or anywhere in the Object View or Object Explorer). This option is used for properties such as internal IDs, relation-columns for links to other objects, and so on.
- Render Hints
  - Long text: This option, marking a property as a long text (usually for long strings, such as descriptions, comments, etc.), enables displaying texts in a separate section of the Properties widget.
  - Keywords: Render a property as a keyword, which changes the way it is viewed in the Properties widget, and also allows to display it in a separate section within the widget.
- Type classes: This allows you to define type classes to properties, which would affect their functionality.
- Make a property editable: If you wish to make a property editable, set up an action type or an inline action.

API Reference ↗

Send feedback



Send feedback



Configuration of the Properties widget itself

In the widget configuration, first select if you wish to display properties of the Current Object, or properties of a Linked Object:

- In case of displaying a Linked Object, first select the link that you wish to display.
- Displaying properties of a Linked Object is possible only if the Current Object is linked to only one object, which is possible either (1) in a one-to-one relationship; (2) in a many-to-one relationship, with the Current Object defined as the "many".
  - Example: Configuring a Flight object, it is possible to display the properties of the Aircraft linked to this flight (Flights to Aircraft has a many-to-one relationship). However, looking at the Aircraft object, it would not be possible to display properties of the linked Flight, as there are many Flight objects linked to any single Aircraft.

After choosing Current Object / Linked Object, there are 3 components to configure: Data, Sections, View Options.

Data

- Select what properties you wish to display:
  - All Properties: Display all properties that are defined as Prominent or Normal, but no Hidden properties. Allows exclusion of a specific set of properties using the `Properties to Exclude` selector (see below).
  - Prominent Properties: Display only the properties defined as Prominent in the Ontology Manager. Allows exclusion of a specific set of properties using the `properties to exclude` selector (see below).
  - Specific Properties: Opens a multi-select dropdown with all properties of that Linked Object, to select between.
  - No Properties: Do not display any property.
- Properties to Exclude: Select which properties you wish to exclude from display - only available for the first two options - of All Properties and Prominent Properties.

S

By default, there are three sections of properties, which are:

- Normal properties: All Normal and prominent;
- Long text properties: Only long text properties, which are defined as such in the Ontology Manager (see details above);
- Keyword properties: Displays keyword properties of object as tags. Only properties that were marked as keywords in the Ontology Manager, under “Edit Properties and Backing Dataset” would be displayed here.

You can remove or change the order of these sections. There are no additional types of sections to these three.

View options

- **User Editable:** Whether to allow users to edit properties. This requires two settings in the Ontology Manager in advance (see more details above):
  - Dataset level: Edits must be enabled on the object type.
  - Property level: There must be an inline edit action attached to the property.
- **Hide Undefined Values:** Hide all properties with undefined (null) values. If this toggle is off, the property will be displayed with “no value” text greyed out.
- **Include link to 'More properties':** This option will render a **View all** button that takes the user to a **Properties** tab within the same object view.

Common issues and notes

- The order in which properties are displayed is alphabetical. The main way to order properties in a different way is by using the "Sections" mentioned above, under the widget configuration.
- For more details on how to setup properties as Prominent, set properties as a Long Text or a Keyword, or making them user editable, see the documentation on editing properties.

Property Cards

The **Property Cards** widget lets you visualize cards with properties and aggregations over li

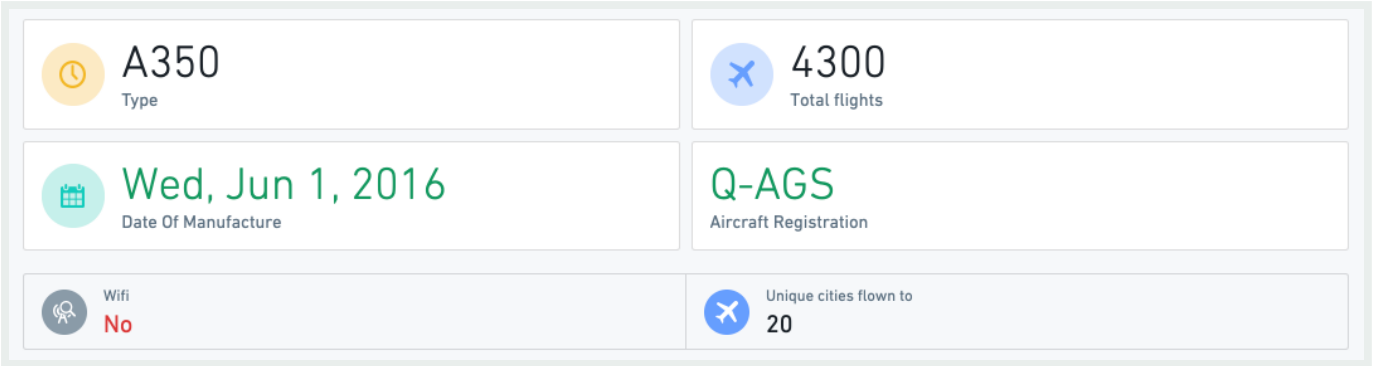
API Reference ↗

Use the Property Cards widget to display important properties (nume

Send feedback

s, strings, etc.), aggregations, statistics, metrics, KPIs, and any other key information for the

current object or for objects linked to it.



- This widget is affected by filters, which apply to any aggregations of linked objects.
- The Property Cards widget supports a rich UI for conditional formatting and value formatting, including both global (based on ontology-level configurations in the ontology editor) and local override options.

Property Card configuration

Start by adding a new card for each visualization of a property / linked object aggregation. Then, configure the overall layout of all cards within the widget.

Configuration per card

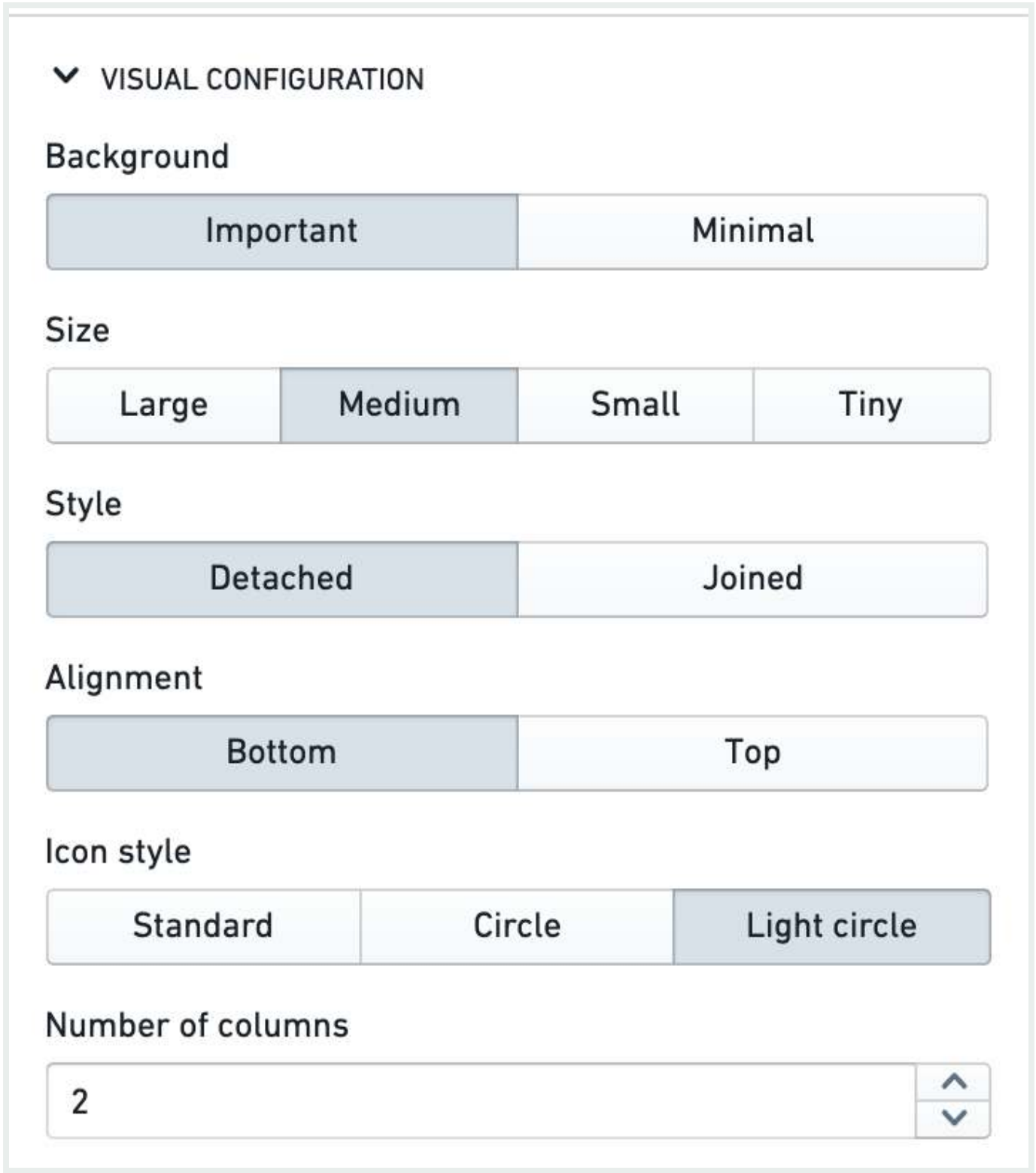
- Property cards can either display **a property** or an **aggregation of linked properties**.
  - When displaying a property, a card can show both properties of the current object, as well as properties of a linked object with a one-to-one relation or a many-to-one (e.g. if the current object is Flight, with a many-to-one relation to Aircraft, it would be possible to show the properties of the Aircraft object).
  - When displaying a linked object, the card can show aggregations of any object linked to the current object in a one-to-many or many-to-many relation.
- In case of an aggregations, select the type of aggregation - count, unique count / cardinality, average, min, max, sum.
- Add a label which will be displayed on the card, as well as an icon and icon color.
- **Conditional formatting and value formatting:** By default, the widget uses the global ng rules defined at the ontology level for that specific object and
  - This widget supports conditional formatting (e.g. color value to to predefined rules). Supported options are global formatting (as configured in

Ontology Manager (see conditional formatting documentation) and an optional local override. Note that this applies only for properties, not for aggregations.

- The Property Cards widget supports value formatting (e.g. `$3.6M` instead of `3,625,329`). Supported options are global value formatting (as configured in Ontology Manager; see value formatting documentation) and a local override. Note that this applies to properties and to aggregations of linked properties.
- For local overrides, both value and conditional formatting use the same UI as the ontology level formatting, check the ontology documentation (linked above) for further details.

Configuration for all cards on a Property Cards widget

- Once all property and aggregation cards are configured, use the layout configuration to determine styling of background, size, style, alignment and icon style of cards in display.
- If you wish to use different types of visual configurations for different cards, consider adding a new widget instance for each different configuration.



Common issues and notes

API Reference ↗ the supported widget for visualizing prominent properties, aggregations, statistics, metrics, KPIs and any other key informati ct and linked objects.

Send feedback

    - This widget is the preferred solution over any other widget for purposes of visualizing single key properties and aggregations. Consider using the Property Cards widget instead of the following widgets: Statistics, Context Stat Section, Linked Statistics, Advanced Statistics, or Property Plus.
- The Property Cards widget does not support Functions on Objects. If this support is required, consider using Workshop or Quiver and embedding it in the Object View using the dedicated widgets.

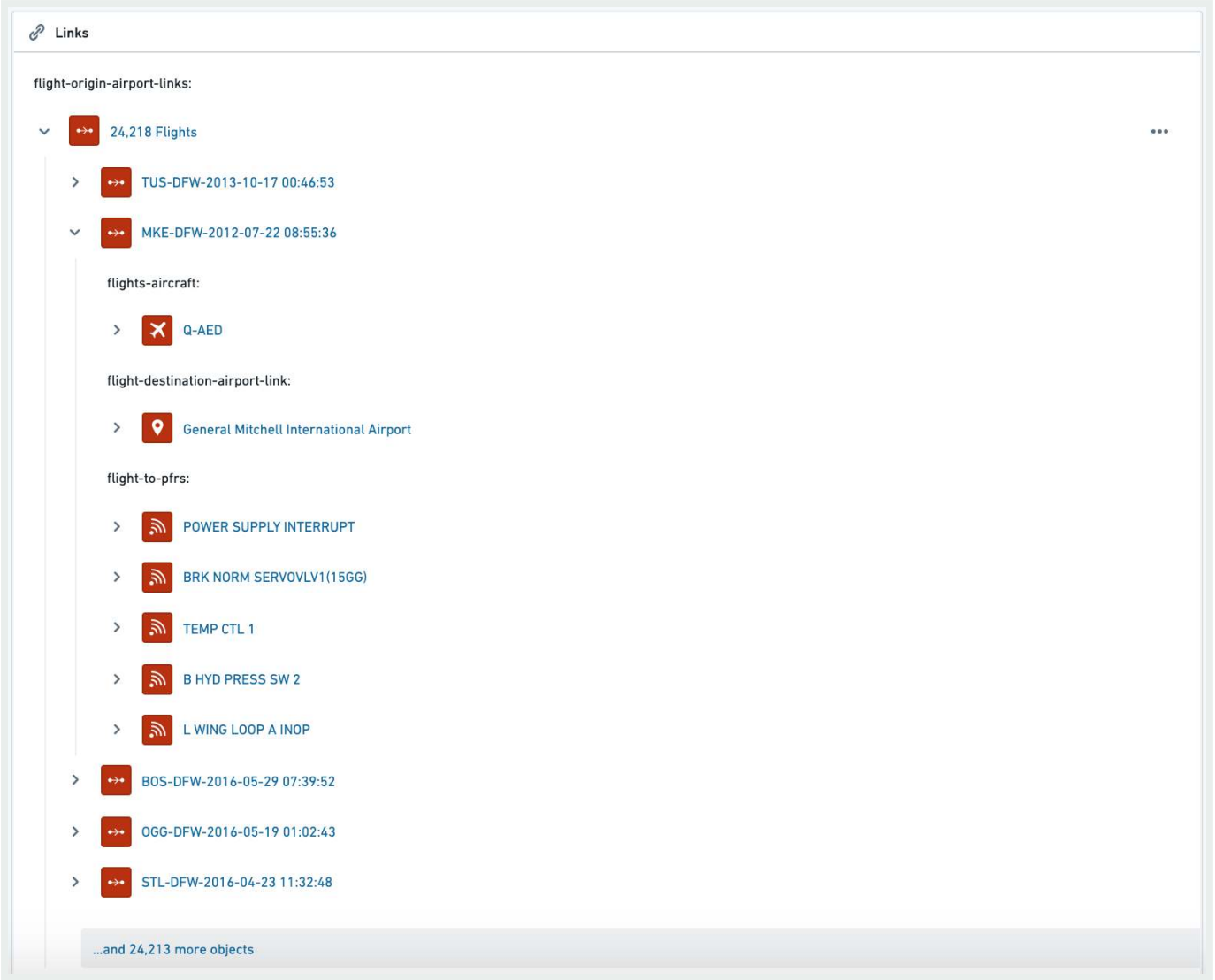
Links

The **Links** widget displays an object's links in a tree view, with the ability to traverse through Links and navigate to Linked Objects. This includes:

- Display a tree of all Linked Objects, with the ability to expand the view for each Linked Objects, to display all Objects Linked to it, and continue hopping through Links ("nested links");
- Hover over any Object to see a tooltip with all prominent properties of that Object;
- Click on any object to jump to its Object View.

See an example below, which includes 3 steps:

- Starting with an Airport object;
- Moving to all Linked Flights;
- Moving to all Linked Objects of one of these Flights: Aircraft, Destination Airport, all related Post-Flight Reviews.



Links widget configuration

- Link types (first links only) - allows you to determine which types of links will be displayed on the widget. Users can traverse through links and hop to other linked objects (e.g. from an Airport → to all Flights → to the Aircraft carrying these Flights → to the Airlines owning these Aircraft, etc.)
- Filter Nested Links by Type - this setting allows you to only display Linked Objects that are chosen under the “Link types (first links only)”.

Common issues and notes

API Reference ↗

- possible to sort or determine the order of
  - The list of values of Linked Objects (e.g. show all Linked Flights alphabetically).
  - The *types* of Linked Objects under this widget (e.g. show Linke

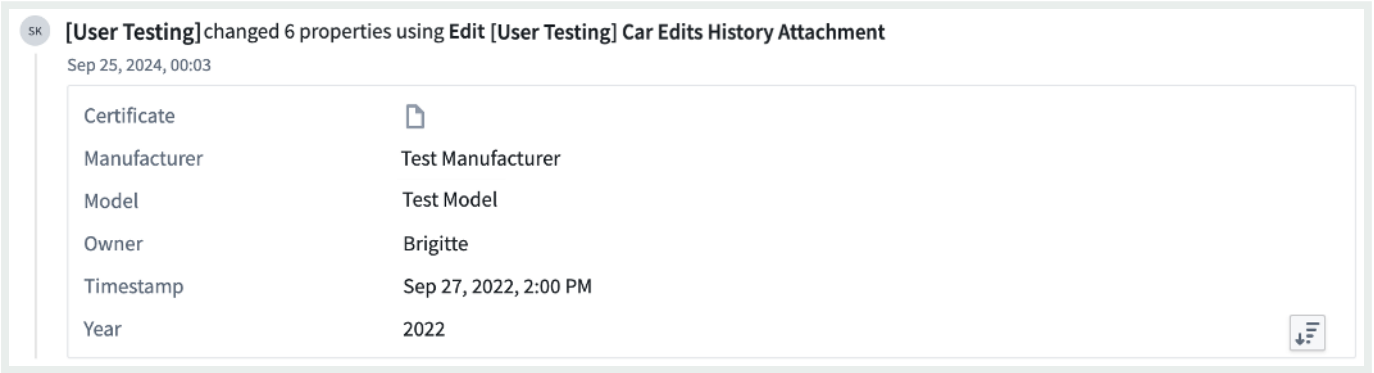
Send feedback

Linked Aircraft second).

- This widget is currently not affected by filters. Links displayed are always *all* Objects linked to the current Object.

Edit History

The **Edit History** widget displays a list view with the history of all edits made for the current object in view. If the object has never been edited or the subset of selected columns (more below) has no edits, this widget will display a `No Edits` message.



Edit History configuration

Option	Description
All Properties	Displays edits for all properties on the given object type.
Specific Properties	Specific Properties, Only displays edits for properties selected in the `Properties to Include` selector in the widget config.

Common issues and notes

- Only changes made via Actions to the object will be shown in Edit History. Changes made in the backing dataset or in the pipeline upstream will not be reflected on this widget.
- nly reflect the changes made to objects indexed into Object Storage v2 er edit history toggle is enabled within Ontology Manager.
- Tracking user edits for object types with marking properties is not e.

- Each submitted value is logged as an edit, even if it is coming from a default value configured in an action parameter.

PREVIOUS
← Configure profiles

NEXT
Visualization →

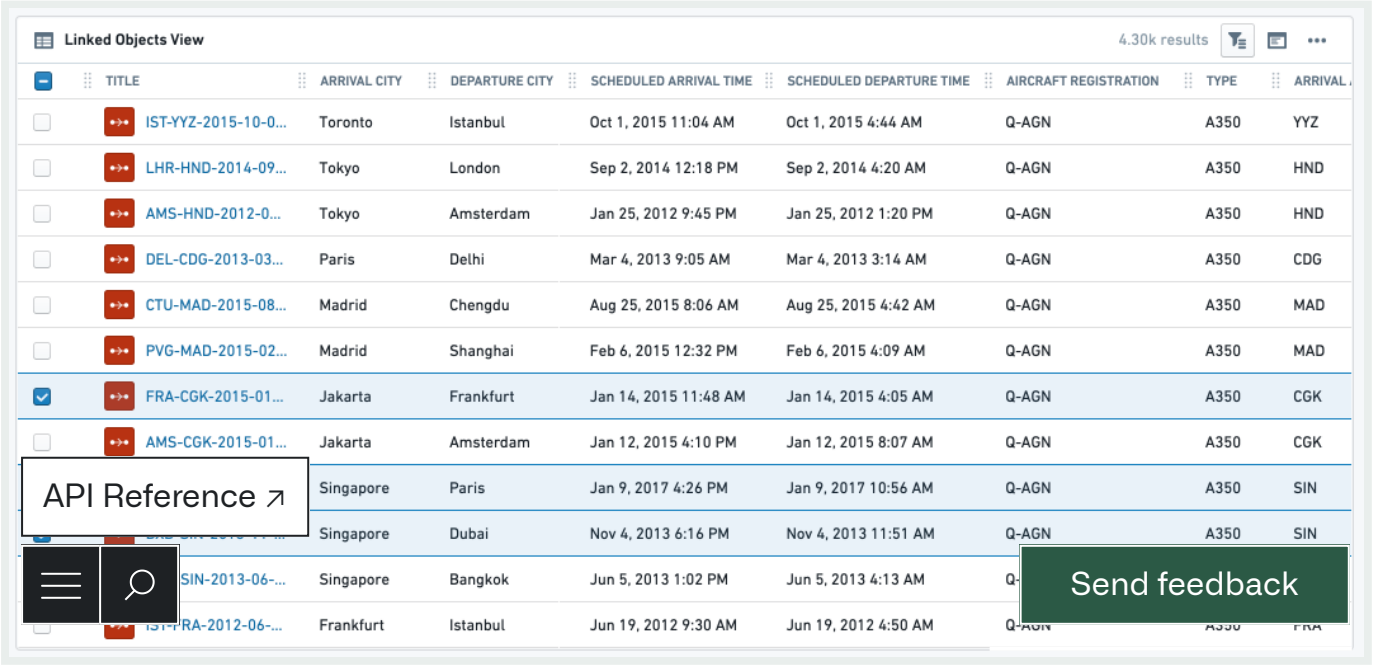
Visualization

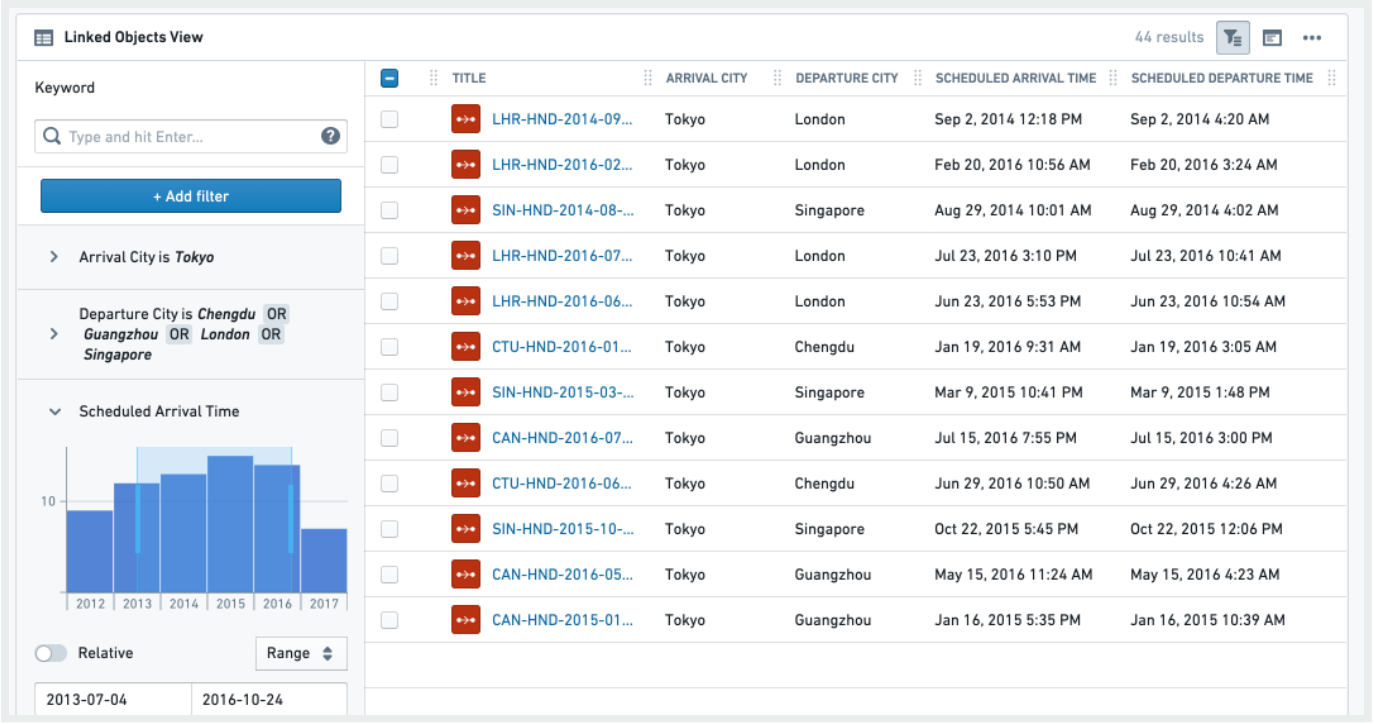
Visualization widgets display data related to the current object in charts, pivot tables, timelines, maps, and other visualizations, including simple text and metrics. Visualizations created in other Foundry apps can be embedded as well - see the Apps and Files category to learn more about embedding Slate applications, Contour boards, Foundry reports, Fusion spreadsheets, and Quiver time series or charts.

Linked Object View

This widget is mainly used to display a *table* view of all Linked Objects of a certain type along with their relevant properties. The table also supports selection of subset of linked objects to open in other Foundry apps or perform configured Object Actions. In addition to a table view, a simple list view and a card view are also available.

Example for Linked Object View (Table) without a Filter Sidebar, and with a Filter Sidebar:





Configuration

Settings tab

The configuration has 3 parts:

- Data
  - *Linked Object to display*: Select the Linked Object you wish to display. The widget configuration offers objects that are linked to the current object as defined in the Ontology configuration. Choose either an object type either via a direct link or through a transitive link using an intermediate object type.
  - *Properties to display*:
    - All Properties - display all properties that are defined as Prominent or Normal, but no Hidden properties. Allows excluding a specific set of properties using the `Properties to Exclude` selector (see below).
    - Prominent Properties - display only the properties defined as Prominent in the Ontology Manager. Allows exclusion of a specific set of properties using the `properties to exclude` selector (see below).
    - Specific Properties - opens a multi-select dropdown with all properties of that object type, to select between.
    - No Properties - do not display any property.

    - *Properties to Exclude*: select which properties you wish to exclude from display. Available for **All Properties** and **Prominent Properties** options.
- View Options - choose which view type will be displayed to the end-user by default. The end-user can still change to the other 2 views in the UI (on the widget header).
    - Table view - the most commonly used view, displays a table of all values, with quick filter search.
    - Card view - a condensed cards view of each Linked Object, with properties (as selected on 1b). This option does not have the full functionality of the table view. For example, it does not support selection or Object Actions.
        - Note: For the Card display option, there is a secondary configuration called “Card Width” which determines the visual density of these cards. It has no effect for the Table or List view options.
    - List view - a simple list view of all instances of the chosen Linked Object, without any option of selection or any display of linked properties.
- Search and Filtering - add search and filtering capabilities to the table view
    - Results limit - limit the maximum number of results to be shown. The more results you wish to display, the longer it would take the view to load. This, together with the minimum/maximum height configuration (see below on the Format tab) determine how much of the entire view this widget would take.
    - Enable search and filtering - toggle on to have search and filtering functionality on the widget. This includes either basic or advanced search functionality:
        - Basic - have a simple search bar for free text searches at widget header.
        - Advanced - have a full filter sidebar with detailed filtering options, including a toggle to expand or collapse the sidebar by default to end-users. Unlike the basic free-text search, this search allows more complex filtering based on property type (free-text and multi-select for strings, date range for timestamps, numeric for double/integer). This search bar is based on the legacy Object Explorer functionality and UI.

Format tab:

Under the format tab within the widget configuration, aside from defaults of Title, Icon, Info and Layout, you can control the minimum and maximum height of the section (optional).

C l Notes:

API Reference ↗

- This widget is *affected by filters* from other widgets - if the toggle i
publish filters to other widgets sharing cross-filtering with it, even

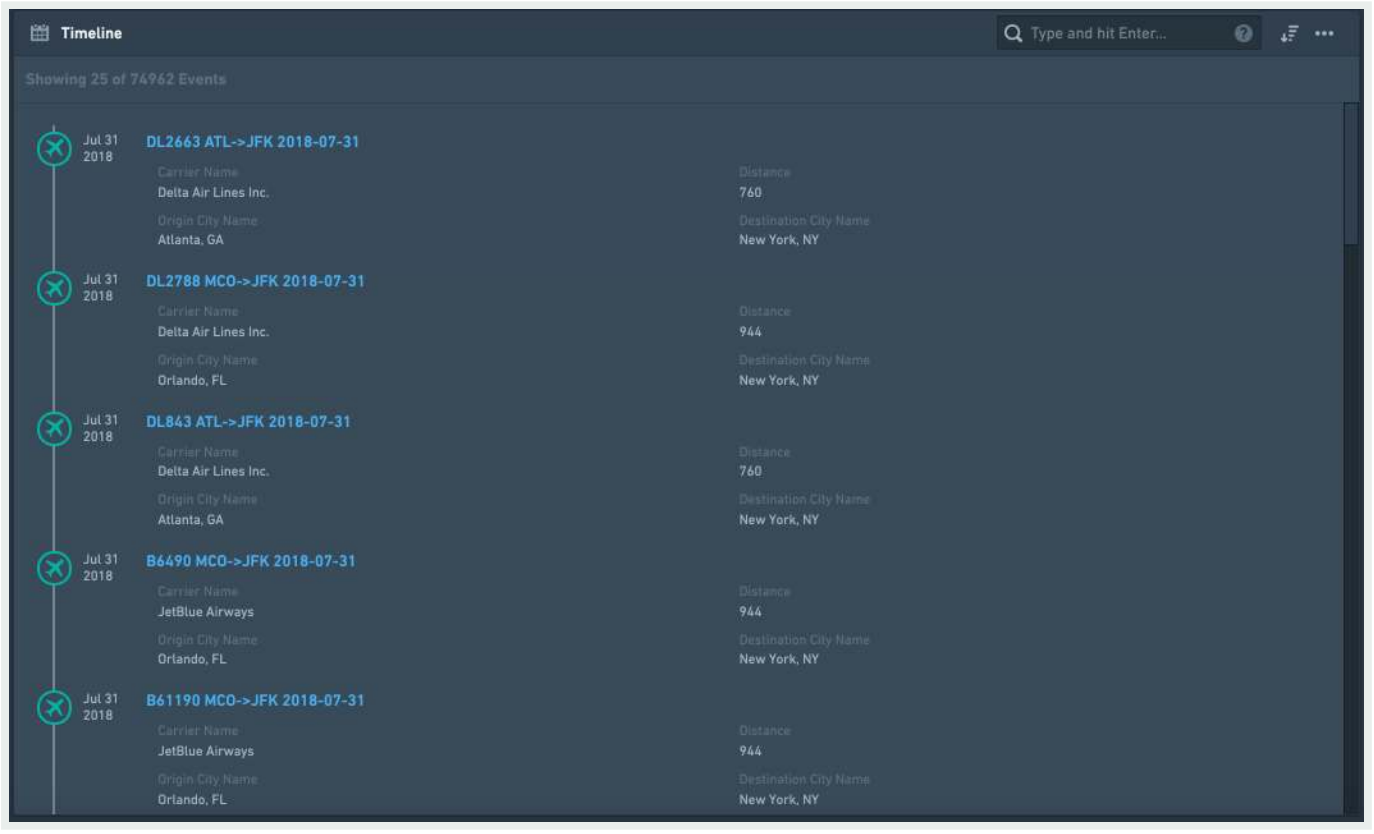
Send feedback

open. In effect, internal filters are limited to the context of the object table widget.
- Conditional Formatting and Value Renderers - Values in the table can be rendered with conditional formatting and value renderers.

Timeline

Create a chronological list of events, displayed top-down, sorted by date and time. This timeline is built of linked objects that must have at least one timestamp, and are usually events, that is an object with a start and end timestamp that define some duration. This widget can include one or more types of objects linked to the current object directly, or through a linked object. It can also display the same event by different date properties as in the examples below.

For a graph timeline, use the Grouped Events Timeline and Table widget.



Configuration

API Reference ↗

Settings tab

Send feedback

- **Add one or more Linked Objects** (**event**) - First, add at least one Linked object type that you wish to display on the timeline by clicking on “Add Event Type”. You can add many types of Linked Objects:
  - Display different object types on the timeline, that are completely different events. Example: Linked Objects to an Aircraft that are both Flights events and Aircraft Maintenance events.
  - Display events of the same Linked Object type, but by different date properties, or with different types of Links defined. Examples:
    - Flight events linked to an Airport object displayed on the same timeline by different date properties: (1) “Planned Arrival Time”, (2) “Actual Arrival Time”, (3) “Expected Departure Time”. The timeline will display a chronological list of all flights, and each flight will appear up to 3 times in this case.
    - An Airport object can have both Arriving Flights as well as Departing Flights that are linked to it displayed on the same timeline.
- **Select the type of event object you wish to add** - for each event object you wish to add, configure the details of that object type:
  - Select a linked object type - either directly Linked Object or an Extended Link.
    - Example of a directly Linked Object: current object is an Airport and you want to display a timeline of Arriving Flights and Departing Flights.
    - Example of an Extended object (2nd degree): current object is an Airport, and you want to display a timeline with all Aircraft objects that are linked to Arriving Flights linked to the Airport.
- **Define the date property used to sort events on the timeline** - select the property of the Linked Object you wish to sort the events on your timeline by. This property has to be a timestamp / date, so the configuration only suggests such properties.
- **Select which properties to display from the Linked Object**.
  - Select what properties you wish to display:
    - All Properties - not recommended if the Linked Object has many properties, as it would be crowded and hard to navigate through the timeline.
    - Prominent Properties - only the properties defined as Prominent in the Ontology Manager.
    - Specific Properties - opens a multi-select dropdown with all properties of that Linked Object, to select between.
    - No Properties - only the title of the Linked Object would be displayed on the
  - Select which properties you wish to exclude - only available for the first two options of All Properties and Prominent Properties.

- **Select which title would be displayed** for each event on your timeline. There are three options:
  - **Object Name** as title (e.g. name of the event) - this is the most commonly used option and the default.
  - **Date Property name** as title - this is useful if you wish to display events from a Linked Object using more than one property (e.g. “Planned Arrival Time”, “Actual Arrival Time”, “Expected Departure Time”, etc.). Reminder: in such a case, each link based on a different property, should be configured separately within the Timeline widget - see bullet 1 above).
  - **Custom Name** - an option to add a free-text description for all Linked Objects of that type. E.g. if you link both Arriving Flights and Departing Flights under the same timeline, you may want to describe all Arriving Flights with the title “Arrival” and all Departing Flights with “Departure”.

Format tab

Under the format tab within the widget configuration, aside from defaults of Title, Icon, Info and Layout, you can control the minimum and maximum height of the section (optional).

Common issues and notes

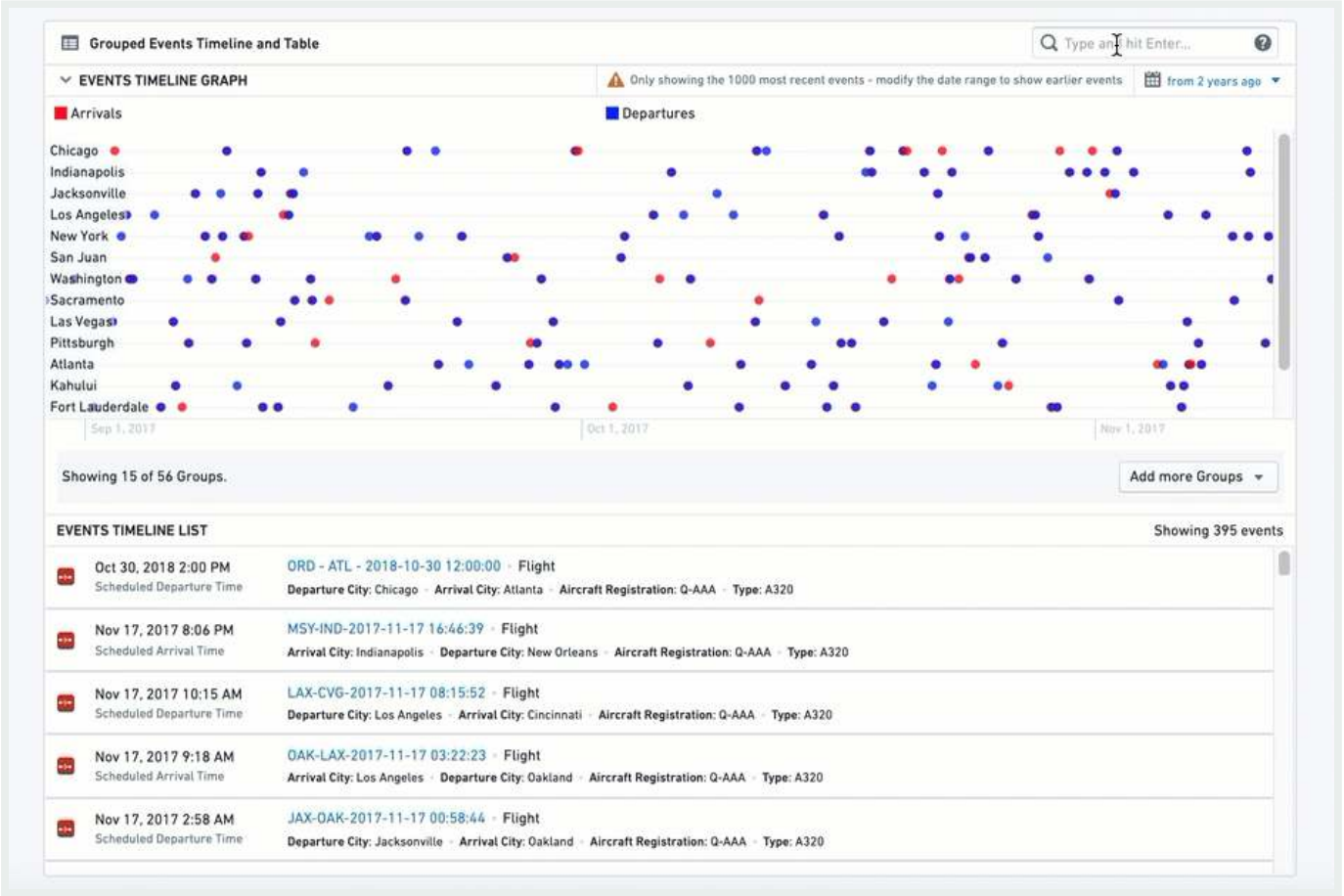
- This widget is searchable and sortable (oldest first, newest first). However, it doesn't support filtering, so it is not affected by filters. For filtering functionality, consider using the Grouped Events Timeline and Table widget.
- If you wish to display events on a graphic timeline (and not just be a chronological list), consider using the Grouped Events Timeline and Table widget.
- If you wish to display a Gantt chart style timeline, there is a “Linked Objects Gantt Chart” widget. If this is not available, contact your Palantir representative for more details. This is relevant for displaying a smaller number of events, with distinct start and end dates, such as a project timeline.
- For Timeseries visualizations, contact your Palantir team for more details about Quiver, which can be embedded in an Object View. You can also use Charts widgets (or embed charts from Contour or Quiver charts), with a date/time property on the X-axis.
- [illegible]ng dates will not be displayed on the events list. There will be a [illegible] the user about missing events.
- For performance reasons, only 50 Linked Objects are displayed b[illegible] scrolls down through the list, additional objects will appear, 100 at a time.

Grouped Events Timeline and Table

This widget includes two components: (1) a timeline of Linked Objects, and (2) a table listing the Linked Objects underneath. Linked objects can be grouped by properties and filtered by text, date range and external filters.

Grouped events are plotted on separate parallel lines, with each line including only events that have a property with a certain value, similar to a pivot table. In the example below, each city has a different line, with all flights that are arriving (red) or departing (blue) from that city on that line.

This widget can include one or more types of objects linked to the current object directly, or through a linked object. It can also display the same event by different date properties (examples below).



API Reference ↗

Send feedback

Settings

- **Add one or more Linked Objects** (**event**) - First, add at least one Linked Object Type that you wish to display on the timeline by clicking on “Add Item”. You can add many types of Linked Object:
  - Display different object types on the timeline, that are completely different events. Example: Linked Objects to an Aircraft that are both Flights events and Maintenance events.
  - Display events of the same Linked Object type, but by different date properties, or with different types of Links defined. Examples:
    - Flights events linked to an Airport object displayed on the same timeline by different date properties: (1) “Planned Arrival Time”, (2) “Actual Arrival Time”, (3) “Expected Departure Time”. Each flight will appear up to 3 times in this case.
    - An Airport object can have both Arriving Flights as well as Departing Flights that are linked to it displayed on the same timeline.
- **Select the type of event object you wish to add** - for each event object you wish to add, configure the details of that object type:
  - Select Linked Object - either directly Linked Object or an Extended Link.
    - Example of a directly Linked Object: current object is an Airport and you want to display a timeline of Arriving Flights and Departing Flights.
    - Example of an Extended object (2nd degree): current object is an Airport, and you want to display a timeline of all Aircraft objects that are linked to Arriving Flights linked to the Airport.
- **Configure visual format options**
  - Max Number of Events - this option is currently not functional.
  - Color - select the color for the all events under this item’s configuration. Use the Blueprint standard colors ↗ or basic color names - “Blue”, “Red”, “Yellow”, etc.
  - Series Name - the description of all events under this timeline, which will be displayed on the legend of the graphic timeline.
- **Define the date property used to sort events on the timeline** - select by which property of the Linked Object you wish to use to display the events on your timeline. This property has to be a timestamp / date, so the configuration only suggests such properties.
- **Group Events By** - select how to group the events on the graphic timeline. Each group would be displayed as a separate horizontal line of events in the graphic timeline, stacked one on top of the other. There are two options:
  - Constant - current item would be grouped on a single timeline, with a title defined as ue. In this case, if you wish to add other timelines as separate lines, you must define additional items.
  - Property - split the events to a number of separate timelines, w each timeline according to the different values of that property.

        - Example: displaying a timeline of all Arriving Flights of an Airport object, one could select the “Airline Name” property and have a separate timeline for each Airline with all flights of that Airline on its dedicated timeline. The list below would include a chronological list of all Flights events from all Airlines.
- **Properties Displayed in List** - select the properties you wish to display on the second part of the widget, the List:
    - Select what properties you wish to display:
        - All Properties - not recommended if the Linked Object has many properties, as it would be crowded and hard to navigate through the timeline.
        - Prominent Properties - only the properties defined as Prominent in the Ontology Manager.
        - Specific Properties - opens a multi-select dropdown with all properties of that Linked Object. Choose specific properties to display.
        - No Properties - only the title of the Linked Object is displayed on the timeline.
    - Property Filter - an optional toggle to add a filter based on a single property, to include only specific values. These values must be explicitly typed as case-sensitive free-text strings.
- Additional configurations for the graphic timeline (these only apply to the timeline component and have no effect on the table component)
    - **Groups Displayed by Default** - how many timelines, or which ones exactly, should be displayed as separate lines in the timeline. In both cases, the user would have a multi-select helper with all other groups on it.
        - Max Number - the maximal number of different groups and different timelines would be displayed in the graphic timeline, as a number (Value).
        - Explicit List - write the exact list of values of the groups you wish to display (text, case sensitive).
    - **Sort Groups By** - the timeline can have multiple horizontal lines, and this allows you to sort these lines by 1 of 3 ways:
        - Configuration Order - the order in which you configured the different items
        - Name - alphabetical order of all groups, from A at the top - to Z at the bottom
        - Most Recent Event - the group with the most recent event at the top
    - **Max Graph Height** - set the maximum height of the graphic timeline in the widget (in pixels). To set the height of the entire widget, see the Sizing options under the Format tab in the configuration.

Format

Under the format tab, aside from defaults of Title, Icon, Info and Layout, you can control the minimum and maximum height of the section. Setting either is optional.

Common Issues and Notes

- The end use can filter the timeline using a free-text filter or a date range. Note that these filters are not published and do *not* affect other widgets.
- This widget is affected by other filtering widgets, if cross-section filtering is enabled.

PREVIOUS
← Properties and Links

NEXT
Filtering →

Filtering

Many Object Views require the user to filter on visual components such as charts, tables, aggregative metrics and KPIs, and so on. **Filter Widgets** allow users to apply different types of filters in order to drill-down into a specific subset of Linked Objects on that Object View.

This category includes several Filter Widgets:

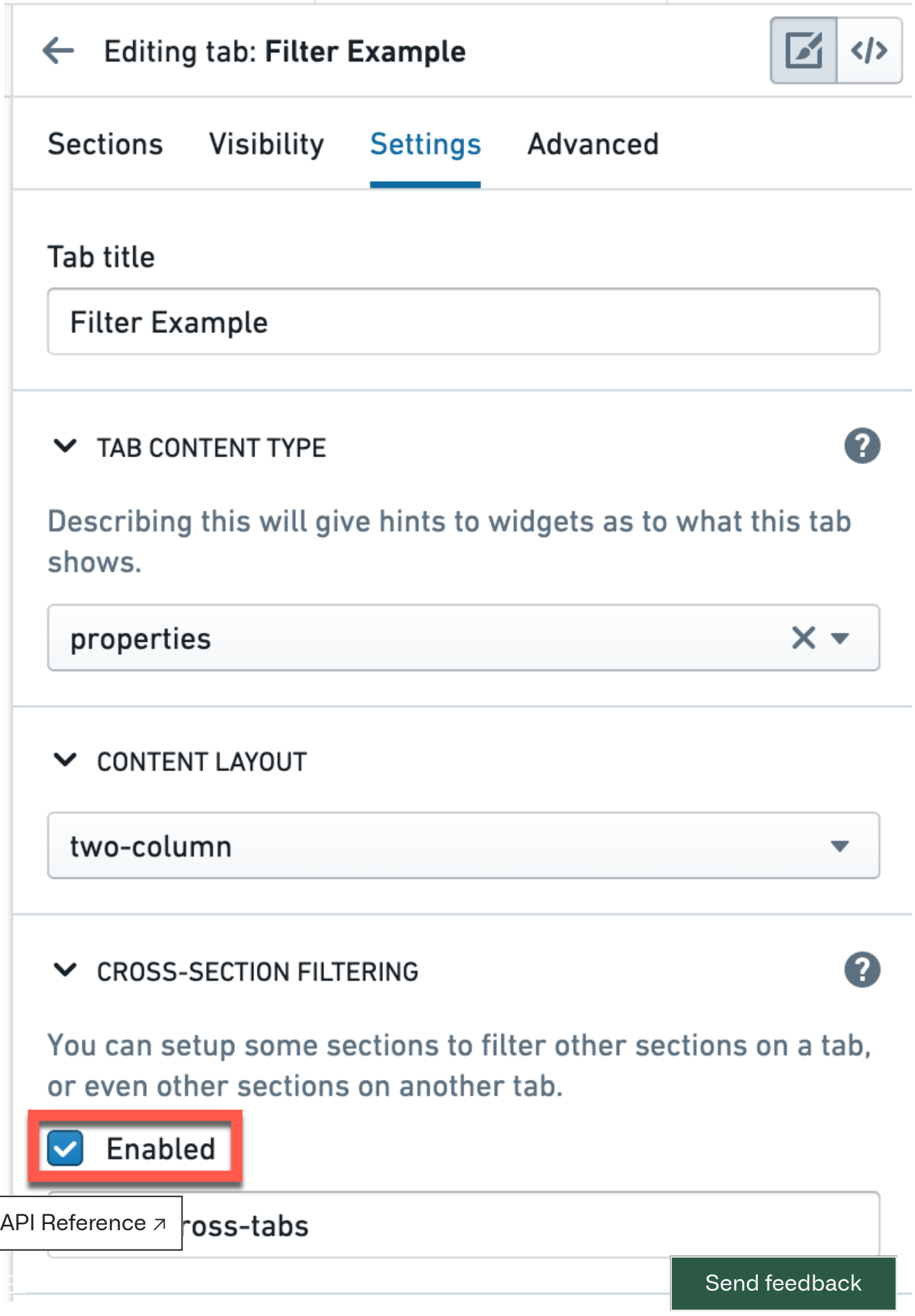
- Filter by pre-defined properties, including strings, numerical values, and dates by using the Multiselect Filter, Dropdown Filter, Button Filter, or Date Range Filter.
  - Use the Linked Object Filter Sidebar for more flexible filtering functionality
- Use the Filter Sandbox Container to have certain filters applied only on part of the current view.
- Use the Filter Container for pre-defined filters applied only on a subset of widgets within the Object View.
- Use the Active Filters for a summary of all filters currently applied on your Object View.

Apart from these core Filtering Widgets, which are dedicated to filtering only, there are additional widgets that are dedicated to visualization, but also allow some filtering, such as Charts. The Conditional Container enables displaying/hiding content according to filters that the user interacts with.

Common Issues and Notes:

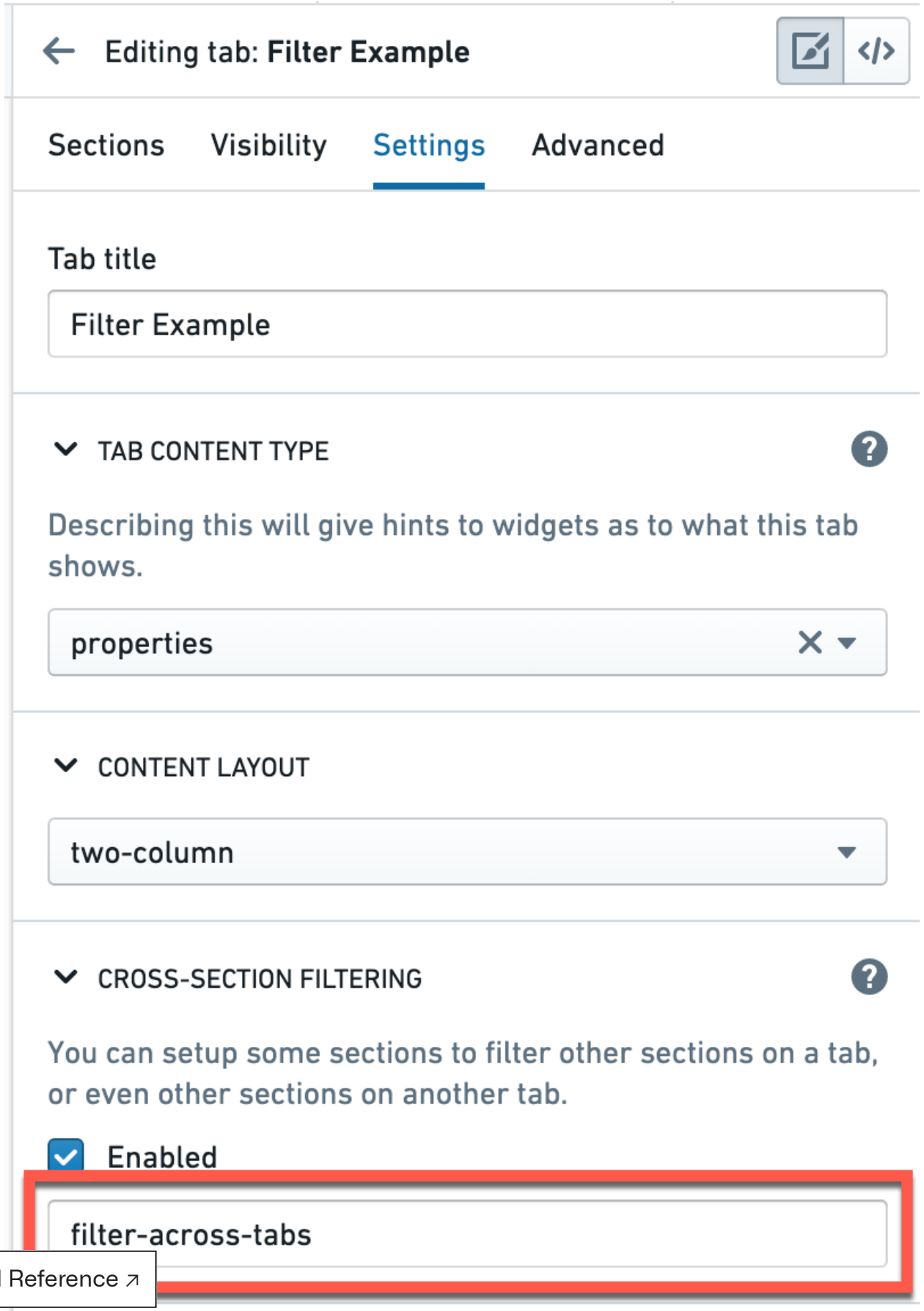
- In order to activate filters to apply across different widgets on a single tab of an Object View, or even across tabs, **you have to mark the checkbox of "cross-filtering"** on the right-bar editor, under "Settings". Filters will not apply in tabs that were not marked with

this checkbox.



- In order to have a filter applied **across different tabs**, make sure that the “filterSet value“ under the tab Settings has an identical text value across all tabs you wish to filter across. This value is case-sensitive.
    - In that case, the filters that use the same filterSet value will be affected by the same active filters. For example, if Tab A and Tab B share the same filterSet value, any filters

applied on Tab A will be applied on Tab B and vice versa.



API Reference ↗

- In all filter configurations, you will select a Linked Object to the ob
  currently editing, and not the object that you are editing itself.

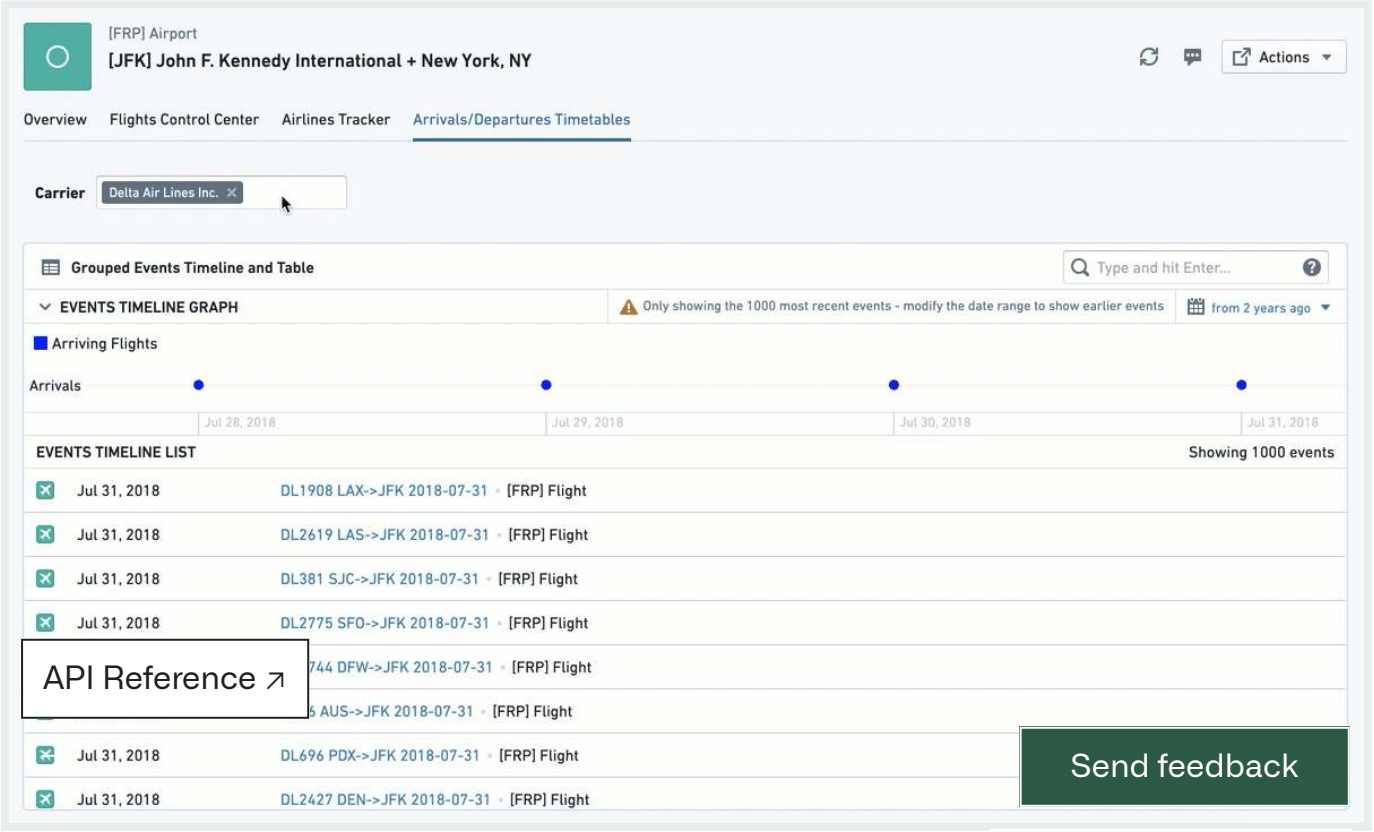
Send feedback

    - Example: If you're configuring the Object View of an "Airport", with "Flights" connected to it, you would probably have different visuals on flights (timelines, charts, list of all flights in a table), and would want to set your filters on different properties of "Flights" (e.g. filter per airline, filter per date, filter per origin city).
    - Therefore, before you configure a filter, your current object needs to be linked to another object. Setting up the links is done in the Ontology Manager.
- Most Filter Widgets only add an optional functionality that allows the user to apply filters locally for the current view of the object they currently browse.
    - Most of the Filter Widgets do not enable you to pre-configure filters to be active by default, such that would narrow down the view for the user (Filter Container is an exception, as it does allow to set up pre-configured filters).
    - The user would be able to activate filters on their current view of the current object, but once they move to a different object or refresh, these filters would not apply.

Multiselect Filter

The multiselect filter allows users to filter the Object View by multiple values, with an "OR" statement between them. It keeps all entries that satisfy at least one of the chosen values.

Once configured, this is how it looks:



Configuration

Note: Each setting that is always required is signed with a (*) sign next to it.

- [Required] **Linked Object to Filter:** Choose the Linked Object for this filter to apply on. The widget configuration would offer you only objects linked to the object you are currently configuring (as defined in the Ontology Manager).
  - Example: In an *Airport* Object View, which includes views of *Flights* objects, involving both types of links to the *Airport*: "*Arriving Flights*" and "*Departing Flights*". If you set up the Multiselect Filter with the Linked Object "*Arriving Flights*", it would apply to all other widgets that are also involving that same object "*Arriving Flights*".
- [Required] **Property to Filter:** Choose the property you want to filter down by: (1) you first need to choose the object type for the filter; and then (2) choose the property of that object by which you want to filter.
  - Once selected, this list of values will be available to users in the Object View, in a filter box with a multi-select dropdown. All other widgets affected by this filter and sharing the object with this filter, would be filtered down to only object instances that have the property values (one or many) chosen.
  - Example: Looking at an airport object with arriving flights from 20 origins, a user has an additional widget showing a table of all flights arriving from these origins. The user chooses a multiselect filter for the object `Arriving flights`, and then chooses the property `Origin City` for the `Flights` object. The dropdown filter would present to the user all 20 origins. Once the user chooses a subset of origins, the table widget would only show arriving flights from this subset of origins.
- [Optional] **Label For Filter:** will be displayed to users in the Object View, next to the filter box.
- [Optional] **Maximum Number Of Filter Options:** determines how many distinct values of the property to filter will be displayed. Set by default to 100.

Common Issues and Notes:

- Once configured, you would need to mark the checkbox "allow cross-filtering" in the configuration editor settings on the right side (see details above) in order for the filter to affect the Object View.

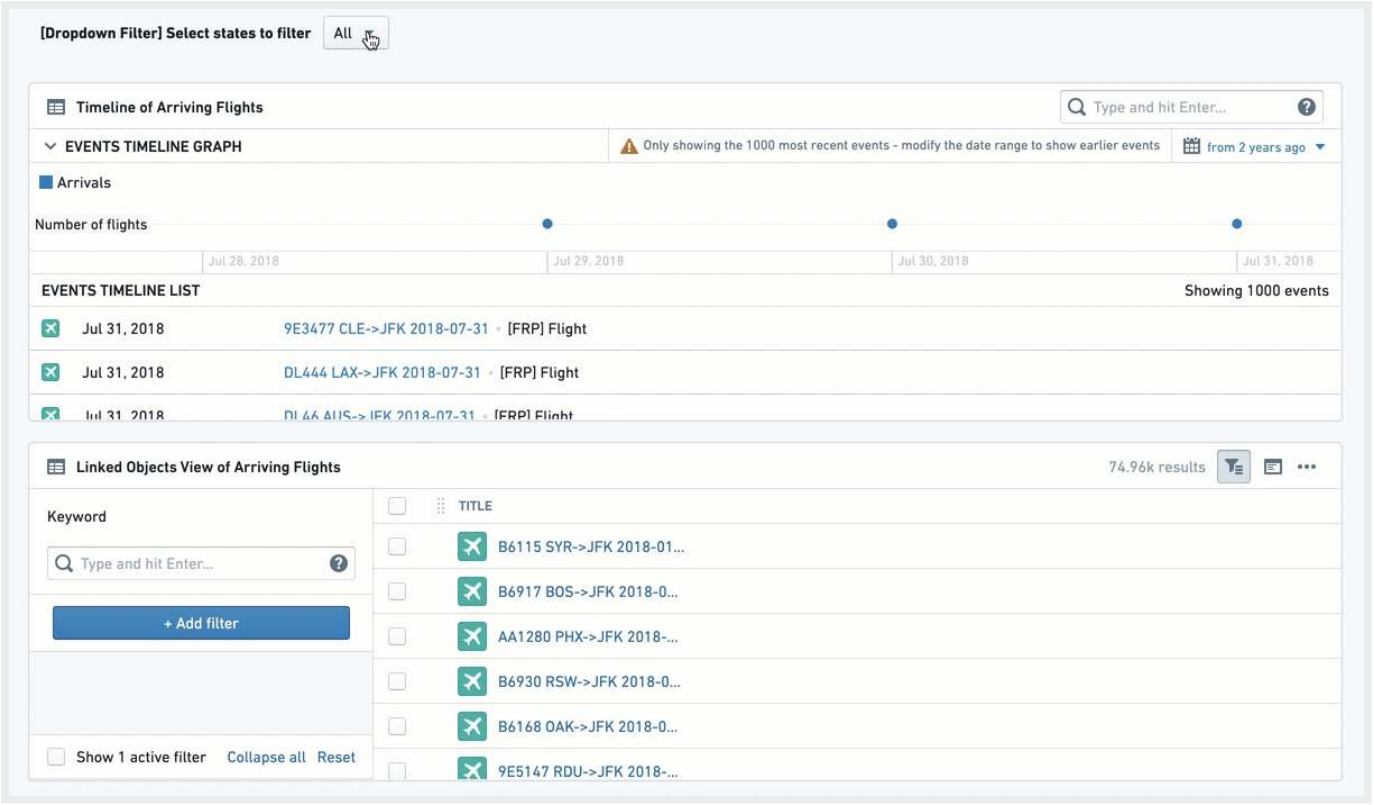
API Reference ↗

Dropdown Filter

Send feedback

This filter enables having one or more filters with a single-selection dropdown of a single property, allowing the user to filter other widgets in the Object View using a dropdown of values.

Once configured, this is how it looks:



Configuration

For each dropdown filter option you would like to configure, you need to first click on “Add Item”. You will be able to add several dropdown filters under a single widget by just clicking “Add Item” again (they will all appear one next to the other).

Once an item is added, the configuration menu offers two options of "Filter Type":

Option 1 - Dynamic List:

This is the simpler option, allowing an easy selection of an object and a property for the dropdown like most filter widgets:

API Reference ↗

- [Required] **Linked Object to Filter:** Select the Linked Object for th
  The widget configuration would offer you only objects linked to th

Send feedback

currently configuring (as defined in the Ontology Manager).

- [Required] **Property to Filter:** Select the property you want to filter down by: (I) you first need to select the object type for the filter; and then (II) select the property of that object that you wish to filter by.
  - Once selected, all other widgets affected by this filter and related to the same object type as this filter, would filter down from all Linked Objects to only those where their property has the same value as selected in the dropdown.
- [Optional] **Label For Filter:** Will be displayed to users in the Object View, next to the filter box.
- **Enable 'All' filter option toggle:** If the filter needs to be mandatory, keep this toggle off, to force users to select one value. If it can have all values selected (do not filter by any of them), turn the toggle on. If you wish to select multiple values, select the Multiselect Filter widget instead.

Option 2 - Value List:

This is the more complex option, specifically for cases where you wish to define an exact list of values to filter from, which would be a sub-list of the full dropdown list. You would need to manually type each and every value you wish to include in the Dropdown Filter for the user. How to configure this:

- [Required] **Linked Object to Filter:** Select the Linked Object for this filter to apply on.
- [Required] **Property to Filter:** Select the property you want to filter down by: (I) you first need to select the object type for the filter; and then (II) select the property of that object that you wish to filter by.
  - Once selected, all other widgets affected by this filter and related to the same object type as this filter, would filter down from all Linked Objects to only those where their property has the same value as selected in the dropdown.
- [Required] **Filter property value(s):** You need to manually type down each and every value that you wish to include in this dropdown filter for the user, and click on “Create” to add it.
  - Values entered are case-sensitive and will appear in the order they are entered; once entered, values cannot be re-arranged except by removing values (click on the ‘X’) and re-entering them.
  - The number of displayed values is limited to 100.
  - [illegible] is offered by default. You can remove this option and re-add it if desired (“All”, case-sensitive).

- [Optional] **Label For Filter:** will be displayed to users in the Object View, next to the filter box.

Common Issues and Notes:

- Consider using the Multiselect filter instead of the Dropdown filter if (a) you want the functionality of selecting more than a single option; (b) you don't need multiple dropdowns (though this could also be solved with a filter container).
- The interaction of the Dropdown widget with the "Active Widget" is limited. Even if "Enable 'All' filter option" is toggled and enabled, users will still not be able to "Clear filter", unless they explicitly select "All" under the Multiselect filter.
- Once configured, you would need to mark the checkbox "allow cross-filtering" in the configuration editor settings on the right side (see details above) in order for the filter to affect the Object View.
- There is a limit to the number of dropdown filter boxes the UI is able to present under a single dropdown filter widget (around 7-8 different dropdowns). In the unlikely event that you need more dropdown filters for a single Object View, use an additional Dropdown filter or use a Filter Container.

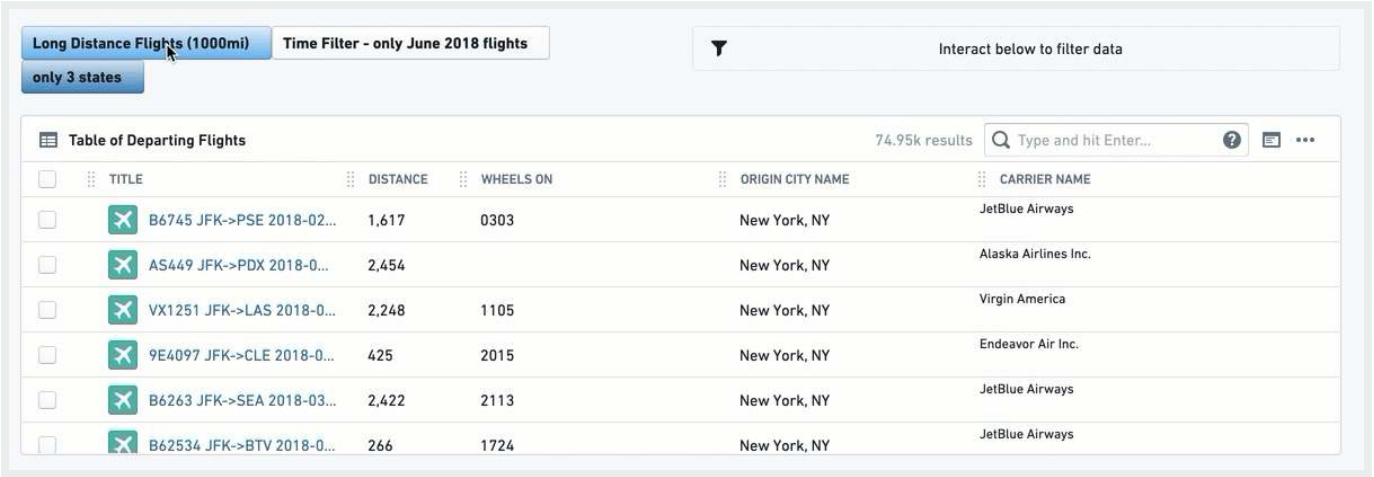
Button Filter

This widget creates a button that with a single click filters to a pre-defined set of values: either text (string) values, a range of numerical values, or a range of dates. A second click on the button un-filters the selection.

This is a rigid filter; once configured by the Object View Editor, there is no configuration choice available to end users.

Note that the button changes color as you select or unselect it. You can add an Active Filters to make the state visually clear.

Once configured, this is how it looks:



Configuration

First, select a button filter type:

- Value List: To filter text properties (strings).
- Range: To filter numerical properties (like integers or doubles).
- Date Range: To filter down to a list of date properties.

For all 3 types of button filters, you will have the following:

- [Required] **Object to Filter:** Select the Linked Object for this filter to apply on.
- [Required] **Property to Filter:** select the property you want to filter down by: (I) you first need to select the object type for the filter; and then (II) select the property of that object that you wish to filter by.
  - Once selected, all other widgets affected by this filter and related to the same object type as this filter, would filter down from all Linked Objects to only those objects with properties value that the filter condition applies to.
- [Required] From this point, the setting of the button is different per button type:
  - **Value List:** manually type in the exact list of values you wish to filter by, with an “OR” statement between them. Note that it is case-sensitive and the order cannot be simply re-arranged. The way to re-arrange values is by deleting and re-writing the entire list of values. Note that if you apply both the “All” option as well as another X, then the filter will only apply on value X.
  - both lower and upper boundary of numerical values. Enter a value to at least one of them:
    - Lowerbound = greater or equal to;

        - Upperbound = lower or equal to;
        - Fill both to have a range filter.
        - Make sure to delete the 0 from the Upperbound if you wish to have a “greater than”, and vice versa for “lower than”.
    - **Date Range:** offers a toggle with 2 options - exact dates (“from 1\1\2019 to 31\12\2019”) or relative to current time (“Last Month” or “Between two years ago and one year ago”). The configuration itself is done with a standard calendar date picker.
- [Optional] **Button Label:** the text to display on the button. It’s optional, but it's best practice to create a button with a label.
- [Optional] **Button color:** The default color is grey. To select a different color for the button, use the Blueprint standard colors (see https://blueprintjs.com/docs/#core/colors) or use the internal Palantir Blueprint library. Basic colors (“Blue”, “Red”, “Yellow”, etc.) would also work.
    - An un-selected button would have a faded color. Only once clicked, it would change to the chosen color.
    - The Date Range Button does not have an option to pick a Button Color.

Common Issues and Notes:

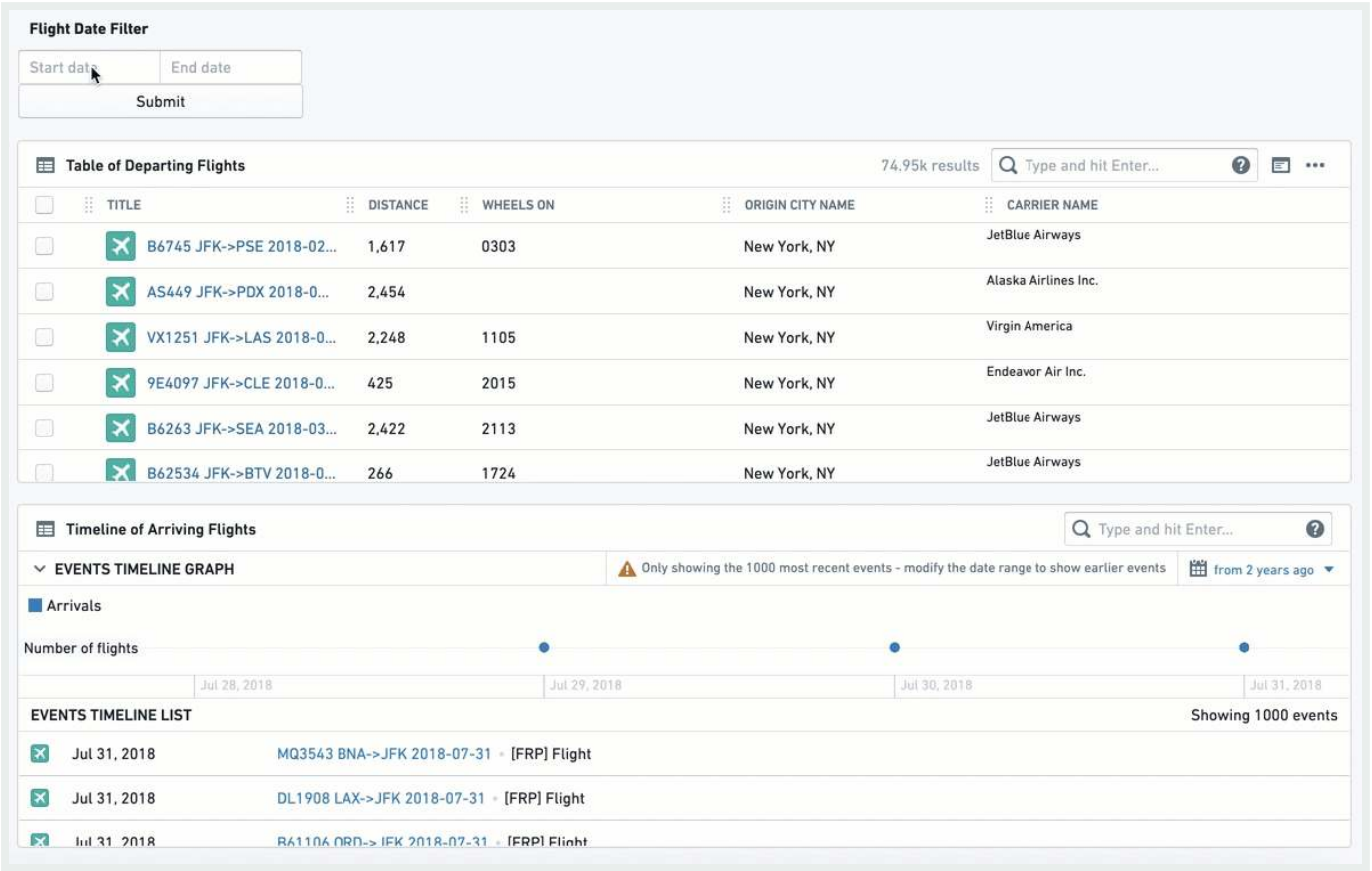
- To grant users with more flexibility, consider using:
    - “Date Range Filter“ instead of the ”Date Range Button Filter“;
    - “Multiselect Filter” or the “Dropdown Filter” instead of the “Value List Button Filter”.
- Note that there are 2 ways to indicate what a button filter does: (I) label the button well; (II) add an “Active Filter” widget, which shows all filters, including the actual values filtered by clicking on the button filters.
- The Button Filter is off by default, so that it has to be clicked for the filter to apply.
- Once configured, you would need to mark the checkbox “allow cross-filtering” in the configuration editor settings on the right side (see details above) in order for the filter to affect the Object View.

Date Range Filter

The Date Range Filter allows users to filter down their Object View to a range of dates, based o[API Reference ↗]pperty. Once configured, any widget in the Object View is affected once a date range is chosen and the user selects **Submit**. Views will only display object instances where the date in the chosen date property is within the range.

Note that you should insert the object ID of the object that you want to filter down, which is usually a Linked Object to the object that you are currently configuring.

Once configured, this is how it looks:



Configuration

- [Required] **Object to Filter:** Select the Linked Object for this filter to apply on.
- [Required] **Property to Filter:** Select the property you want to filter down by: (I) you first need to select the object type for the filter; and then (II) select the property of that object that you wish to filter by.
  - Once selected, all other widgets affected by this filter and related to the same object type as this filter, would filter down from all Linked Objects to only those where the chosen date property is within this dates range.
- [Optional] **Label For Filter:** will be displayed to users in the Object View, next to the filter box.

C API Reference ↗ **l Notes:**

- Once configured, you would need to mark the checkbox “allow cr
configuration editor settings on the right side (see details above) in order for the filter to

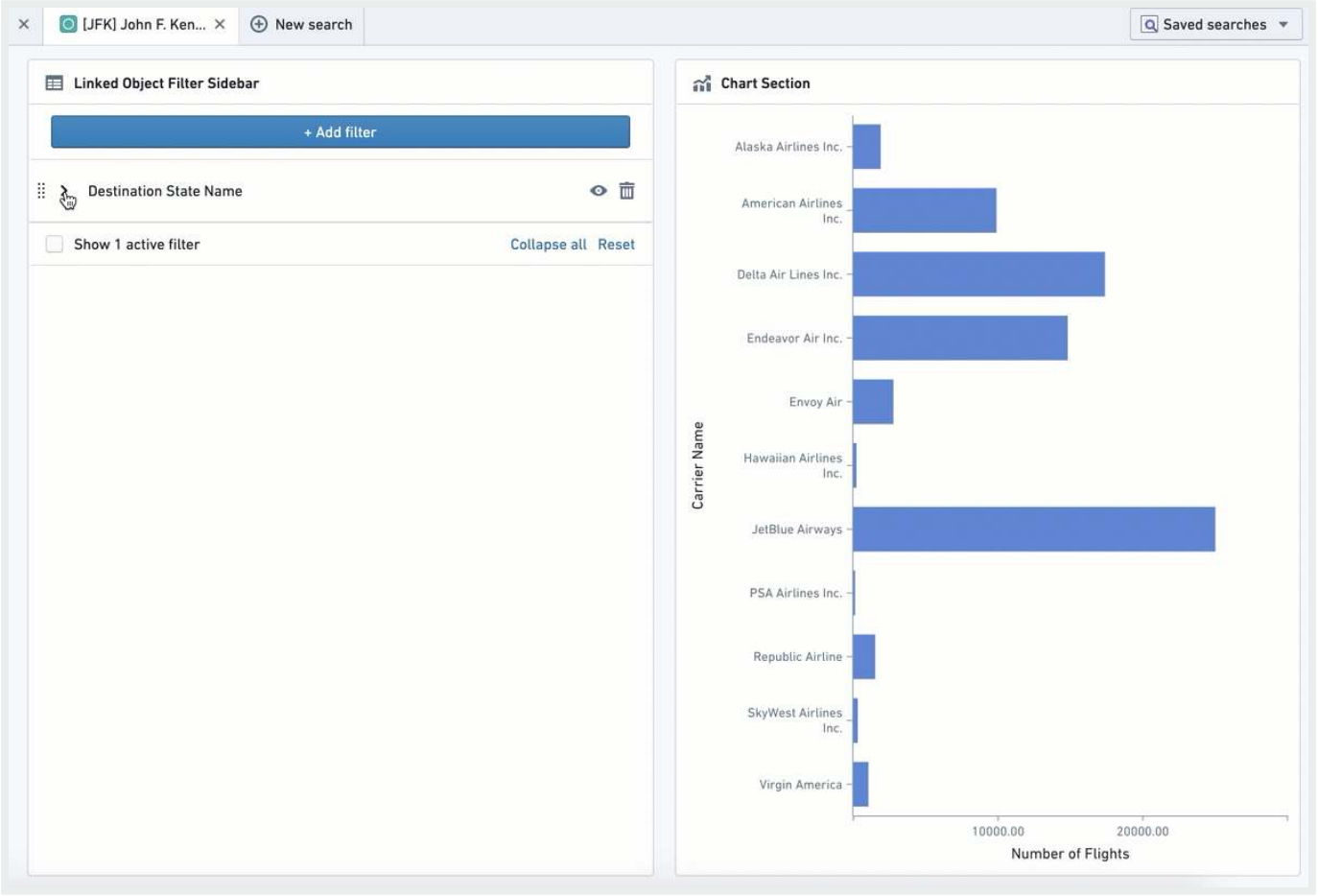
affect the Object View.

Linked Object Filter Sidebar

This widget creates a flexible filtering experience, allowing the user to filter the current Object View by any specific set of values out of all properties of that Linked Object.

This widget enables a high degree of choice, but also higher complexity for the user. It requires the user to understand and know the properties of the Linked Object, while all other Filter Widgets (e.g. Dropdown Filter, Button Filter) pre-configure them.

Once configured, this is how it looks:



Configuration

API Reference ↗

Configuration of this widget only requires selecting the Linked Object [illegible] Object View to filter by. The rest of the configuration, i.e. choosing the [illegible] and values to filter by, is completely up to the user and would apply locally on their view only.

Send feedback

Once the user refreshes the page or turns to a different object, they would need to re-configure these filters again.

Common Issues and Notes:

- All values chosen for a certain property would have an “OR” statement between them. Values of different properties will have an “AND” statement between them.
  - Example: When working with an “Airport” object, the Object View Editor chose to add a Linked Object Filter Sidebar for the Linked Object “Arriving Flights”. A user can select to filter down by Property “Month” and select only January to June; as well as filter by “Carrier Name” and select Delta Airlines and United Airlines. The view would be filtered down to all arriving flights within that airport that are between January and June (January or February ... or June), and are either United or Delta.

Filter Sandbox Container

The Filter Sandbox Container enables you to organize widgets into a container such that all filters inside it are sandboxed, meaning they only affect and are affected by other widgets inside this container.

Configuration

Add the Filter Sandbox Container widget, then add existing widgets into the container by clicking on "Add Item". Select the widget to add from the widget selector.

The other configuration option is the filter set identifier. By default, a new unique value is generated for every new filter sandbox. It can be changed if you want to share filters with:

- Filter Sandbox Container - set the same value of filter set identifier for both of them.
- All widgets on the other tab - enable cross-section filtering on the target tab and set the same value of filter set identifier for it.

Common Issues and Notes:

API Reference ↗

- Cross-filtering (filters affecting and being affected by other widgets inside the container) is always enabled for this container.

Send feedback

- Use this container over Filter Container widget for sandboxing if you need cross-filtering for widgets inside the container. Use the Filter Container with subscription to filters disabled if you need widgets affected only by predefined filters.
- If you need both cross-filtering inside the sandbox and predefined filters, use Filter Sandbox Container and add a Filter Container inside it with both publishing and subscription to filters enabled.

Filter Container

This widget enables creating a contained subset of widgets with a *pre-defined set of filters* applied only on them.

This subset of filters can be defined on any combination of the following property types: (1) list of text (string) values; (2) numeric range; and\or (3) date range.

Do not use this widget as a filter sandbox, as it was built to only support pre-defined filters.

Note: To have a filtering sandbox container (that is, to have filters apply only within a container and not be affected by external filters), use the Filter Sandbox Container instead.

Configuration

The Filter Container configuration has two parts:

Part 1 - Apply pre-configured filters on all widgets within the container

These pre-defined filters would be applied by default *only to widgets within the container*. Pre-defined filters in the container can be any combination of the following three filter types:

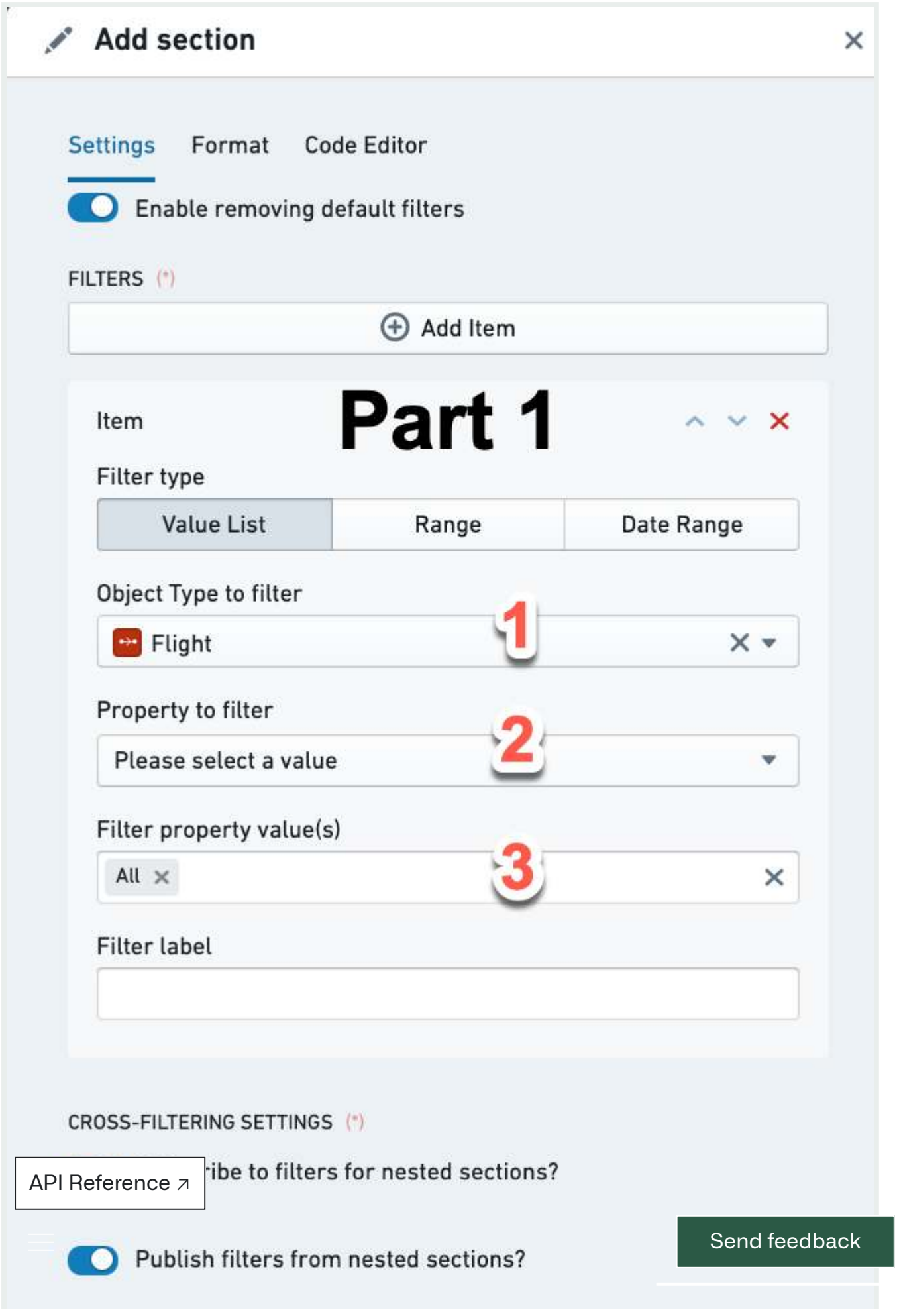
- Value List - to filter text properties (i.e. string)
- Range - to filter numerical properties (i.e. integer, double)
- Date Range - to filter down to a list of date properties

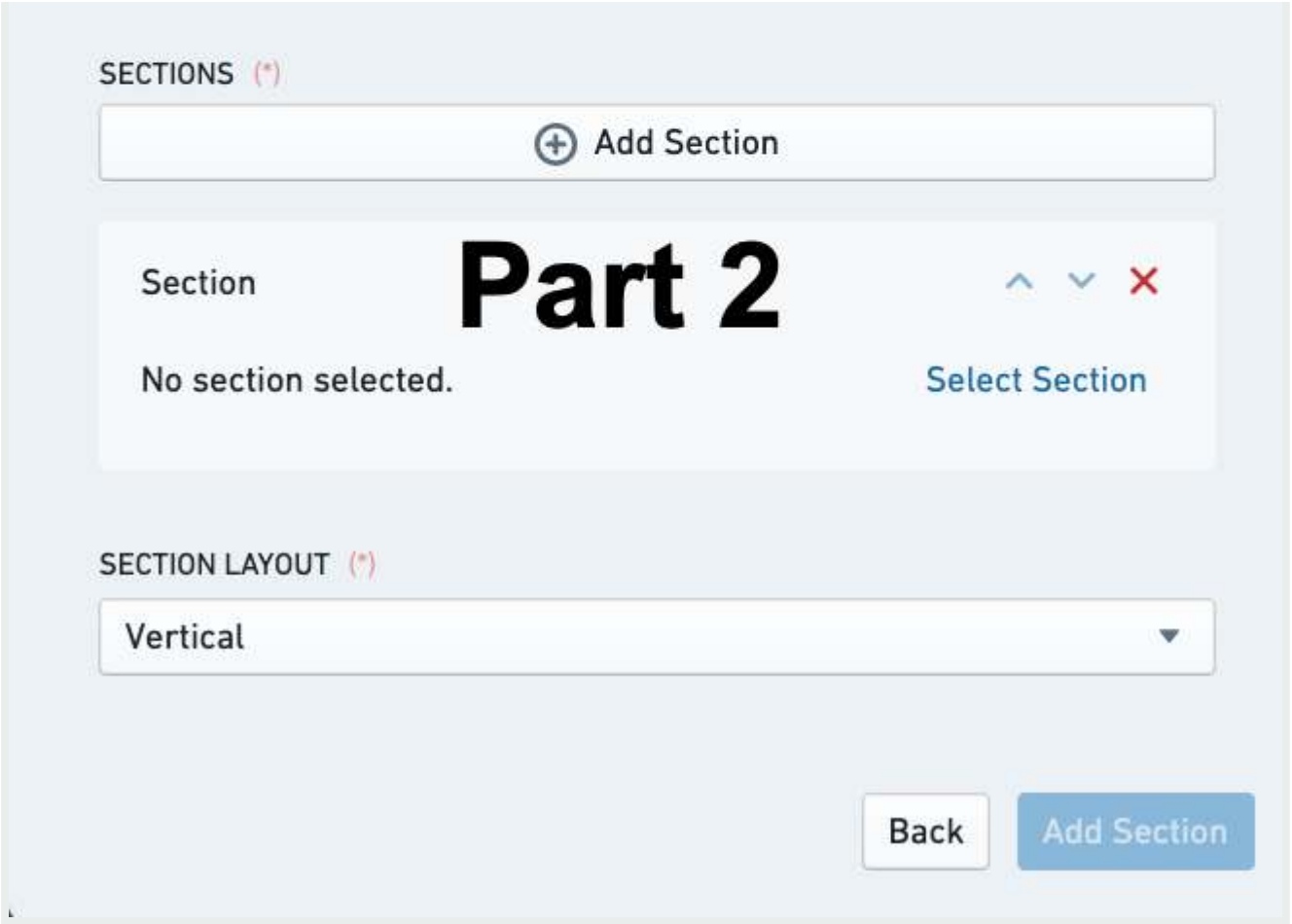
API Reference ↗

For each default filter, you need to configure the following:

Send feedback

- [Required] **Linked Object to Filter:** Select the Linked Object for this filter to apply on. The widget configuration only offers objects linked to the current object (as defined in the Ontology Manager).
- [Required] **Property to Filter:** Select the property you want to filter down by: (I) you first need to select the object type for the filter; and then (II) select the property of that object that you wish to filter by.
  - Once selected, all other widgets within this container, which are related to the same object type, would filter down from all Linked Objects to only those objects with properties value that the filter condition applies to.
- [Required] From this point, the setting is different per filter type (Text Value List, Numeric Range, Date Range):
  - **Value List:** manually type in the exact list of values you wish to filter by, with an "OR" statement between them. Note that it is case-sensitive and the order cannot be simply re-arranged. The way to re-arrange values is by deleting and re-writing the entire list of values. Note that if you apply both the "All" option as well as another specific value X, then the filter will only apply on value X.
  - **Range:** This allows you to set both the lower and upper boundary of a range of numerical values. Enter a value to at least one of them:
    - Lowerbound = greater or equal to;
    - Upperbound = lower or equal to;
    - Fill both to have a range filter.
    - Make sure to delete the 0 from the Upperbound if you wish to have a "greater than", and vice versa for "lower than".
  - **Date Range:** offers a toggle with 2 options - exact dates ("from 1\1\2019 to 31\12\2019") or relative to current time ("Last Month" or "Between two years ago and one year ago"). The configuration itself is done with a standard calendar datepicker.
- [Optional] **Filter Label:** An internal label for documentation, will not appear anywhere for the user. Optional.


Add section
Settings
Format
Code Editor
Enable removing default filters
FILTERS (*)
Add Item
Item
Part 1
Filter type
Value List
Range
Date Range
Object Type to filter
Flight
1
Property to filter
Please select a value
2
Filter property value(s)
All
3
Filter label
CROSS-FILTERING SETTINGS (*)
ribe to filters for nested sections?
API Reference ↗
Publish filters from nested sections?
Send feedback



Cross-filtering settings:

- ***Subscribe to filters for nested sections?*** If you wish the Filter Container to be affected by filters applied outside the container - both from the tab containing it, as well as other tabs sharing cross-filtering with the current tab. This would also make it consume other filter widgets within the container, e.g. Chart widget with a bar selected, or a multiselect filter.
- ***Publish filters from nested sections?*** If you wish the filters inside the Filter Container to apply outside the container - both on the tab containing it, as well as on other tabs sharing cross-filtering with the current tab. This does not apply to the predefined filters, but only to any additional filter widgets within the container.

How to use these toggles?

- Both toggles off - predefined filters apply within the container only, but it neither publishes or consumes any other filters - internal or external to the container.
- [illegible]ggle on - the container is affected by external filters, but filtering within the container does not work, unless it is the pre-defined filters.

- Only Publish toggle on - filters widgets within the container apply only outside the container, and not within the container. Only the predefined filters apply within the container.
- Both toggles on - all filters on the tab apply everywhere, except for the pre-defined filters in this container.

Common Issues and Notes:

- The pre-defined filters are only applied within the container and do not apply on any widget outside the container. This is true only for these pre-defined filters, and not for other filter widgets which could be configured within the container. Looking for a sandbox container capability? Try Filter Sandbox Container.
- The pre-defined filters in the "Filter Container" will not be visible for the user in the UI, and will not be possible to remove by users, unless your follow the following steps:
  - Add an "Active Filter" widget within the Filter Container to display it.
  - Make sure the toggle of "Enable removing default filters" is switched-on (it is on by default). Otherwise, users cannot remove these filters, even with the "Active Filter" removal option.
- Applying filters inside a filter container will also remove non-matching options in filter widgets, such as Dropdown Filter. If this behavior is not desired, use Filter Sandbox Container instead.

Part 2 - Filter Container as a Container of widgets

Configure views within this container as with any other container. Every widget you add would be subscribed to the pre-defined filters of the Filter Container, as well as any other filter defined to affect it.

Once you click on "Add Section", you get the same configuration experience as with any other "Add Section" on the main Object View or on other widgets with nested tabs (e.g. any Container widget).

Section layout:

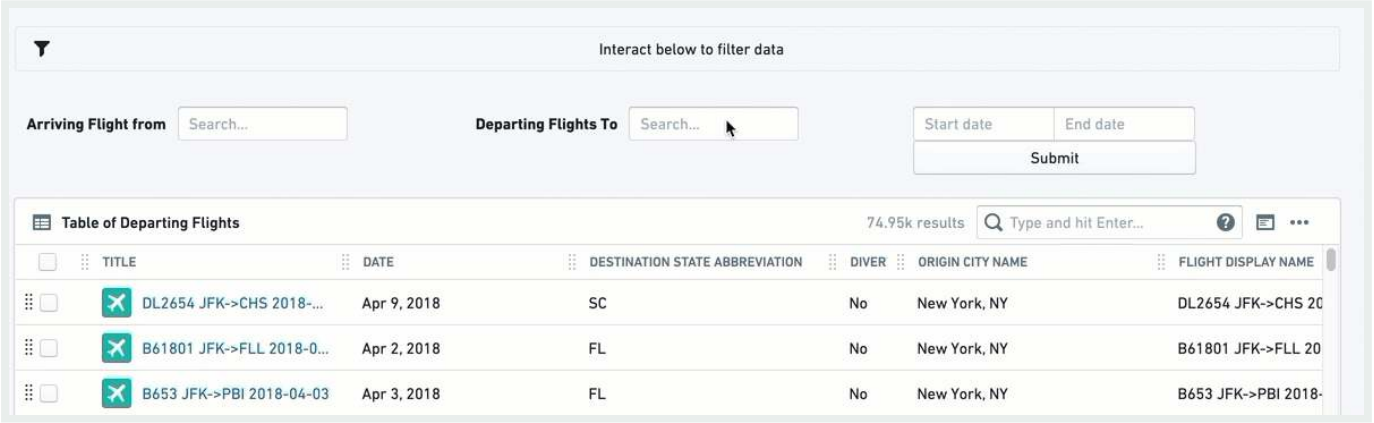
This determines whether widgets within the Filter Container would be stacked vertically (c , this is the default) or horizontally (one next to the other, from left to right).

Active Filters

This widget displays a summary of all filters that are currently applied on the Object View, and allows the user to either remove individual filters or clear all filters. There is no configuration required for this filter.

Active Filter is a useful visual indication for the user to understand which filters are currently active on their Object View. Once your Object View contains several filtering widgets (filtering using charts included), and especially if you use Filter Container or Button Filter, the visibility of active filters is important for the user experience.

Once configured, it will look like the interactive widget below, with a filter icon to the left. In this example, it is affected by the different filter widgets:



Configuration

There's no configuration for the Active Filter widget. Simply add it as a new section.

Common Issues and Notes:

- This widget will display all filters applied from all sections sharing a cross-section and cross-tabs filtering.
- Some filter widgets might not interact with this widget. Currently, it will not remove a filter applied by the Dropdown filter, and users would still need to manually change the value This makes sense when the Dropdown limits you by default to one value, but it still applies even when the "All" option is enabled.

PREVIOUS
← Visualization

NEXT
Layout →

API Reference ↗

Send feedback

Layout

Layout widgets are used to organize and structure the layout of an Object View page within a certain tab, by arranging the view into different containers.

Each Object View can have three levels of Layout control:

- Tabs, with a set of functionalities described on Configuring Tabs
- Layout Widgets within a tab (also called Containers), enabling you to organize the Object View within boxes of content.
- All content widgets (visualization, properties and links, filters, etc.), which are the building blocks of the Object View.

These are the types of Layout containers:

- Visual design of the page - Horizontal Distribution and Vertical Stack Container.
- Add another level of content nested within a tab - Tabbed Container.
- Enable displaying/hiding content according to filters that the user interacts with - Conditional Container.
- Display text, enriched with object values - Markdown widget.

Horizontal Distribution

This widget enables you to organize the layout of your Object View visually by displaying widgets distributed horizontally within a container, one next to the other. It is just a container of other widgets, and has no other functionality by itself.

API Reference ↗

Once you add the Horizontal Distribution widget, the configuration enables you to add different widgets within this container, by clicking on “Add Item”, which would open up the widget selector.

There are two options to determine how much width would be allocated to each widget within the container:

- Relative - choose an integer number for each widget within the container. Widgets are distributed according to their relative number. Example: 3 widgets of sizes A=1, B=3, C=2; Widget A would take 1/6 of the length and widget B would take 3/6 of the length.
- Pixel - choose an absolute number of pixels to allocate to each widget. If the sum of pixels exceeds Object View limits (about 1150 pixels), widgets will “spill out” of the page.

Common Issues and Notes:

- Make sure to add a title to the Horizontal Distribution widget, which is mandatory to be able to add it. The title will be displayed on the widget header.

Vertical Stack Container

This widget enables you to organize the layout of your Object View visually by displaying widgets distributed vertically within a container, stacked one on top of the other. It is just a container of other widgets, and has no stand-alone functionality by itself.

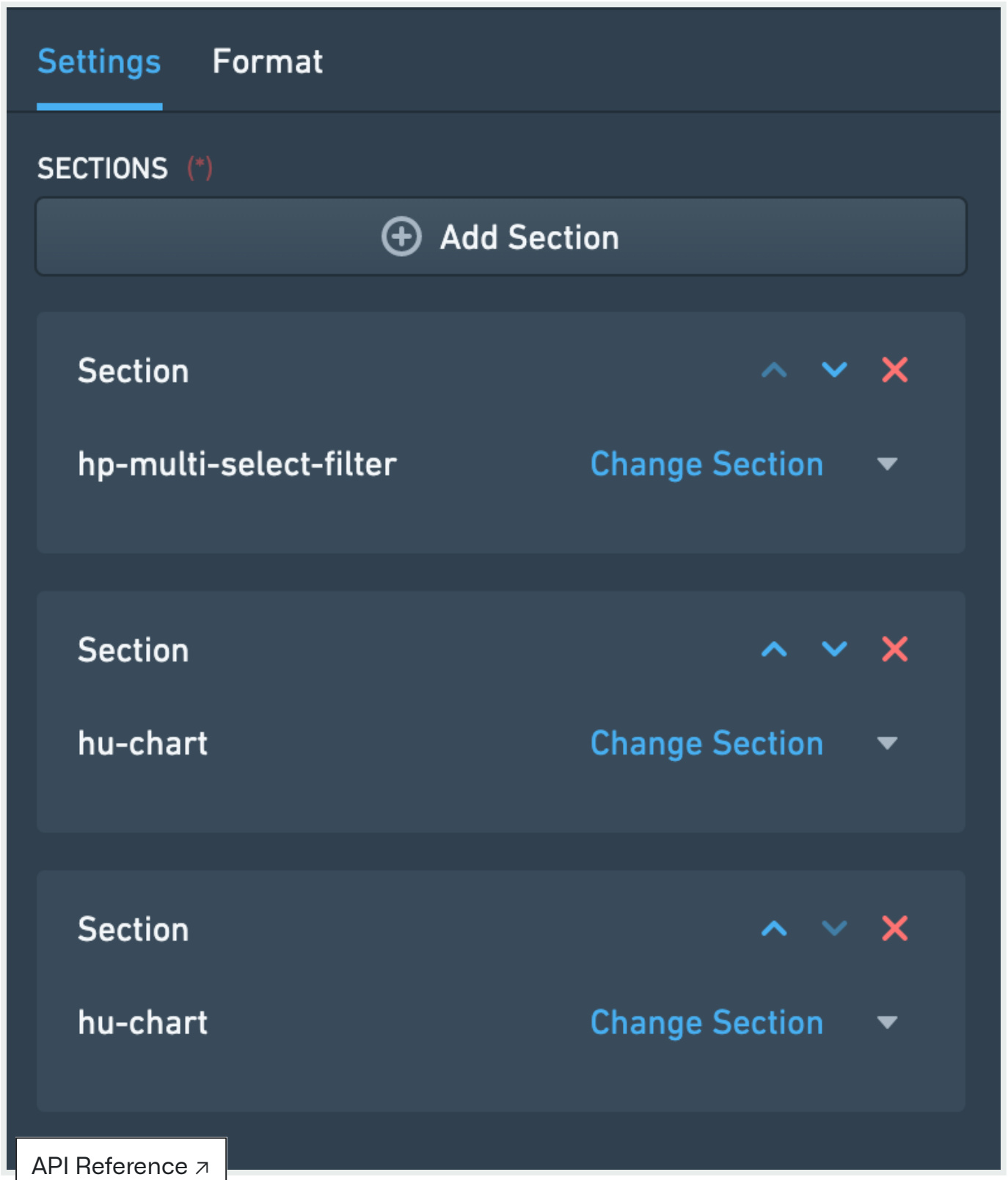
Configuration

Once you add the Vertical Stack Container widget, the configuration enables you to add different widgets within this container, by clicking on “Add Item”, which would open up the widget selector.

Common Issues and Notes:

- Make sure to add a title to the Vertical Stack Container widget, which is mandatory to be
- To align the Vertical Stack Container next to other widgets, go to “Format” tab in the configuration and select a Left/Right alignment, instead of the de

- Choose the order of the tabs by using the up/down arrows next to each tab under “Settings” in the configuration. This also allows you to delete tabs or replace the widget within each tab.



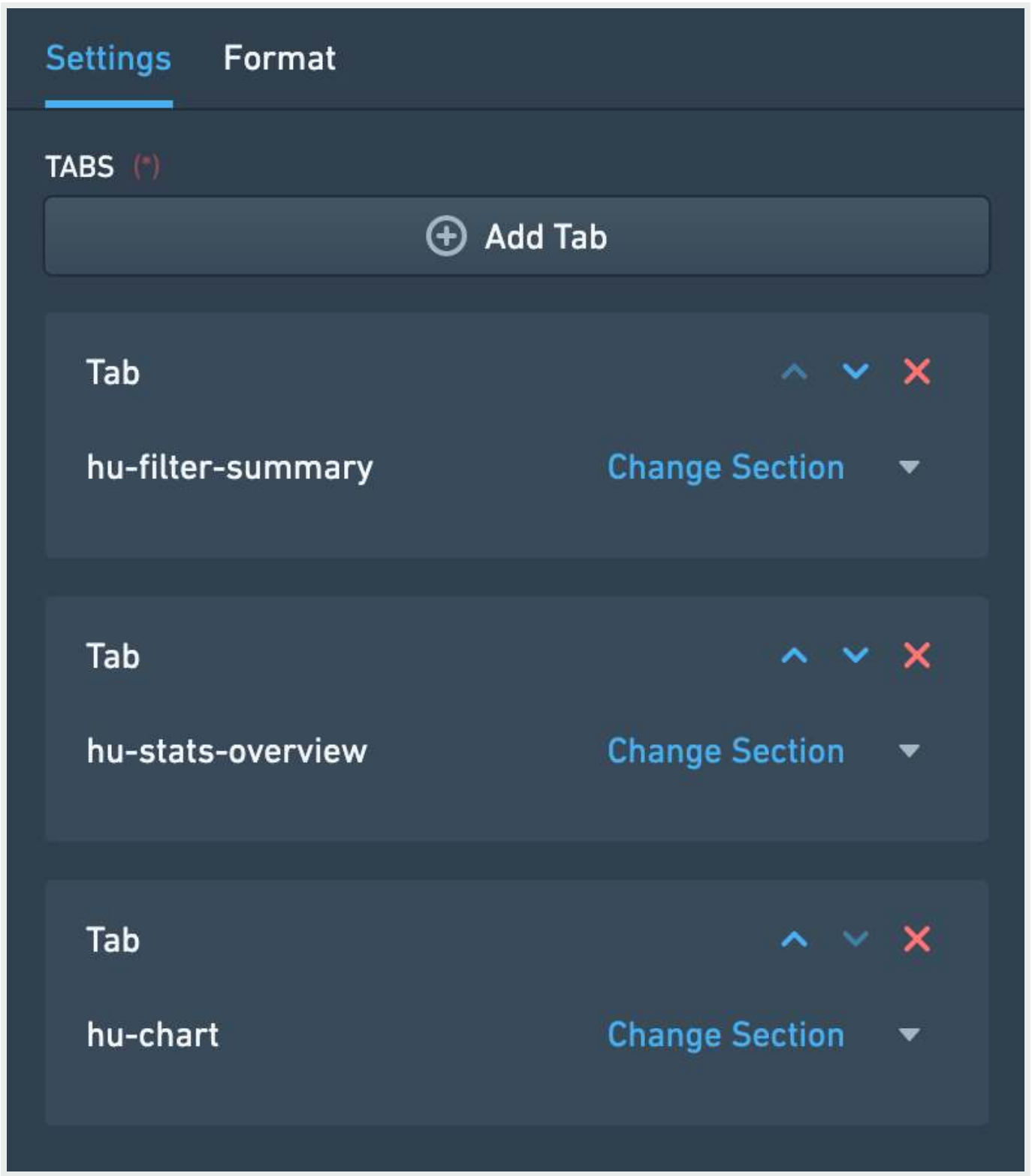
Tabbed Container

Add another layer of tabs to your Object View, by creating a container of tabs within the current tab. Users can browse between these tabs, each one containing a single widget (which can also be a container of a number of widgets).

The Tabbed Container is just a container of other widgets, and has no stand-alone functionality by itself.

Configuration

- Once you add the Tabbed Container widget, the configuration enables you to add different tabs of widgets within this container, by clicking on “Add Tab”, which would open up the widget selector.
- The name of each tab is the title of the widget on that tab, which appears under “Format” in the configuration.
- Choose the order of the tabs by using the up/down arrows next to each tab under “Settings” in the configuration. This also allows you to delete tabs or replace the widget within each tab.



Common Issues and Notes:

API Reference ↗ ometimes necessary in central objects in the ontology. However, be mindful that it adds complexity to the user experience, with “tabs

Send feedback

- Make sure to add a title to the Tabbed Container widget, which is mandatory to be able to add it. The title will be displayed on the widget header.

Conditional Container

A conditional container enables content to be displayed or hidden according to a condition. This condition can be based on:

- Filters that the user applies on the Object View
- Property values of the object or a linked object
- The existence of linked objects of a certain type

Configuration

This widget supports adding one or several *conditional sections*. Each one of these sections includes a *condition* and one or more widgets that are *conditionally displayed* according to that condition. To set up a conditional section, you follow these three steps:

Step 1 – Set the condition

The first step is to define the condition according to which the contents of the container should be displayed or hidden. There are three different types of conditions:

Condition 1 – Filters

The *Filters* condition displays or hides the contents of the container based on whether or not filters are applied to the Object View. This condition can be configured in three different ways:

- **Specific Filter** – The contents of the container are displayed only when there is a filter applied on a specific property. That property can either be a property of the current object in view or of a linked object. This means that in order to use this condition, there must be another Filter Widget which filters specifically on the same object and property condition.
  - *Example*: Configuring an Object View of an Airport, there is a fi[illegible]s that were cancelled (Status = “Cancelled”). A Conditional Con[illegible]

on cancelled flights, is configured to be displayed only when such a specific filter is applied.
    - *Note*: In order for this condition type to work, the same object type needs to be selected in the dropdowns for "Filter linked object" and "Filter property".
- **No Filter** – The container is displayed only when no filter is applied within the current tab or any tab sharing cross-filters with this tab. This can be any type of filter, e.g. a date filter, a dropdown filter, a button filter, etc.
- **Any Filter** – The container is displayed only when at least one filter is applied within the current tab or any tab sharing cross-filters with this tab. Just as for *No Filter*, this can be any type of filter, e.g. a date filter, a dropdown filter, a button filter, etc.

Condition 2 – Properties

The *Properties* condition displays or hides the content of the container based on the value of a property of an object. The object in question can either be the current object in view or a linked object. In the case of a linked object, the relationship to the current object needs to be either *1-to-1* or *many-to-1*, in which case the current object needs to be on the "many" side of the relationship.

To use this condition type, you first select the property you want to use. Next, you define when the contents of the conditional container should be shown based on that property. There are four options for this:

- **Is defined** – The value of the property is not `null`.
- **Is not defined** – The value of the property is `null`.
- **Is one of** – The value of the property matches one of the values that you define.
- **Is not one of** – The value of the property *does not* match one of the values that you define.

For **is one of** and **is not one of**, the values you define are translated to match the property type (if it's integer, double, date or boolean). See below for more details on how this property comparison is done.

Condition 3 – Linked Objects

T API Reference ↗ condition displays or hides the contents of the container based on the existence of linked objects of a certain type. To set up this condition, you first select a link path. Then, you decide if the contents of the conditional container sh Send feedback d objects of the selected path *exist* or *do not exist*.

The logic of this condition can be reasoned about by comparing it to a Linked Objects View. The contents of a container with this condition should only be shown if a Linked Objects View with the same link path would show at least one object (if linked objects should *exist*), or no objects (if linked objects should *not exist*).

Step 2 – Add Widgets

Once a condition has been defined, the second step is to configure the actual content to be displayed within the container. Click on the "Add Section" button and add as many widgets as needed. Note that you can order the widgets displayed within the Conditional Container by using the up/down arrows.

Step 3 – Choose a Layout

Finally, the third and final step is to choose the layout of the container. The layout can be either:

- **Horizontal** – Widgets are displayed from left to right, like the Horizontal Distribution Container widget
- **Vertical** – Widgets are displayed from top to bottom, like the Vertical Stack Container widget

After completing these three steps, your Conditional Container should be set up and good to go!

Common Issues and Notes:

- How is the property value comparison done?
  - String values are matched normally
  - For numeric properties, all input values in the widget are turned into numbers and compared to the numeric property (e.g. enter the string 3.1415 and it would turn to a double)
  - For boolean properties, if you use "true", "yes" and "1" in the widget, they are all considered truth input values, all the rest is false.
  - Date and timestamp values are matched after string conversion
  - ently not supported
- If several conditions are added, conditions are evaluated from top
  sections of the first condition met will be rendered, and the others

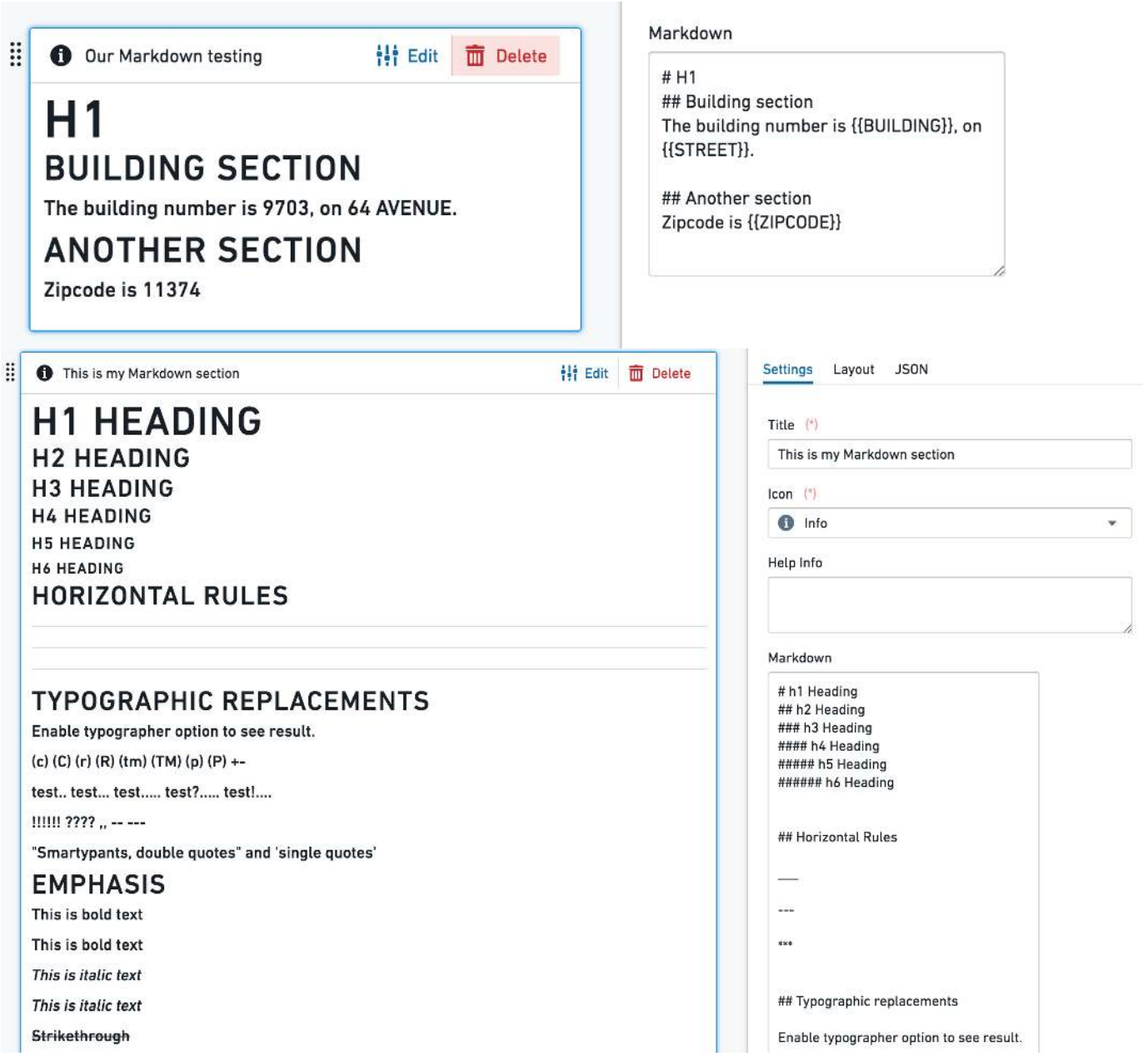
- The conditional container by property value option works on either a 1-to-1 or many-to-1 relationship, where the current object in display is on the related side (the “many”).
  - It *does not* allow conditions of many-to-many or 1-to-many where the current object is on the primary side, and it *does not* allow conditional visibility by aggregations of values in Linked objects.
  - *Example*: looking at an aircraft object, I could add a condition dependent on what airline it belongs to (assuming it belongs to only one airline), but I can’t add a condition dependent on “does it have a linked flight object with property X?”
- This widget is an extension of the tab-level optional “Visibility” configuration, but at the widget level. However, it is not a complete equivalent:
  - “Visibility” enables you to define conditional tabs according to users profiles.
  - This widget allows you to set conditional views (1) within a single tab; (2) option of conditional visibility by applied filters (dependent on user’s interaction); (3) does not include conditional view per users profiles.
- In order to enable the conditional section option to display/hide by filters applied, make sure to mark the checkbox of “cross-filtering” on the right-bar configuration editor, under “Settings”.
- The Conditional Container is only a container of other widgets, distributed either horizontally or vertically.

Markdown

This widget enables adding rich text as a part of an Object View layout. It provides a plain text editor, based on the Markdown lightweight rich text formatting syntax (`markdown-it` library). As an addition, this widget allows templating of object properties values as part of the text.

API Reference ↗

Send feedback



Configuration

- The text box is a simple text editor, and supports the standard `markdown-it` library.
- **Object properties templating** - use the {{propertyName}} format, with double curly brackets, to template your Markdown content with the current object properties values. propertyName is the exact case-sensitive name of the property in Ontology. The values {{objectId}}, {{objectTypeId}} and {{objectTypeRid}} are also supported.

Additional configurations:

API Reference ↗

- Line breaks, with 2 options:

Send feedback

    - Enable line breaks - when on (default), a single line breaks in the editor works as an actual line break; when off, line break in the editor doesn't affect the result. Using headers formats would still serve.
    - Convert "\n" to line breaks - shows `\n` as a line-break (requires the "Enable line breaks" to be on).
- Enable sanitized HTML rendering - Safe HTML rendering with markdown-it. Embedding HTML from object properties are disabled; all property values are escaped for security.

Common Issues and Notes:

- Currently, long texts and arrays included using the {{propertyName}} format might spill out of the text box and are not rendered by default.

← PREVIOUS
Filtering

NEXT
Apps and Files →

Apps and Files

Apps and Files widgets enable embedding, displaying, and linking other Foundry apps within the current Object View. These Embedded Widgets can support assets built in other apps, such as Quiver or Slate. Additionally, they enable the Object View to display media, upload files, add hyperlinks or allow comments and conversations on that object.

If you are interested in building a more sophisticated object view, consider creating a Workshop-backed tab.

Some of the widgets below are not object-aware. This means interaction with other widgets in the Object View is limited. * Example 1: Using the Comments Widget is only saved within that specific Object View, and does not write back to the actual object. If these comments might be useful outside the context of the specific object, consider using Actions to capture them. * Example 2: Adding files using “Linked Files” is saved in Foundry, but it’s not linked to the object. If you wish to have these files re-usable, consider using Actions attachments. * Example 3: Embedding Slate or Contour does allow you to pass parameters, but it doesn’t publish or consume filters and doesn’t allow cross-filtering with other widgets such as the Charts Widget.

Quiver Dashboard

For detailed steps on how to embed a Quiver dashboard in an Object View, see the Quiver dashboards documentation.

S ation

T displays a Slate application within an object view and sup and interactivity between the two applications. From the object view to the Slate application,

the current object context and the active filter state are made available. From the Slate application to the object view, a set of events are provided within Slate which map to behaviors within Object Explorer, such as opening new Object or Exploration tabs and updating the object view filters.

Configuration

Slate Resource Chose a Slate resource to display. Ensure that all users with permission to view the object can also view the application.

A "responsive" design for the Slate resources will give the best results as the application will resize based on the object view layout and the available screen dimensions.

Default Parameters By default the IDs of the current object and its object type are passed to the Slate dashboard and can be mapped to variables. These can be toggled off or have their target variables changed. The default variable names are `objectId` and `objectTypeId`. In the Slate application, ensure that the matching Variables are manually created in the Variable tab.

Custom Parameters Use additional custom parameters to pass either property values or static, pre-defined values into the Slate application. Create a matching Variable in the Slate application to capture these parameters and use them in the application.

Using Parameters in Slate

The configured parameters are passed into the Slate application when it loads. When configuring your Slate application, each parameter must have a corresponding Variable created in the Variable panel. Learn more about how to use variables. To check the keys and values of the parameters being passed from Object Explorer to Slate, you can select “View parameters” from the debugging toolbar within the object view editor.

Accessing object view filters

API Reference ↗

The object view filters are shared in the IObjectSetFilter format for easy use with the Object Set APIs available within Slate. They are automatically sent to Slate u

`postMessage` when they change and can also be requested manually using an event sent

from Slate to Object Explorer. This request event can be used to trigger sending the filters when your Slate dashboard is first ready to receive them.

To capture the filters within Slate you should configure a `slate.getMessage` event handler which takes the post message payload, parses it as JSON and then sets the result in a variable. Learn more about events. The following should be enough to capture the filters in a variable:

```
const payload = {{slEventValue}}["payload"]
return JSON.parse(payload);
```

There are two formats that the filters can be consumed in. One format contains all filters without the origin object type they may be based on, the other format contains the filters grouped by object type as well as a separate list for filters which are not based on a specific object type.

The payload for all filters has the following shape:

```
{
    "type": "HUBBLE_SLATE_WIDGET // ACTIVE_FILTERS_UPDATED",
    "payload": <IObjectSetFilter[]>
}
```

The payload for filters grouped by object type has the following shape:

```
{
    "type": "HUBBLE_SLATE_WIDGET // ACTIVE_FILTERS_BY_OBJECT_TYPE_ID_UPDAT
    "payload": {
        "filtersByObjectType": {
            [objectTypeId]: <IObjectSetFilter[]>
        },
        "globalFilters": <IObjectSetFilter[]>
    }
```

API Reference ↗

Send feedback

Triggering events from Slate

Event types available for Slate to trigger events within Object Explorer include:

- Open a new object tab in Object Explorer (using object RID)
- Open a new object tab in Object Explorer (using object primary key)
- Open a new exploration tab in Object Explorer for a given object set
- Publish object set filters to the object view
- Clear object set filters on the object view
- Refresh the data on the current object view
- Request resending the object view filters to Slate

Trigger these events using the `slate.sendMessage` action. From this action, return the event message object from the custom logic. Learn more about events. The expected format for each event is listed below. For help debugging your event integrations, use the object view editor's debugging toolbar. This will show a warning when a post message is captured but the event payload is incorrect in some way.

Open a new object tab in Object Explorer (**using object RID**) This event primarily relies on the `objectRid` parameter but can optionally take a `tabId` if the object view should be opened on a specific tab.

```
{
    "type": "HUBBLE_SLATE_WIDGET // OPEN_OBJECT_BY_RID",
    "payload": {
        "objectRid": "...",
    },
}
```

Open a new object tab in Object Explorer (**using object primary key**) If the object RID is not known then the object view can also be loaded using the object's primary key properties along with its object type ID.

API Reference ↗

```
    "type": "HUBBLE_SLATE_WIDGET // OPEN_OBJECT_BY_PRIM
    "payload": {
        "objectTypeId": "...",
```

Send feedback

```
        "primaryKey": {
            ...
        },
    },
}
```

Open a new exploration tab in Object Explorer for a given object set A new search/exploration tab can be opened by providing an Object Set. These object sets should be of limited complexity in order to avoid issues with representing them within the exploration UI.

```
{
    "type": "HUBBLE_SLATE_WIDGET // OPEN_NEW_SEARCH_FOR_OBJECT_SET",
    "payload": {
        "objectSet": {
            ...
        }
    },
}
```

Publish object set filters to the object view Many object view widgets can publish filters which effect other widgets in the view. The Slate widget can do this by sending the filters to be published using this event. The latest published filters will replace any filters previously published by the same Slate widget so the state of these filters should be managed internally within Slate. The filters should be filtered as Object Set Filters, the same format used with requests to the Object Set Service APIs. Filters can be provided for specific object types (`filtersByObjectType`) or for global properties (`globalFilters`).

```
{
    "type": "HUBBLE_SLATE_WIDGET // PUBLISH_OBJECT_SET_FILTER",
    "payload": {
        "filtersByObjectType": {
            ...
        },
        "globalFilters": {
            ...
```

```
    },
}
```

Clear object set filters on the object view The published filters present on the object view can be quickly cleared using this event. It requires no payload.

```
{
    "type": "HUBBLE_SLATE_WIDGET // CLEAR_PUBLISHED_FILTERS"
}
```

Refresh the data on the current object view If updates have been applied to the data present within the object view it may be required to trigger a refresh of the data. This event can be used to do so. It requires no payload.

```
{
    "type": "HUBBLE_SLATE_WIDGET // REFRESH_OBJECT_VIEW"
}
```

Request resending the object view filters to Slate As described above in the “Accessing object view filters” section, the filters present on the object view published by other widgets are sent to Slate using a post message. When the Slate dashboard has initialized and is ready to handle the filters is should fire this event to request them. A separate event will be sent from Hubble containing the filters. There are two types of filters you can request, one format contains all filters regardless of their origin object type, the other contains the filters grouped by their origin object type. These requests require no payload.

For all filters send:

```
{
    "type": "HUBBLE_SLATE_WIDGET // REQUEST_ACTIVE_FILTERS"
}
```

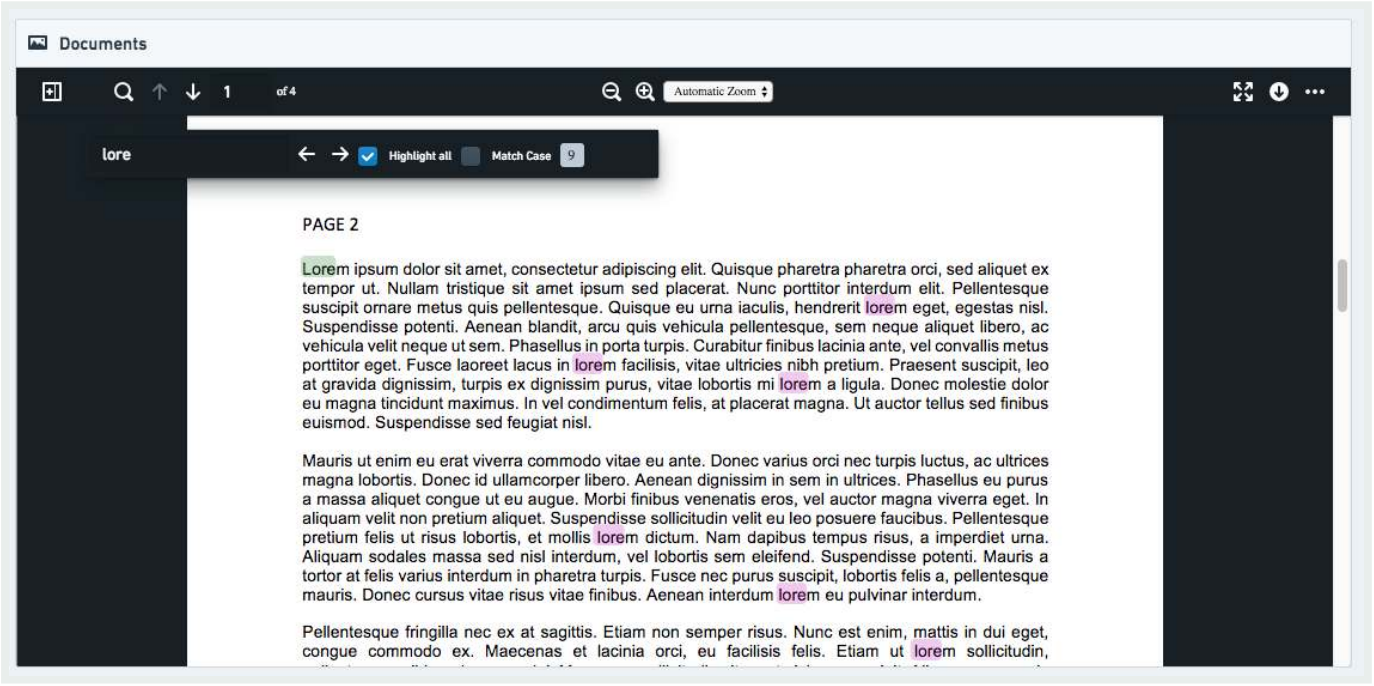
API Reference ↗

For filters grouped by object type send:

```
{
    "type": "HUBBLE_SLATE_WIDGET // REQUEST_ACTIVE_FILTERS_BY_OBJECT_TYPE"
}
```

Media Preview

This widget shows a single large inline preview of a media file, given an attachment property or a property that is a URL of some type of media (image, PDF, etc.).

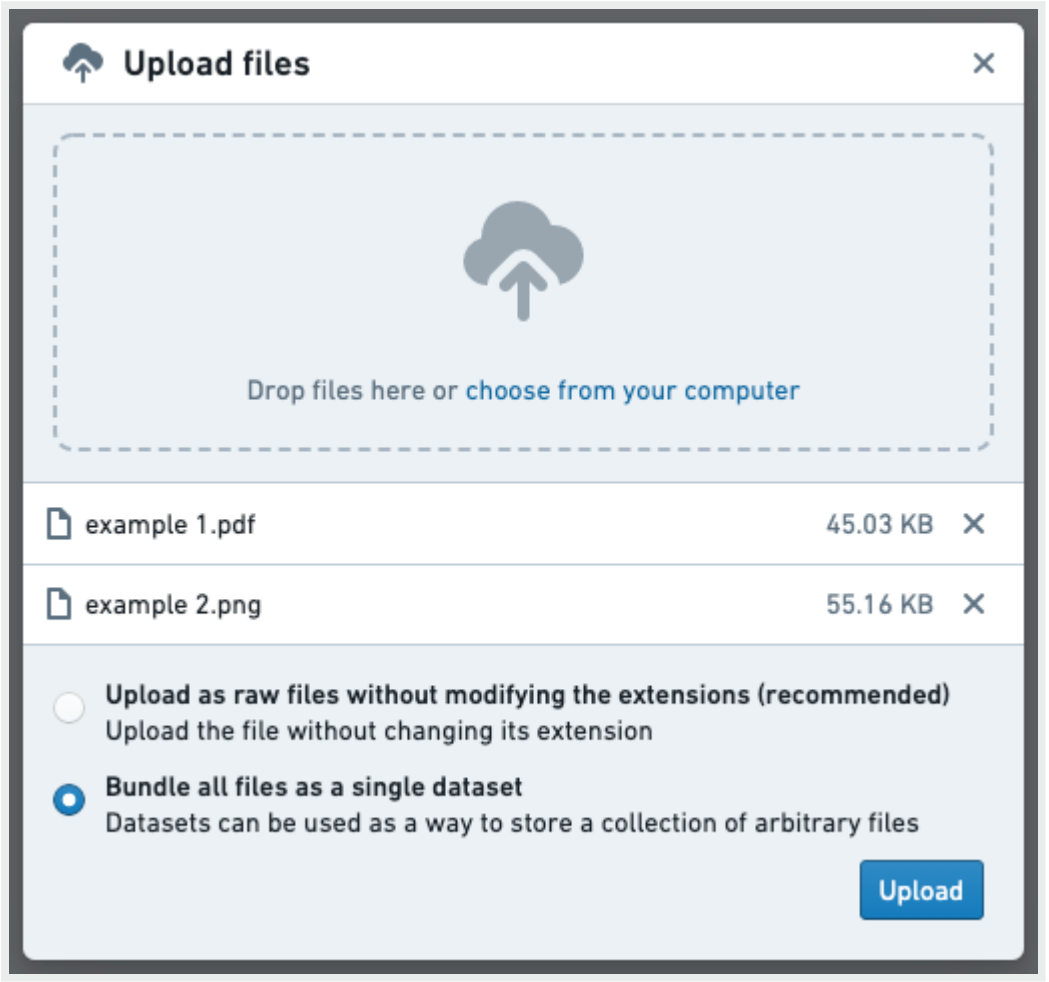


Configuration

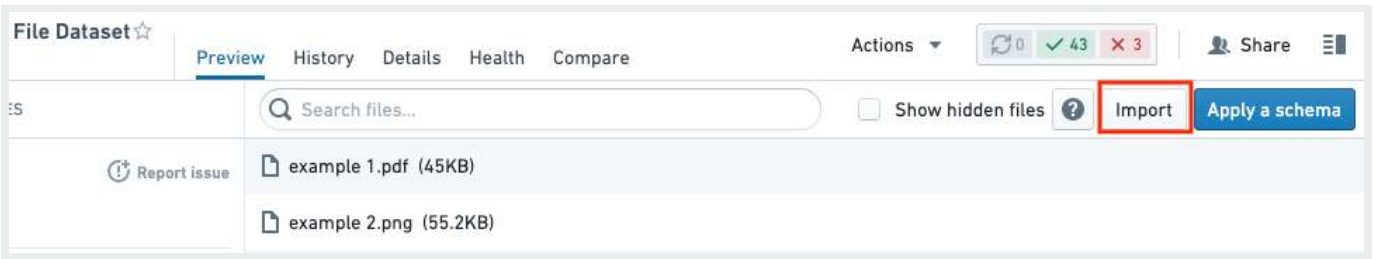
To use the Media Preview widget, you will need to configure the current object in view to include an attachment property. Attachment properties store the relevant media within Foundry and ensure that the media is correctly permissioned by inheriting the permissions from the object they have been added to. See this page to learn how to add media to your attachment property.

A[API Reference ↗]dia Preview widget can also be used to display existing media on Foundry using URLs stored in a property. Users viewing the object view will require access to both the object and the location the media is stored. To add media to follow these steps:

1. **Upload media to Foundry:** Datasets can be used as a way to store a collection of arbitrary files. To create such a dataset, you can begin by uploading the files to a folder and, in the pop-up that appears, selecting **Bundle all files as a single dataset**. If you're only uploading a single file, this option will appear as **Upload to a dataset without a schema**.



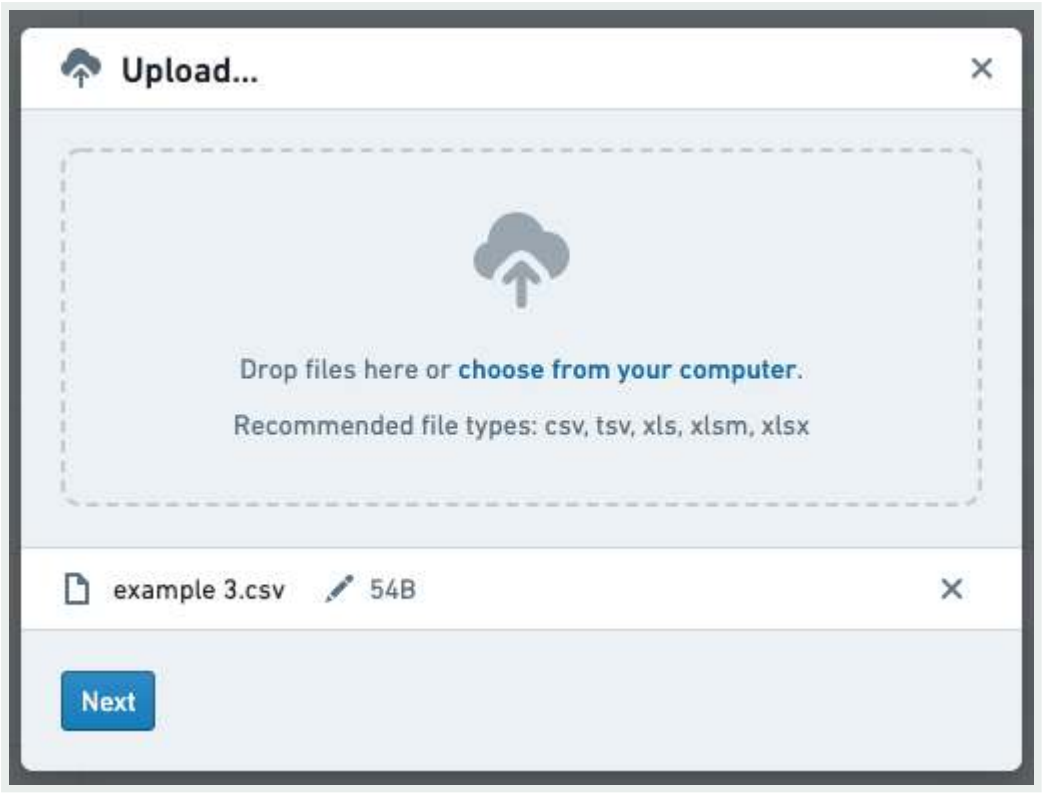
Once this dataset has been created, you can add additional files as needed. To do so, click **Import** on the top right of the dataset preview.



In the pop-up that appears, select additional files to upload.

API Reference ↗

Send feedback



2. **Add a URL column:** In the backing dataset of the current object in view, add a new column that contains the URL of the media file for each corresponding row. The URL should be of the format: `https://{my-foundry-url}/foundry-data-proxy/api/web/dataproxy/datasets/{dataset rid}/transactions/{transaction rid}/{filename}` OR `https://{my-foundry-url}/foundry-data-proxy/api/web/dataproxy/datasets/{dataset rid}/views/{branch name}/{filename}`
   - Example: `https://{my-foundry-url}/foundry-data-proxy/api/web/dataproxy/datasets/ri.foundry.main.dataset.39ce332b-1d74-40ca-be35-5a5b48459a9a/transactions/ri.foundry.main.transaction.00000000-0000-30d2-8067-4b5d9c819f4c/sample-doc.pdf`
   - Note: If you are using PDFs, the URL should start with `/foundry-data-proxy/api/dataproxy` and NOT `/foundry-data-proxy/api/web/dataproxy`
3. **Mark the column as a `hubble:media_url` in the Ontology:** Create a property for the column in the Ontology, and give it a Typeclass with kind = `hubble` and name = `media_url`.
   - Other Media Typeclasses: Other possibilities are `hubble:icon` and `hubble:thumbnail`. These will use this URL as the icon for an object or as a thumbnail in the search results cards, respectively.
4. **get to the object view in Object Explorer:** There are currently two types widgets: *Media Preview* and *Media Thumbnails*. If you edit your object view and click **Add Section** you can see a description for each typ

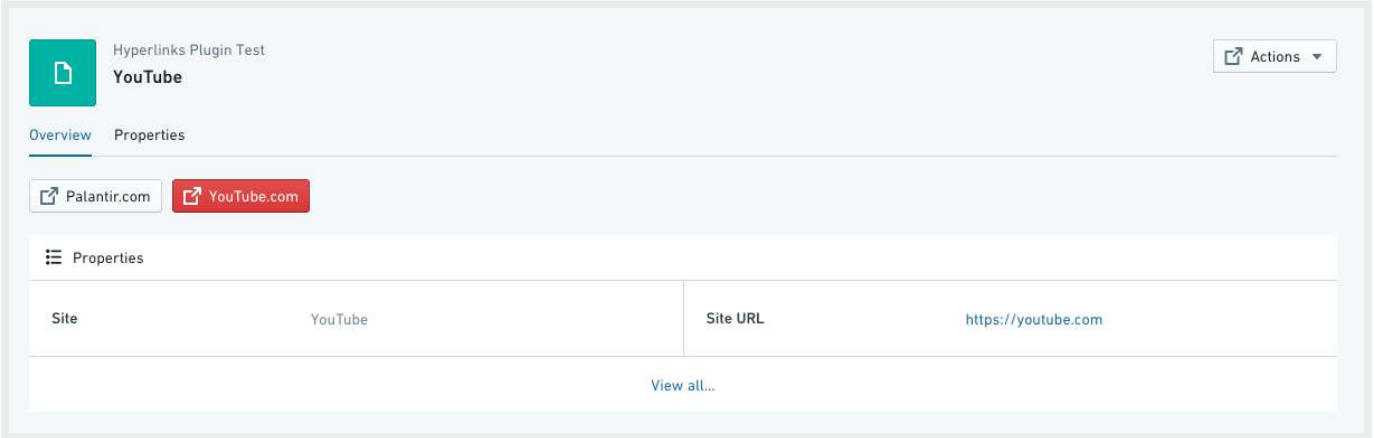
widget requires set-up of the **Media Property**, the property containing the URL for the media you wish to preview.

Other parameters to configure:

- Title (required): The title to be displayed on the widget header.
- Icon (required): The header to be displayed on the widget header.
- Help Info: Display additional help information in a tooltip.
- Height: Height (in pixels) to render the media widget.

Hyperlink

The Hyperlinks widget creates a button that works as a simple link to a webpage. You can add any number of links to an object view and each can be either static (that is, the same link for all objects instances) or dynamic per-object link.



Configuration

1. *Open Link In* - select whether to open on the same tab at the browser (default) on a different tab or in a pop-up window. Pop-ups are less reliable; may cause issues/get blocked depending on browser/installed extensions. This would apply across all objects
2. *Link Intent* - 5 link "intents" to determine the color of the link button. Blueprint Intent ↗ CSS class applied to the button (default is none):

API Reference ↗

   2. "Primary" - blue
   3. "Success" - green
   4. "Warning" - orange

5. “Danger” - red
3. *URL Type* - 3 types of URL configurations:
    1. Property - a dynamic property containing a URL to use as your hyperlink destination.
        1. To configure: (1) select the object type (usually the object currently in view); (2) select the property column that the link should be taken from. There is a toggle option to hide the hyperlink button if the value is null.
        2. For example, you could have 2 "Website" objects, both with the field `site_URL` defined in your ontology, where object 1 has the `site_URL` set to `https://palantir.com` and object 2 has the `site_URL` set to `https://palantir.com/uk`.
    2. Hardcoded - a static URL to appear across all objects. Just copy-paste the url, and make sure it has `https://`
    3. Templated - Templated URL allows you to customize the link based on a property of the object. You can bring any property in the URL by encapsulating between single curly brackets `{ }`.
        1. For example, if you have a report RID saved as a property `report_rid`, you could have a button to open the report associated with each object by using the URL template `/workspace/report/{report_rid}`.
4. *Link Title* - the text label displayed on the hyperlink button. This will be the same for all objects.

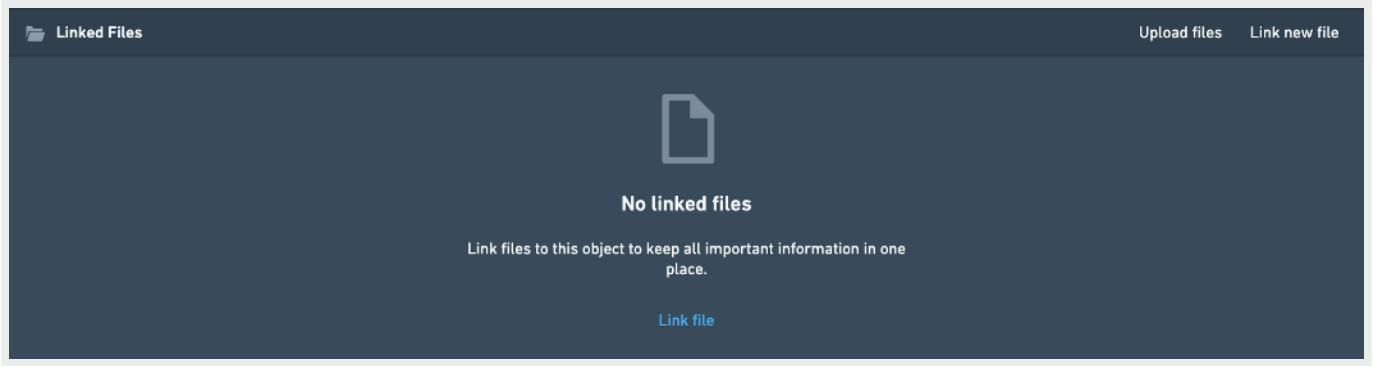
Common issues and notes:

- If the hyperlink is broken, the user will be re-directed to the landing page of Object Explorer.

Linked Files

The Linked Files widget enables users to link the object in view to a file, either by browsing with a resource selector to select a file already in Foundry, or by uploading a new file to Foundry from their local machine.

API Reference ↗



Configuration

This section has no customization options. You can still change the title and other general formatting under the Format tab.

Common issues and notes:

- Files uploaded through this widget are not written-back as a part of the ontology, i.e. they are not saved as a property on the current object. In order to achieve that, consider using Foundry Forms with writeback instead.
- There's currently no way to hide one of the two options, so it always shows both "Upload files" and "Link new file".
- There's currently no way to set up a default destination for file uploads, so the user has to browse for a destination location in Foundry every time they make an upload.

Iframe

You can embed an inline frame of a Slate dashboard or other Foundry application in the Object View as a "webpage within a webpage". Using an iframe enables you to pass values from the current object to filter variables in Slate or parameters in Contour.

Configuration

To embed an iframe, you need to configure the link to the correct Foundry address using H API Reference ↗ described below.

1. **Required:** Copy the full link to the page you wish to embed, and remove all text before `/workspace/`.
   - Report example: For `https://EXAMPLE.palantirfoundry.com/workspace/report/ri.report.main.report.ABCDEF-1234-5678`, you should keep `/workspace/report/ri.report.main.report.ABCDEF-1234-5678`.
   - Slate example: For `https://EXAMPLE.palantirfoundry.com/workspace/slate/documents/SLATE_DOCUMENT_NAME`, you should keep `/workspace/slate/documents/SLATE_DOCUMENT_NAME`.
2. **Required:** Add `embedded=true` to simply show the full view. Add a prefix of `/?` if `embedded=true` is the only statement, or `&` if `embedded=true` is attached to other statements.
   - Report example: `/workspace/report/ri.report.main.report.ABCDEF-1234-5678/?embedded=true`
   - Slate example: `/workspace/slate/documents/SLATE_DOCUMENT_NAME/latest?&embedded=true`
3. **Optional:** Pass values to Slate to filter for specific variables/parameters. Use the object type ID, a specific object ID, or a specific property ID in Object Explorer. Use `&` and state which values you wish to inject to Slate within curly brackets, like `{{propertyID}}`. This is based on Handlebar syntax.
   - Report example: `/workspace/report/ri.report.main.report.ABCDEF-1234-5678/?PARAMETER_NAME={{propertyID}}&embedded=true`
   - Slate example: `/workspace/slate/documents/SLATE_DOCUMENT_NAME/latest?VARIABLE_NAME={{propertyID}}&embedded=true`

Other configurations

- The difference between a standard iframe and a headerless iframe is that a headerless iframe hides the header of the widget; the widget header normally contains an icon, a title, and the option to add a title to the widget under the **Format** tab in the configuration.
- The iframe widget enables you to set the height of the widget. The default is 500, and it is adjusted manually.
- Once you set up the iframe, you will see a helper window underneath the widget in the object view (only displayed when in editing mode). The helper window includes object-l shows which properties and IDs you can pass on to Slate and other Foundry applications.
- Hiding the report header is possible by adding the following to the `&__rp_headerBar=hidden`.

- Embedding external websites outside of Foundry may be possible subject to security and policy requirements. Contact your Palantir representative if you think this is necessary for your use case.

Comments

This widget enables a local dialog box for users collaborating on an object, with the option to mention other users (by tagging `@user_name`).

These comments are not captured on the object itself and do not enable any future search or reuse of this conversation across Foundry.

Configuration

This section has no customization options. You can still change the title and other general formatting under the **Format** tab.

Comment behavior when the source dataset changes

If the source dataset for an object type is changed, the corresponding comment feed will disappear.

Writeback for comments

Comments added through this widget are not written back as part of the Ontology; that is, these comments are not saved as a property on the current object. If your commenting use case includes search, analysis, or learning from comments, consider using Actions.

PREVIOUS
← Layout

NEXT
Generate Object View URLs →

API Reference ↗

Send feedback

Generate Object View URLs

In the course of developing Object Views, you may need to generate URLs that link to a specific object or search for objects.

If you are embedding these views within an iframe rather than providing them as links, append a URL query parameter `embedded=true`, which will load the view without the Workspace sidebar.

> To learn how to create a URL linking to a search or Exploration, see Generating Object Explorer URLs.

Generate object links

There are two ways to link into the URL.

Option 1

```
/workspace/hubble/external/object/v0/<object-type-id>?<primary-key-property-id>=<primary-key-property-value>
```

For example:

```
/workspace/hubble/external/object/v0/aircraft?aircraftId=1234
```

O

API Reference ↗

```
/ /hubble/external/search/v2/?objectId=<objectRid>
```

Send feedback

This way is recommended when the primary key property value could possibly have special characters.

This URL loads the Object View within the context of Object Explorer. To load the Object View with no additional wrapping (for instance, to use within an iframe), create a URL like `/workspace/hubble/objects/<objectRid>`.

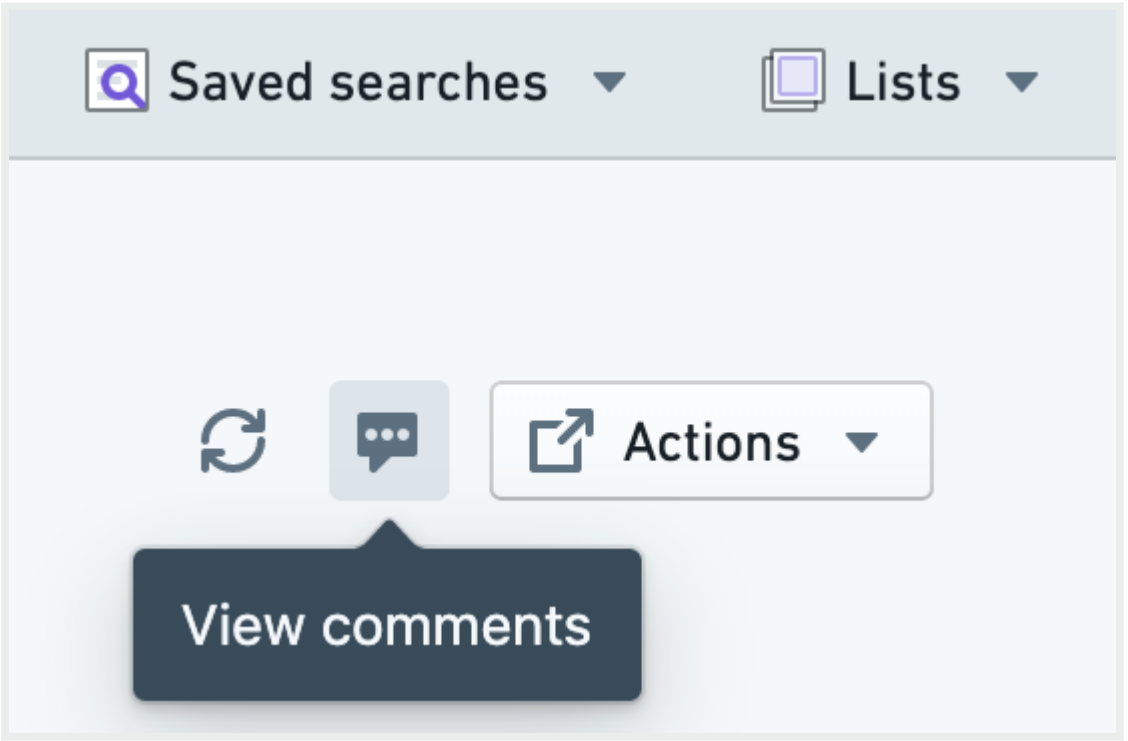
PREVIOUS
← Legacy Object Views / Apps and Files

NEXT
Comment on objects →

Comment on objects

Multiple users often work with a particular object. To facilitate this cooperation, Object Explorer allows users to comment on an object, mention other users, and attach files and images.

You can open the Comments Helper for any object using the **View comments** button in the header of any Object View.



Object Explorer comments on an object are *not* related to the Comment widget in Workshop.

API Reference ↗

Add Object Views to a Marketplace product

Use Foundry DevOps to include your Object Views in Marketplace products for other users to install and reuse. Learn how to create your first product.

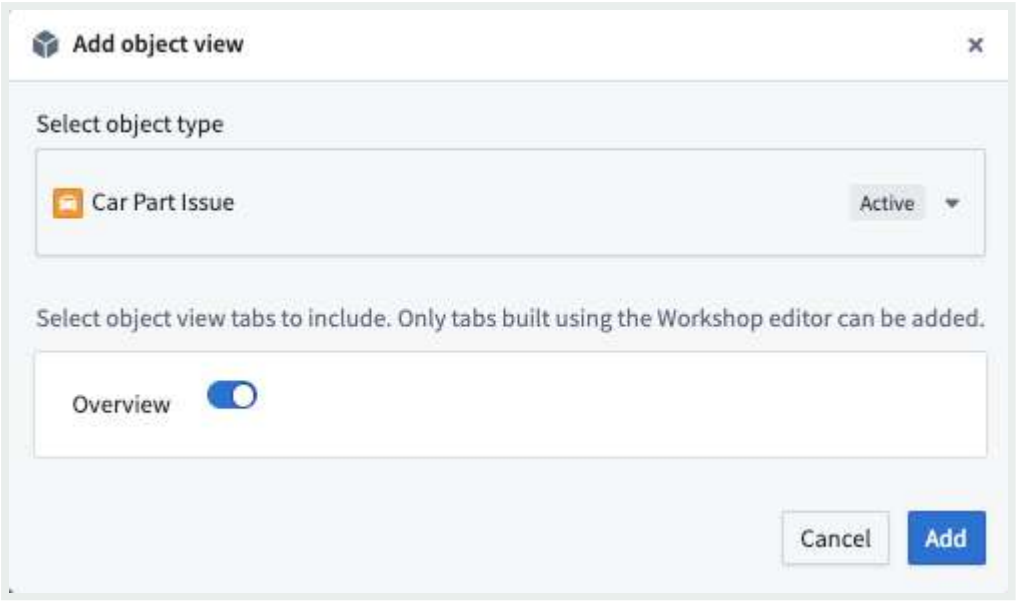
Supported features

Marketplace products only support Object View tabs that use the Workshop tab builder. The legacy Object View builder is not supported. If you would like to package an Object View tab that leverages the legacy builder, you should first rebuild the tab with the Workshop tab builder.

Add Object Views to products

To add an Object View to a product, first create a product. Add outputs and then select the **Add ontology entities** option.

Once you have selected an Object View, you can select which tabs you would like to include in your product.



PREVIOUS
← Comment on objects

NEXT
Ontology Manager / Overview →